

→ Using web frameworks we can just develop web applications.

Using ORM frameworks we can just develop persistence logics.

Using Spring we can develop all kinds of logics like Business Logic, Persistence Logic, Integration Logic, presentation Logic and etc.

→ Using Spring we can develop all kinds of applications like standalone apps, two-tier applications, web applications, Distributed applications, enterprise apps & etc

Spring

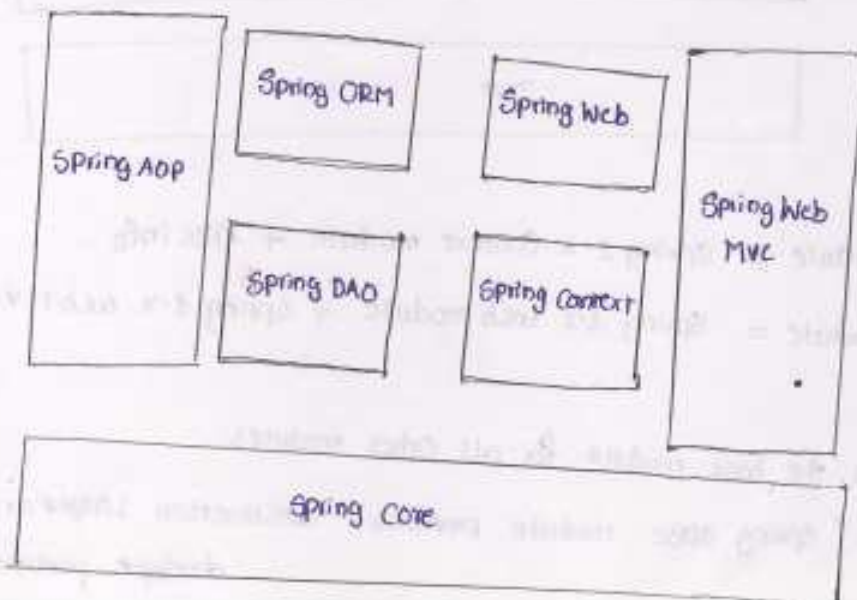
- Type: java-JEE Framework / Application Framework
- Vendor: Interface 21
- Creator: Mr. Rod Johnson
- OpenSource glw
- To Download glw: Download as Zipfile from www.springframework.org
- Version: 2.5, 3.x
 - 2.5 (compatible with JDK 1.6+)
 - 3.x (compatible with JDK 1.5+)
- For docs and FAQs: www.springframework.org
- For online Tutorial: www.roseindia.net
- For good articles: www.java4s.com & www.mkyong.com & www.nataraj.in

Books: Spring in Action from Manning publisher

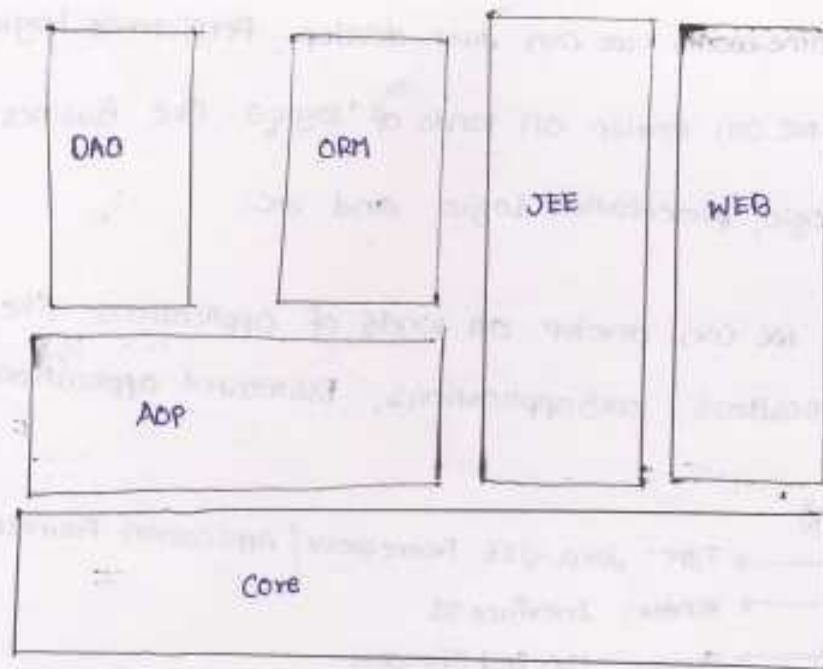
→ Spring 1.x contains 7 modules. where as spring 2.x contains 6 modules.

Spring 3.x contains 20 modules but they are grouped into 6 major categories

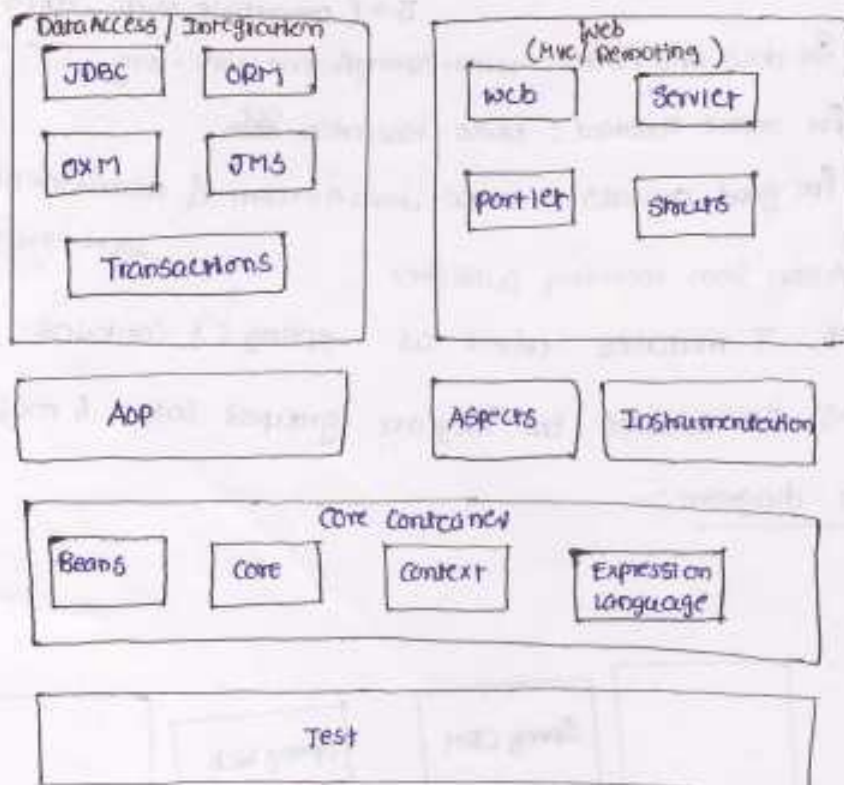
→ Spring 1.x overview diagram:-



Spring 2.x Over view Diagram :-



Spring 3.x Over view Diagram :-



Spring 2.x JEE module = Spring 1.x Context module + Misc info ...

Spring 2.x web module = Spring 1.x web module + Spring 1.x web MVC module

→ Spring core is the base module for all other modules

→ Spring DAO / Spring JDBC module provides abstraction layer on plain JDBC to develop persistence logic

→ Spring ORM module provides abstraction layer on other ORM Flows and allows to develop O-R mapping persistence Logic. like Hibernate

→ Spring AOP module is given to apply Aspects (Middleware services) on Spring Apps.

→ Spring web module is given to make spring applications communicable from other web framework like applications. like struts app, JSF apps etc.

→ Spring web-mvc is given spring's own web framework develop MVC2 architecture based web application.

→ Spring context module / JEE module provides abstraction layers on multiple core technologies like JNDI, EJB, RMI, JMS, JavaMail and etc.... develop enterprise apps, Distributed apps and other applications.

Middleware services are additional, optional and configurable services/logics on our applications to make them more perfect and accurate.

Eg:- security, Transaction mgmt, Logging

To install spring 2.5

→ <Spring 2.5_home> \dist\ spring.jar represents the whole spring api and its dependent jar file
is <Spring 2.5_home> \lib\ jakarta-commons\commons-logging.jar

→ The classes and interfaces of spring.jar uses the classes and interfaces of Commons-logging.jar

→ <Spring 2.5_home> \changelog.rtf use details about various versions of Spring s/w.

Spring definition:-

→ Spring is a Java based open source, light-weight, loosely coupled, aspect oriented and dependency injection based application \ Java-JEE framework to develop all kinds of applications.

→ Opensource sluo means It is not only free sluo but its source code can be exposed to programmers along with installation.

<Spring home>/src folder represents source code of Spring sluo.

→ Spring is light-weight sluo and Spring Applications are light-weight apps. reasons,

① Spring Containers are light-weight Containers. Because they can be activated any where by just creating objects for certain pre-defined classes.

NOTE:- Servlet Container, Jsp container are heavyweight container.

② most of the Spring apps can be executed without the heavyweight application server and webserver sluo's.

③ resources of Spring application can be developed without using Spring API.

④ while executing Spring applications much memory and cpu resources are not required

⑤ Spring Technology is easy to learn and use

→ If the Degree of dependency is less b/w two components then we can say the components are loosely coupled.

If the Degree of dependency is more b/w two components then the components are tightly-coupled components.

Spring is loosely coupled sluo. reasons,

① ~~spring sluo~~ while working with Spring we can use either individual modules or ~~use~~ multiple modules to gather.

② we can integrate Spring with other ~~the~~ Java Technologies in application

development. like, servlets, JSP, Struts, Hibernate and etc...

⇒ ③ Every Spring module provides abstraction layer on providing multiple core technologies and makes programmer free from core technologies based programming while developing applications.

→ It is always recommended to develop ^{large scale} ~~spring~~ applications by enabling middleware services like security, Transaction mgmt and etc...

Spring provides a new methodology called Spring AOP to apply these middleware services on Spring applications.

This makes the Spring as Aspect oriented.

→ What is the difference b/w dependency lookup and dependency Injection?

→ dependency lookup:- Resource explicitly ^{searches and} ~~dependency~~ gathers dependent values

from others. (like other resources)

→ in dependency lookup resource pulls the values from others

Eg ①:- if student gets course material only after passing request for it. It is called ^{then} ~~it is called~~
(resource) (like dependent value)

Dependency lookup.

Eg ②:- The way we collect DataSource object reference from JNDI registry is called

Dependency lookup.

Advantage:- resource can gather only required dependent values.

Disadvantage:- resource should spend some time to gather its dependent values before utilizing them.

→ Dependency Injection:- The underlying server (or) Container (or) P/lo (or) Runtime (or) special resource assigns values to the resource automatically and dynamically.

→ In Dependency Injection (also called as IOC (inversion of control)) the underlying Container (or) Server (or) Runtime (or) Flow and etc. pushes the values to the resource.

Eg(1):- If course material is assigned to student the moment student registers for a course is called Dependency Injection.

Eg(2):- The way ActionServlet writes formdata to FormBean class properties

Eg(3):- The way JVM calls constructor automatically to assign ~~flor data~~ Initial values to object.

Eg(4):- The way Servlet Container calls lifecycle method Service (-) to expose request, response objects to our servlet program.

Advantages:-

resource can use the injected dependent values directly without spending time to get them.

Disadvantage:-

The underlying environment like Server/Framework may inject both necessary & unnecessary values.

- Spring supports both dependency lookup & dependency Injection. Spring Containers are design to perform dependency Injection on the resources of Spring application.
- A File (or) program of an application is called resource. (generally it is class & interface)

All major java projects are based on MVC2 / MVC Architecture

M → Model Layer → (B-logic + persistence logic) (like Accounts Officer)
V → View Layer → presentation logic (like Beautician)
C → Controller Layer → Integration Logic (connecting Logic)

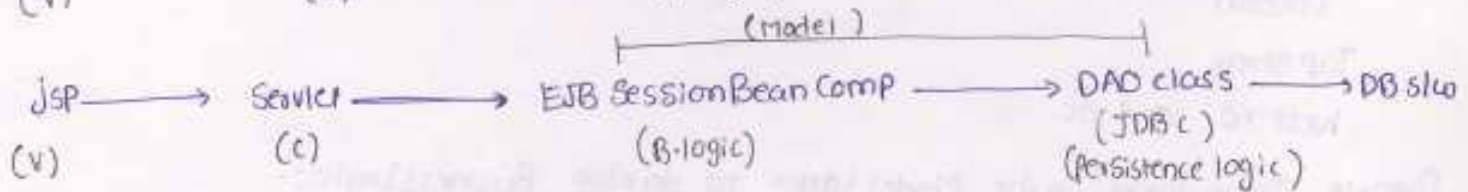
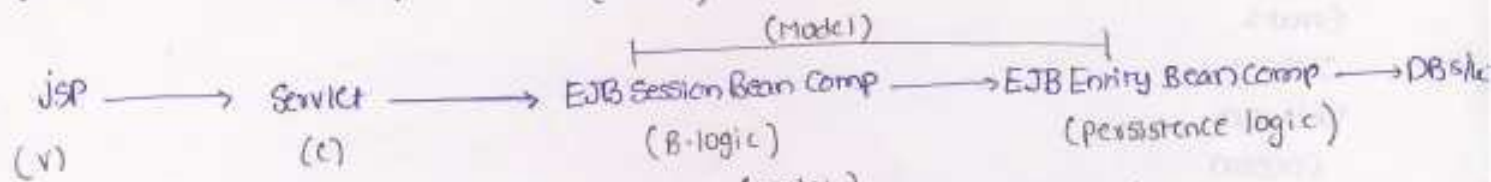
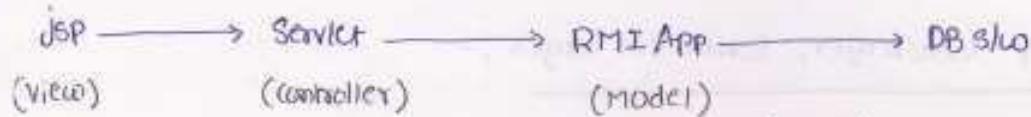
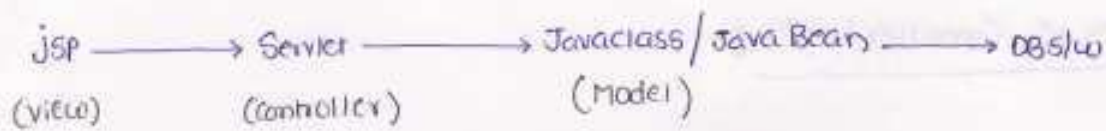
→ The main logic of the application is called Business Logic

→ persistence logic is useful perform CRUD operations on DataBase gluo's

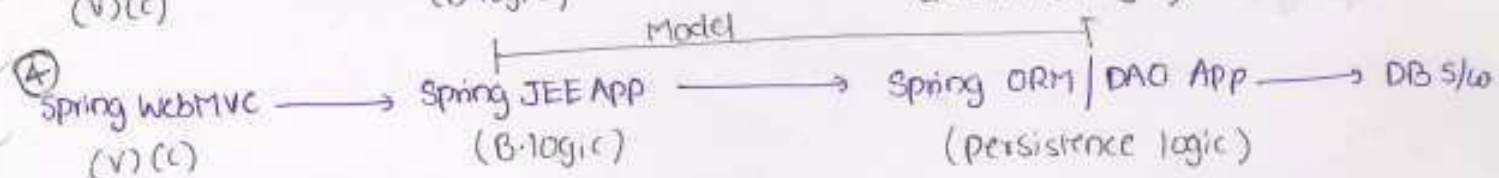
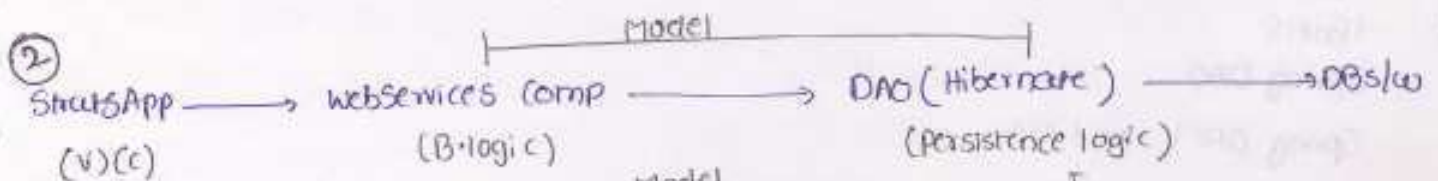
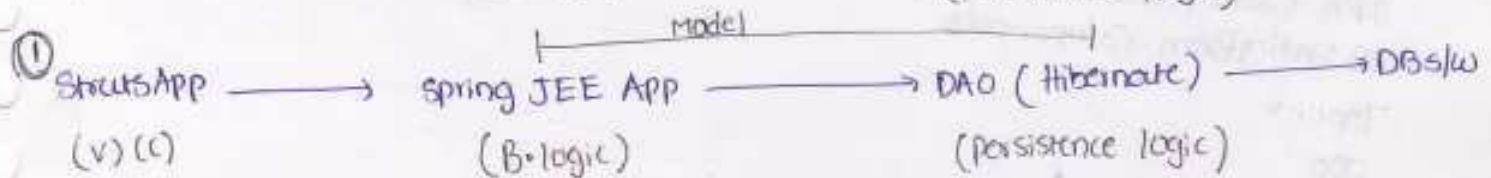
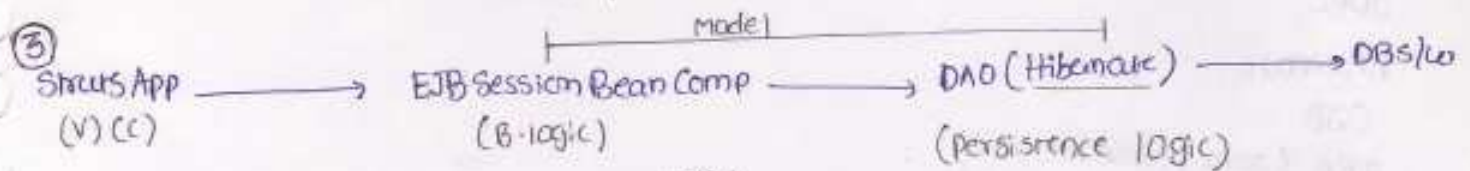
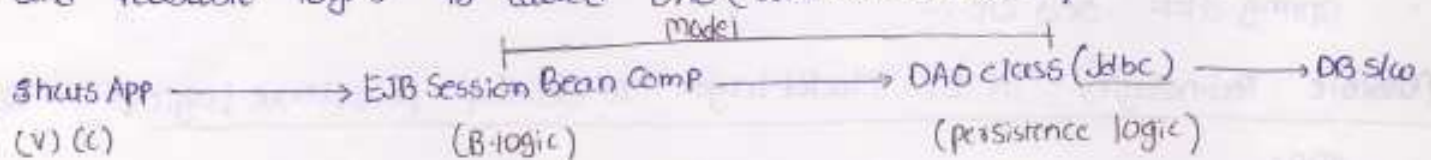
C → create (Insert) U → update R → Read (select) D → Delete (remove)

→ presentation Logic gives user interface for enduser, to supply to inputs to application and to view results of the application

→ Integration logic Controls and monitors all the operations of the application.



"The Java class that separates persistence logic from other logics of the application to make the persistence logic as the flexible logic to modify and reusable logics is called DAO (Data Access Object)"



The possible technologies of view Layer

JSP

velocity

Free marker

html

XSLT (xml Stylesheet Language Transformation)

Jasper Reports / i Reports

Possible Technologies of Controller Layer :-

Servlet
ServletFilter

Possible Frameworks of view layer, controller layer :-

Struts
JSF
Webwork
Cocoon
Taglib
WebMVC and etc....

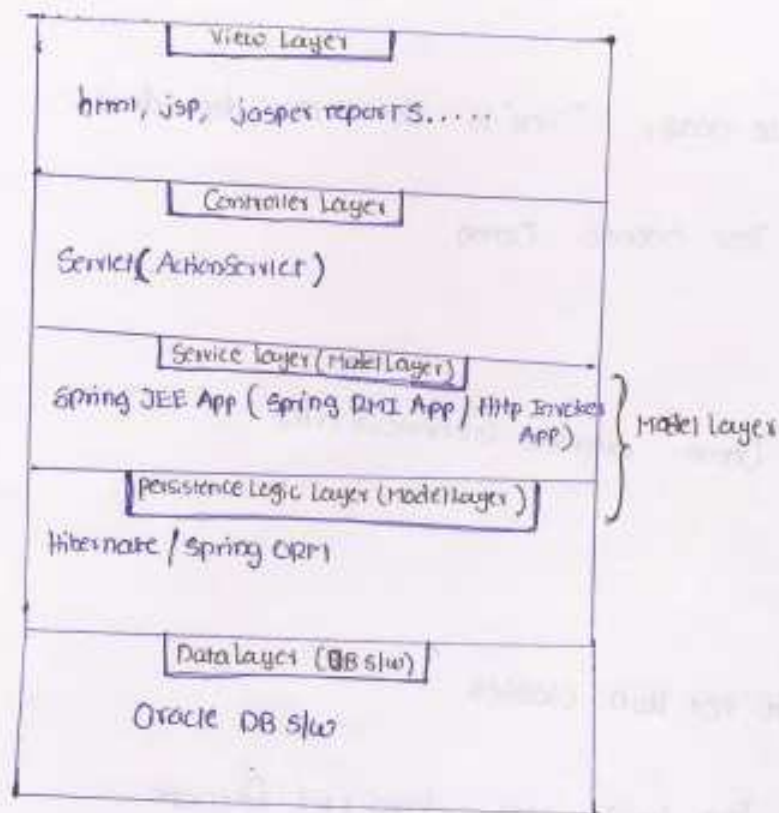
Possible Technologies in the Model layer to develop Business Logic :-

EJBSession Bean, Webservices, java class | Java Bean class
RMI, HTTPInvoker (spring's own Distributed Technology), CORBA,
Spring JEE and etc....

Possible Technologies in the Model layer to develop persistence Logic :-

JDBC,
hibernate
CJB
JPA (Java Persistence API)
EJB Entity Bean Components
TopLink
JDO
iBatis
Spring DAO
Spring ORM and etc....

High Level project Architecture of Struts-Spring-Hibernate Integration



→ All latest Technologies of java and their latest versions (released After 2008) are Supporting POJO/POJI model programming to make the technologies and applications as light-weight.

POJO class :- if a java class that is acting as resource of certain Technology based software application is not extending, implementing predefined classes, Interfaces of that technology specific API then it is called POJO class.

Eg:- if java class is taken as resource of spring application without extending, implementing the predefined class, interfaces of Spring API then it is called POJO class.

Examples:-

①

```
public class Test {  
    //  
}
```

 "Test" is POJO class.

②

```
public class Test extends Demo {  
    //  
}
```

 "Demo" is not extending from any other class.
Test is POJO class.

Eg3:- public class Test extends HttpServlet

{

==

}

"Test" is not POJO class. "Test" is Servlet api dependent.

④ public class Test extends Demo

{

==

}

public class Demo extends GenericServlet

{

==

}

"Test", "Demo" are not POJO classes

⑤ public class Test implements java.rmi.Remote

{

==

}

"Test" is not POJO. bec Remote is a Technology

⑥ public class Test implements java.sql.Driver

{

==

}

"java.sql.Driver" (I) is part of jdbc technology. so "Test" is not POJO class.

⑦ public class Test implements java.lang.Runnable

{

==

}

"java.lang.Runnable" is the ^{part of} core java. so, Test is POJO

⑧ public class Test extends Thread

{

==

}

"Test" is pojo class bec java.lang.Thread is part of basic java apis


```

⑨ public class Test
{
    public void m1()
    {
        // uses Spring API
    }
}

```

Test is POJO class.

Spring supports POJO / POJI model programming due to this Spring apps are light weight apps

POJI:-

When Java interface is taken as the resource of certain technology based application, and if that interface is not extending from predefined interface of the technology specific API's then it is called POJI.

Examples:-

```

① interface Test
{
    // method declarations
}

```

"Test" is POJI

```

② interface Test extends Demo
{
}

```

"Demo" is user-defined interface not extending other interfaces
So, "Test" is POJI

```

③ interface Test extends java.rmi.Remote
{
}

```

"Test" is not a POJI. (because java.rmi.Remote is RMI api interface)

```

④ interface Test extends java.io.Serializable
{
}

```

"Test" is POJI (because java.io.Serializable interface is very much part of ^{basic} Java API's).

Features of Spring:-

- ① Supports POJO/POJI model programming.
 - ② Spring is light-weight software. (Most of the Spring applications can be executed without the heavy weight ~~of~~ ^{webServer / Application Server})
 - ③ Spring gives built-in middleware services and allows to work with server managed and third party supplied middleware services.
 - ④ Spring gives new methodologies called AOP/Aspect to apply middleware services....
 - ⑤ Supports both Dependency injection and Dependency lookup.
 - ⑥ provides abstraction Layer on multiple core Technologies and simplifies the application development.
 - ⑦ provides environment to perform Unit Testing on Applications.
- NOTE:- programmers testing on his own piece of code is called Unit Testing.
- ⑧ provides Built-in plugins to make Spring applications integratable with other Java technology based apps.
 - ⑨ Allows to develop all kinds of applications like standalone, two-tier, three-tier, n-tier distributed applications, web applications and etc...
 - ⑩ Allows to develop all kinds of logics like presentation logic, Business logic, ^{Integration} persistence logic and etc...
 - ⑪ Gives its own webFramework Spring WebMVC to develop MVC2 architecture based web apps.
 - ⑫ provides abstraction Layer on other frameworks like Hibernate, J2B and etc...
 - ⑬ ~~allows~~ ^{supports} Integration with ^{Java/}J2EE technologies and Java based Frameworks.
 - ⑭ Easy to learn and use.

Module-I

Spring Core Module

- Base module for other module of Spring.
- Gives a light-weight SpringContainer called ~~BeanContainer~~ "BeanFactory".
- Using this module we can develop only stand alone applications.
- We can use this module to understand SpringBean lifecycle management and dependency Injection.

The Concrete Java class that is configured in Spring configuration file and through annotations is called SpringBean. JavaBean can act as SpringBean but every SpringBean need not to be a JavaBean.

- Spring gives two built-in container and lightweight containers

① BeanFactory Container (part of core module)

② ApplicationContext Container (part of spring JEE/Context module)

- Container is a Software Appn (or) Java class that can take care of the whole life
_{↳ like aquarium} _{↳ Birth to death}

Cycle of given resource.

_{↳ like fish}

- Spring Containers are given perform SpringBean Lifecycle management and to perform Dependency Injection on SpringBean.

- To activate BeanFactory container in our applications create object for java class that implements org.springframework.beans.factory.BeanFactory(I)

Xm1BeanFactory _{↳ class implementing BeanFactory} _{↳ represents BeanFactory container} factory = new Xm1BeanFactory("obj pointing to spring (cfg file)");

- To activate ApplicationContext container in our Apps create obj for java class that implements org.springframework.context.ApplicationContext(I)

Eq:-

ctx = new FileS

* ApplicationContext +
Container

* ApplicationContext +
Container

→ We can't activate ServiceContainers, Supercal, etc. because we need to start certain classes. So, they are heavy weight containers. Bcz we need to start heavy weight servers to activate these containers.

But Spring Containers are light-weight Containers

→ ApplicationContext container is enhancement of BeanFactory container.

Spring CoreModule Application contains the following resources.

- 1) Spring Interface (Java Interface)
- 2) Spring Bean class (Java class)
- 3) Spring Cfg file (xml file)
- 4) Client Application (Java class with main method)

Spring Interface

- it can be a POJI
- it is optional resource. But recommended to take
- contains declaration of B-methods/utility methods
- acts as a common understanding document between client application (Spring Boot)
- it like course brochure having declaration of topics (which acts as agreement document b/w faculty and student).

Spring Bean

- A concrete Java class. configured in Spring Configuration file
- Can be a JavaBean
- implements Spring Interface and provides implementation to Spring interface methods.
- Contains methods, constructors supporting Dependency Injections.

- Contains utility methods
- Contains lifecycle methods.
- It can be POJO class (or) non-POJO class.
- It can be predefined / userdefined / Third party supplied concrete Java class / JavaBean.
- Contains Business methods implementation having Business Logic.

Note:- All Spring Beans must be configured in Spring Configuration file. either by using ~~xml~~ ^{xml} entries (or) ~~xml~~ ^{xml} annotation

- Specifying the details of resource through xml entries (or) Annotations to make underlying container (or) server (or) Frameworks recognizing that resource is called resource Configuration

14/02/2013

Spring Configuration File :-

- any <filename>.xml can be taken as spring cfg file.
- We can one or more spring cfg files in our spring apps
- Spring beans must be cfg in spring cfg file.
- Contains Dependency Injection / IOC related instructions
- Contains lifecycle methods configuration
- Can be developed either based on DTD / schema rules.

Client Application :-

- Activate Spring Container by specifying spring cfg file
- Get Spring Bean class obj from Spring Container
- call Business methods on Spring Bean class object.

→ Spring Supports 3 types of Dependency Injection

1) Setter Injection

(by using `setXxx(-)` of Spring Bean class)

2) Constructor Injection

(by using parameterized constructors of Spring Bean class)

3) Interface Injection

(by implementing special interfaces on Spring Bean class)

→ ① Setter Injection:-

If spring container calls `setXxx(-)` methods automatically and dynamically to assign values to the properties (member variables) of Spring Bean class. Then it is called setter injection.

Example Scenario

// TestBean.java (Spring Bean)

```
public class TestBean
```

```
{
```

```
    // Bean property
```

```
    String msg;
```

```
    // Write setXxx(-) supporting setter Injection
```

```
    public void setMsg(String msg)
```

```
    {
```

```
        this.msg = msg;
```

```
    }
```

```
    // Write B-methods
```

```
    public void brn1()
```

```
    {
```

```
        ....
```

```
        .... // use msg here
```

```
    }
```

```
}
```

Demo.xml (Spring cfg file)

<beans>

① msg property configuration for setter Injection


```

<bean id="tb" class="TestBean">
    <property name="msg" value="hello"/>
</bean>
</beans>

```

id/object name Bean class
 Bean property Value to be injected

Spring Bean configuration

When Spring Container is activated by giving this Spring cfg file the Spring Container creates TestBean class object having "tb" as the Object name and calls tb.setMsg("hello") to inject the value "hello" to the bean property "msg" through setter Injection.

→ Spring Container manages the whole lifecycle of SpringBean in that process it performs dependency Injection on SpringBean class properties the moment it creates SpringBean class object.

→ In real time projects we make Spring Container injecting special objects to SpringBean like JDBCConnection obj, JDBCStmt obj, RmiBusiness obj and etc...

(2) Constructor Injection :-

→ If Spring Container uses parameterized constructors to create SpringBean class obj and to assign values to Bean properties then it is called Constructor injection.

→ Setter injection takes place immediately after SpringBean class object creation where as Constructor injection takes place along with SpringBean class object creation.

Example Scenario :-

// TestBean.java (Spring Bean)

```
public class TestBean
```

```
{
```

```
    // Bean Property
```

```
    String msg;
```

```
    // Parameterized Constructor supporting Constructor injection
```

```
Public TestBean (String msg)
```

```
{  
    this.msg = msg;  
}
```

```
// Write B-methods
```

```
Public void bml()
```

```
{  
    ....  
    .... use msg here  
    ....  
}
```

```
} // class
```

Demo.xml (spring cfg file)

```
<beans>
```

```
<bean id="tb" class="TestBean">
```

```
<constructor-arg value="hai"/>
```

```
</bean>
```

```
</beans>
```

↳ since <constructor-arg> tag is placed only for one time the spring container uses one param constructor to create SpringBean class object and to assign value "hai" to "msg" property.

When SpringContainer is activated by giving the above Demo.xml springcfg file the SpringContainer creates TestBean class ^{object} ~~obj~~ having name "tb" by using 1-param constructor (TestBean tb=new TestBean("hai");)

NOTE:- assigning values directly to Beanproperties like name="hello" gives tight coupling between Beanproperties and its dependent values. Assigning values to Beanproperties by specifying the values in Spring Configuration File ~~file~~ and ^{by} enabling dependency Injection shows loose coupling b/w Beanproperties and its dependent values.

Naming Conventions

X → Spring interface name

XBean → SpringBean class name

XCfg.xml → Spring cfg file name

XClient → Client App name

Eg:- Test → Spring interface name

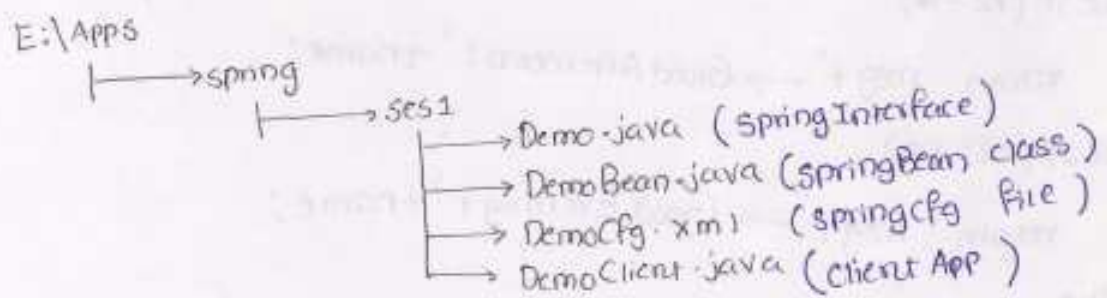
TestBean → SpringBean class name

TestCfg.xml → Spring cfg file name

TestClient → Client App name

1st Example Application of SpringCore module to demonstrate Setter Injection by using BeanFactory Container.

Step-I:- Develop the source code as shown below



// Demo.java (POJI) (SpringInterface)

```
public interface Demo
```

```
{
```

```
    // B-method Declaration
```

```
    public String generateWishMsg(String name);
```

```
}
```

// DemoBean.java (POJO class) (SpringBean)

```
import java.util.*;
```

```
public class DemoBean implements Demo
```

```
{
```

```
    // Bean Property to hold injected value
```

```
    private String msg;
```

```
    // Setter method supporting setter injection
```

```

public void setMsg(String msg) {
    { this.msg = msg; }
}

```

// implement B methods

```

public String generateWishMsg(String name)

```

```

{
    // get current date and time
    Calendar ca = Calendar.getInstance();

    // get current hour of the day
    int h = ca.get(Calendar.HOUR_OF_DAY);

    // generate wish msg
    if (h <= 12)
        return msg + " -> Good morning: " + name;
    else if (h <= 16)
        return msg + " -> Good Afternoon: " + name;
    else if (h <= 20)
        return msg + " -> Good Evening: " + name;
    else
        return msg + " -> Good Night: " + name;
} // method

```

} // class

<!-- DemoCfg.xml -->

<!DOCTYPE -----> indicates that we are developing this xml document based on DTD rules given in spring-beans-2.0-DTD file. collect this content from <spring-home>|dis|resources|

<beans>

<bean id="db" class="DemoBean"> -----> (e)

Spring-beans-2.0-DTD file.

<property name="msg" value="hello"/> -----> (p)

</bean>

DemoBean db = new DemoBean();

db.setMsg("hello");

</beans>

// DemoClient.java

```
import org.springframework.beans.factory.BeanFactory; // (1)
import org.springframework.beans.factory.xml.XmlBeanFactory; // (c)
import org.springframework.core.io.FileSystemResource; // (c)
```

```
public class DemoClient
```

```
{
```

```
    public static void main(String[] args) —→ (b)
```

```
{
```

```
    // Locate Spring Configuration File
```

```
    FileSystemResource res = new FileSystemResource("DemoCfg.xml");
```

```
    // Activate BeanFactory container
```

```
    XmlBeanFactory factory = new XmlBeanFactory(res);
```

```
    // get SpringBean class obj from SpringContainer
```

```
    Object obj = factory.getBean("db"); —→ (d)
```

```
    // type casting
```

```
    Demo bobj = (Demo) obj; —→ (h)
```

```
    // call B-method
```

```
    String result = bobj.generateWishMsg("raja");
```

```
    System.out.println("Result is " + result);
```

```
}
```

```
}
```

Step-II:-

add the following 2 jar files to CLASSPATH

<spring_home>\dist\spring.jar (Main jar File)

<spring_home>\lib\jackson-core\commons-logging.jar (Dependent jar
File to main jar file)
→ spring.jar

Step-III:- Compile all the java resources and execute the client application

> javac *.java

> java DemoClient

(a)

→ When BeanFactory container is activated the entries of Spring Configuration file will be verified by using SAX XmlParser
↳ SimpleAps for Xml

`Object obj = factory.getBean("db");`

- ① BeanFactory container to load "DemoBean" class based on the given Bean id "db" using configurations done in Spring Cfg file
- ② makes BeanFactory container to create "DemoBean" class object using 0-param constructor.
- ③ Since "msg" property is configured using <property> tag the BeanFactory container calls setMsg("hello") method on DemoBean class object perform setter injection
- ④ factory.getBean("db"); method returns "DemoBean" class object and we are referring that object using java.lang.Object class ref "obj"

NOTE:- getBean() method is the predefined method in predefined class "Object". if we want to get the Object (Eg: db) from BeanFactory container we use this getBean() method. If we call B-method generateWishMsg() it is not part of the predefined class "Object". it is part of the implement class of SpringInterface. so we must typecast the ^{Object class to DemoInterface} Demo bobj = (Demo) obj;

→ When Superclass reference variable refers its subclass Object then we can't call direct methods of subclass using that reference variable. To make this happening typecast that reference variable either with subclass or

with interface implemented by subclass having that Directmethod declaration
Demo obj = (Demo) obj; is placed to satisfy above statement in our
Spring apps.

→ Spring Container performs Dependency Injection only on those SpringBean objects
that are created by the Spring container itself.

That means SpringContainer can't perform Dependency Injection on SpringBean
class objects that are created by programmer manually.

→ Keeping DTD related or schema related header statements in SpringCfg file
is mandatory. But this work is optional in web.xml.

→ Spring application flow start with main ends with main. Control goes to
one resource to another resource.

In the client app we don't take SpringBean class manually as Typecast. The interface &
implementation class Demo will be taken.

→ When BeanFactory container is activated ~~it~~ it doesn't create any
SpringBean class objects. It creates SpringBean class objects and
Completes dependency injection only when getBean(-) method is called.

Example application on Constructor injection :-

Demo.java → same as previous appn.

DemoBean.java → same as previous appn. But add the following 1-param constructor

```
public DemoBean (String msg)
{
    this.msg = msg;
    System.out.println ("DemoBean 1-param constructor");
}
```

DemoCfg.xml

```
<DOCTYPE ..... >
<beans>
    <bean id="db" class="DemoBean">
        <constructor-arg value="hello" />
    </bean>
</beans>
```

DemoClient.java → same as previous appn.

→ While configuring SpringBean to place only <property> tags under <bean> tag then SpringContainer uses 0-param constructor for SpringBean class instantiation and also uses setterXxx(-) methods for setter injection on bean properties

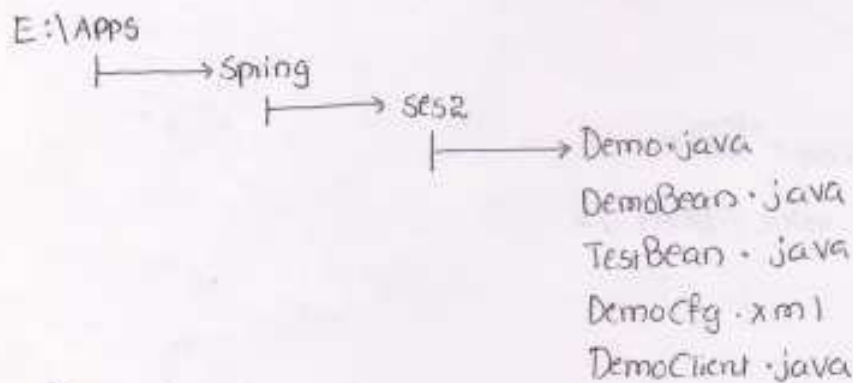
→ While configuring SpringBean if you place only <constructor-arg> tags under <bean> tag then SpringContainer uses parameterized constructor to create SpringBean class object & to perform Constructor Injection on Bean Properties.

Q:- What happens if both setterInjection, ConstructorInjections are enabled on same Bean properties of SpringBean class?

Ans:- SpringContainer creates SpringBean class object by using parameterized constructor. but the values given by setterInjection become final values of Bean property. Because setter method executes after construction. So, values injected by ConstructorInjection will be overridden because of the setterInjection.

- To configure dependent values to Simple bean properties (like String, simple data type & > use `<value>` or value attribute in Spring cfg file.
- To configure one Spring Bean class obj as dependent value other Spring Bean class properties use `<ref>` or ref attribute.
- if one spring Bean class is holding another SpringBean class obj then it called Bean Collaboration.

Example Application on `<ref>` tag / ref attribute (Bean Collaboration)



// Demo.java (POJI)

```

public interface Demo
{
    public String sayHello(); // Business Method
}
  
```

// DemoBean.java (POJO class)

```

import java.util.*;

public class DemoBean implements Demo
{
    // bean properties
    String name;
    int age; // simple property

    // user defined class
    TestBean tb;
    // predefined class
    Date d;
}
  
```

reference type properties to hold objects

// Write setXxx(-) methods supporting setter Injection

```

public void setAge(int age)
{
    this.age = age;
    S.o.p("DemoBean: setAge(-) method");
}
  
```

```

public void setTb(TesBean tb)
{
    this.tb = tb;
    S.o.p("DemoBean: setTb(-) method");
}

public void setD(Date d)
{
    this.d = d;
    S.o.p("DemoBean: setD(-) method");
}

// implement BusinessMethod

public String sayHello()
{
    return "Good Morning" + " age=" + age + "tb=" + tb + "d=" + d;
}

} // class

```

NOTE:- When object is passed in S.O.P() statement toString() will be called on that object automatically. In most of the classes toString() will be overridden to display object data. toString() originally belongs to java.lang.Object(c)

```

// TestBean.java (Dependent Bean to TestBean)

public class TestBean
{
    // Bean property
    String msg;

    // Write setter method of Setter Injection
    public TestBean(String msg)
    {
        this.msg = msg;
        S.o.p("TestBean: setMsg(-) method");
    }

    public void setMsg(String msg)
    {
        this.msg = msg;
        S.o.p("TestBean: setMsg(-) method");
    }

    public String toString()
    {
        return "TestBean.msg=" + msg;
    }
}

```


NOTE:- Here DemoBean, TestBean classes are Taken as Collaborates beans

because the "tb" property of DemoBean class can hold TestBean class Object.

```
<!-- DemoCfg.xml -->
```

```
<beans>
```

```
  <bean id="dt" class="java.util.Date">
```

```
    <property name="year" value="113"/>
```

Takes it as 1900+113

```
    <property name="month" value="10"/>
```

Takes it as 0 to 11

```
    <property name="date" value="29"/>
```

```
  </bean>
```

Eg: Boy and girl names

```
  <bean id="t1" class="TestBean">
```

```
    <property name="msg" value="hello"/>
```

```
  </bean>
```

```
  <bean id="db" class="DemoBean">
```

```
    <property name="age" value="30"/>
```

```
    <property name="tb" ref="t1"/>
```

```
    <property name="d" ref="dt"/>
```

<!-- or <property name="d">

<ref bean="dt"/>

```
  </bean>
```

</property> -->

```
</beans>
```

```
// DemoClient.java
```

Same as 1st application but call sayHello() B-method instead of generateWishMsg() B-method.

```
String result = bobj.sayHello();
```

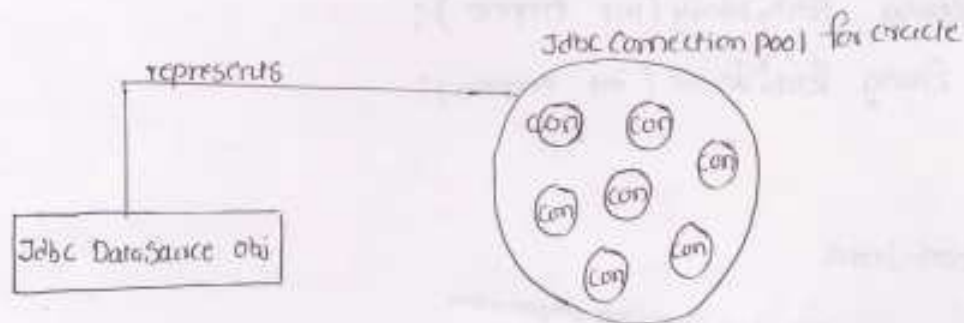
```
S.o.p("Result is: " + result);
```

```
}  
}
```

Jdbc Connection Pool :-

Jdbc Connectionpool is factory that contains set of readily available Jdbc Connection Objects. All ^{the} Connection objects in the Connectionpool represents Connectivity with Same DataBase slw.

→ DataSource object represents Jdbc Connection pool and this can be used to access Jdbc Connection Objects from Connection pool.



Jdbc DataSource obj is nothing but it is the object of underlying Jdbc driver Supplied java class that implements java.sql.DataSource(I)

→ org.springframework.jdbc.datasource.DriverManagerDataSource class gives Jdbc DataSource object pointing to jdbc connection pool. This class creates jdbc connection pool based on the jdbc driver details Supplied to bean properties.

The important bean properties are

- 1) driverClassName
- 2) url
- 3) username
- 4) password

If these properties are filled up with oracle thin driver details then jdbc connection pool for oracle will be created.

Eg:- Example application to inject jdbc DataSource object that points to jdbc connection pool to our SpringBean class using Dependency Injection.

E:\APPS

└─> Spring

└─> se63

└─> DBOperation.java (Spring Interface)

DBOperationBean.java (Spring Bean)

SpringCfg.xml (Spring Configuration File)

DBOperationClient.java (Client app)

//DBOperation.java (POJI)

```
public class Interface DBOperation
{
    public String int fetchSalary(int empno);
    public String fetchName(int empno);
}
```

//DBOperationBean.java

import java.sql.*; ^{→ Connection, PreparedStatement}

import javax.sql.*; ^{→ DataSource obj}

public class DBOperationBean implements DBOperation

{

 //Bean property

 DataSource ds;

 //Write setXxx(-) method to support setter injection

```
    public void setDs(DataSource ds)
    {
        this.ds = ds;
    }
```

 //implement B-methods

```
    public String fetchName(int empno)
    {
```

 try

 {

^{one} //get Connection obj from Connection pool

 Connection con = ds.getConnection();

 //get emp name

 PreparedStatement ps = con.prepareStatement("select ename from emp where empno=?");

```

ps.setInt(1, empno);
ResultSet res = ps.executeQuery();
String name = null;
if(rs.next())
{
    name = rs.getString(1);
}
else
{
    name = "record not found";
}
return name;
}
catch (Exception e)
{
    return "DB problems";
}
}

```

} //method

```

public int fetchSalary(int empno)
{

```

```

    int int sal = 0;

```

```

    try
    {

```

```

        //get one connection obj from jdbc con pool

```

```

        Connection con = ds.getConnection();

```

```

        //get emp salary

```

```

        PreparedStatement ps = con.prepareStatement("select sal from emp where empno=?");

```

```

        ps.setInt(1, empno);

```

```

        //execute the query

```

```

        ResultSet rs = ps.executeQuery();

```

```

        // process the results

```

```

        if(rs.next())

```

```

            int
            sal = rs.getInt(1);

```

```

        } //try

```



```

    catch (Exception e)
    {
        e.printStackTrace();
    }
    return sal;
} //method
} //class

```

→ <!-- Spring Cfg.xml -->

<!DOCTYPE >

<beans>

gives
dataSource
obj

Ⓐ

```

<bean id="drds" class="org.springframework.jdbc.datasource.DriverManagerDataSource" >
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:orcl">
    <property name="username" value="scott"/>
    <property name="password" value="Hger"/>
</bean>

```

↳ gives DataSource obj representing the spring container managed builtin connection pool

<bean id="dop" class="DBOperationBean" > Ⓒ

<property name="ds" ref="drds"/> Ⓓ → Injects DataSource obj to our Bean Property ds.

</bean>

</beans>

→ //DBOperationClient.java

import org.springframework.beans.factory.xml.*;

import org.springframework.core.io.*; → FileSystemResource class

public class DBOperationClient

{
 public void m(String args[]) throws Exceptions Ⓔ

{
 //Locate Spring Configuration file

FileSystemResource res = new FileSystemResource("springcfg.xml"); Ⓕ

//Activate BeanFactory container

```
XmiBeanFactory factory = new XmiBeanFactory(res);
```

```
// get SpringBean obj from container
```

```
DBOperation bobj = (DBOperation) factory.getBean("dbb"); (d)
```

```
// call B.methods
```

```
System.out.println("Emp salary is " + bobj.fetchSalary(7499));
```

```
S.O.P ("Empname is " + bobj.fetchName(7499));
```

```
} // main
```

```
} // class
```

```
//> javac *.java
```

```
//> java DBOperationClient
```

jar files in CLASSPATH:-

① Spring.jar ② Commons-logging.jar ③ OJdbc14.jar

NOTE:- in the above application SpringContainer is injecting DataSource object to our SpringBean class property "ds".

*) `<property name="ds" ref="didb" />` can also be written as
`<property name="ds" /> <ref bean="didb" /> </property>`

x) `<property name="password" value="tiger" />` can also be written as
`<property name="password" /> <value>tiger </value> </property>`

→ we can locate Spring Configuration file for BeanFactory container by using the following classes.

1) FileSystemResource Class (org.springframework.core.io package)

→ locates the Spring Configuration file from the specified path of FileSystem.

→ Here we can specify Absolute path (or) relative path of Spring cfg file

NOTE:- all files folder of our computer together is called FileSystem.

2) ClassPathResource Class (org.springframework.core.io pkg)

→ locates the Spring cfg file from the directory locations, jar files added to CLASSPATH environment Variable.

Example on FileSystemResource to locate Spring Cfg file :- When springcfg file is there in E:\Apps\temp folder and when clientapp is there in E:\Apps\spring\sess folder

in client App:-

FileSystemResource res = new FileSystemResource("E:/apps/temp/springcfg.xml");
(or)

FileSystemResource res = new FileSystemResource("E:\\apps\\temp\\springcfg.xml");
(or) ↳ absolute path

FileSystemResource res = new FileSystemResource("../..\\..\\temp\\SpringCfg.xml");
↳ relative path

Spring
Folder
(child folder)

Apps
Folder
(Parent
folder)

Example on Utilizing CLASSPATH resource :-

Step-I:- keep spring application ready

Step-II:- get the location on Spring Configuration file and add the location to the CLASSPATH env... Variable.

E:\
├── Apps
│ ├── temp
│ │ └── SpringCfg.xml

my Computer → properties → advanced tab → env variables → user/system
Variables → Variable name: CLASSPATH → OK → OK → OK
Value: E:\Apps\temp; <existing values>;

Step-III:- use CLASSPATH resource in client application

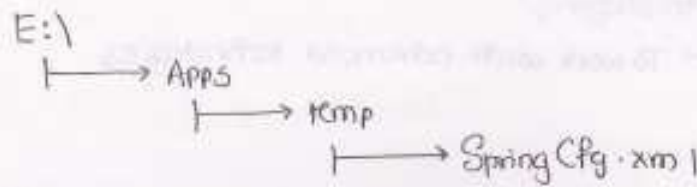
in clientApp

ClassPathResource res = new ClassPathResource("SpringCfg.xml");

Example② on CLASSPATH resource :-

Step-I:- keep spring application ready

Step-II:- prepare jar file representing Spring Configuration file and add that jar file to CLASSPATH.



E:\Apps\temp> jar cf test.jar .

my computer → properties → advanced tab → env variable → user/system
 Variables → Variable name : CLASSPATH
 Value : E:\Apps\temp\test.jar ; <existing values> ; → OK → OK

Step-III :- use ClassPathResource in client appn
in client appn

```
ClassPathResource res = new ClassPathResource("Spring Cfg.xml");
```

While working with Spring Environment always make sure that Current directory
 (.) is added to Classpath Env. Variable.

To cfg Dependent values to diff types of bean properties we need to use diff tags
 and attributes

Property type

Simple data type

java.lang.String

array

java.util.List

java.util.Set (HashSet, TreeSet etc)

java.util.Map

Bean type

java.util.Properties

tag | attribute to use

<value> or "value" attribute

<value> or "value" attribute

<list>

<list>

<set>

<map>

<ref> or "ref" attribute

<props>

note :- To assign null value to Bean property use <null/> tag

MyEclipse = Eclipse + Built-in plugins

↳ To work with advanced technologies.

MyEclipse

type: IDE slw for java

vendor: Eclipse org

Commercial IDE

gives Tomcat as Built-in server and also to cfg other external servers.

version: 10.x (compatible with JDK 1.6+)

To download slw: www.myeclipseide.com (30 days trail)

Procedure to keep MyEclipse IDE ready in our Computer :-

Step I:- install MyEclipse 10.x. (from www.myeclipseide.com)

Step II:- Get the Crack related to MyEclipse 10.x cheat code (in the form of zip file www.naturaz.in blog)

Step III:- Run-Batch file. which comes bcz of Crack extraction

Step IV:- Follow the instructions given by application launched by run-batch file

and get licency name & licence key

Step V:- Submit the above license details ~~through~~^{to} MyEclipse IDE

MyEclipse menu → Subscription Information → Subscriber:
subscriptioncode:

→ Ok.

procedure to develop Spring Core module application in MyEclipse 10.x IDE that performs setter injection on all types of Bean properties.

Step-I:- launch MyEclipse IDE specifying Workspace folder
↳ The folder where the projects will be saved.

Step-II:- Create Java project in MyEclipse IDE.

File → new → Javaproject → project name: → next → Finish

Step-III:- Add Spring Capabilities to the project.

Right click on project → MyEclipse → Add Spring Capabilities → @spring 2.5 →

☒ Spring 2.5 Core Libraries → next → ☐ Aop Builder → File: → Finish

Other read
State section
[2-1] download
one image file
that image file array
then we get real zip file -> this
is imp

Step-IV:- Add SpringInterface, SpringBean class to the Src folder of the project

right click on src folder → New → Interface → Filename: Demo

```
public interface Demo
{
    public String sayHello();
}
```

Step-V:- Add SpringBean class

right click on src folder → New → class → Filename: DemoBean

```
public class DemoBean implements Demo
```

```
{ // Bean properties
    String name;
```

```
    int age;
```

```
    Date d;
```

```
    List fruits;
```

```
    String colors[];
```

```
    Set phones;
```

```
    Map faculties;
```

```
    Properties capitals;
```

ctrl+shift+O package import stmts.

Select all Bean Properties → right click → Source → generate setters and getters → setters → insertion point After capitals

after setters methods

@Override

```
public String sayHello()
```

```
{
    return "good morning" + "name=" + name + "age=" + age + "d=" + d +
        "fruits=" + fruits + "colors=" + colors[0] + "..." + colors[1] + "..." +
        colors[2] + "phones=" + phones + "faculties=" + faculties + "capitals=" + capitals;
```

```
} // method
```

```
} // class
```

Step-VI:- add following content in spring configuration file.

<beans ----- Spring-bean-2.5.xsd >

<bean id="dt" class="java.util.Date" />

<bean id="db" class="DemoBean">

<property name="name" value="raja" />

<property name="age" value="30" />

<property name="d" ref="dt" />

<property name="fruits">

<list>

<value> apple </value>

<value> banana </value>

<ref bean="dt" />

</list>

</property>

<property name="colors">

<list>

<value> red </value>

<value> yellow </value>

<value> blue </value>

</list>

</property>

<property name="phones">

<set>

<value> 9247285621 </value>

<value> 7702349715 </value>

<ref bean="dt" />

</set>

</property>

<property name="faculties">

<map>

<entry>

<key> <value> ramesh </value> </key> → represents key of the element

<value> Java Faculty </value> → represents value of the element

</entry>

represents
one
element

~~represents one element~~

<key> <value> ramana </value> </key>

<value> .NET faculty </value>

</entry>

<entry>

<key> <value> today </value> </key>

<ref bean="dt"/>

</entry>

</map>

</property>

<property name="Capital5">

<props>

represents one element ← <prop key="AP"> ^{key}hyd ^{value}</prop>

<prop key="MH"> Mumbai </prop>

<prop key="KR"> Bangalore </prop>

</props>

</property>

</bean>

</beans>

→ In java.util.Properties Map data structure element key must be string and value must be string.

→ In Other Map data structure element key can be any Java object and value can be any Java object.

Step-VII:- add client appn to Src

right click on Src → new → class → name: DemoClient

// DemoClient.java

public class DemoClient

{

public static void main (String[] args)

{

// Activate BeanFactory Container

XmBeanFactory ~~new~~ factory = new XmBeanFactory (new ClassPathResource ("Springfg.xml"));

// get SpringBean obj

Demo bobj = (Demo) factory.getBean ("db");

// call B. methods

System.out.println (bobj.sayHello());

}

Step-VIII:- Run the appn.

Rightclick in the DemoClient.java → Run as → Java application.

→ While working with DriverManagerDataSource class we can use java.util.Properties type bean property called ConnectionProperties to specified username and Password.

Wrt to SpringCfg.xml of sess appn on 18/02/2013 appn.

<!-- SpringCfg.xml -->

<!DOCTYPE >

<beans>

<bean id="datas" class="org.springframework.jdbc.datasource.DriverManagerDataSource">

<property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>

<property name="url" value="jdbc:oracle:oci8:@orcl"/>

<property name="connectionProperties">

<props>

<prop key="user"> scott </prop>

<prop key="password"> tiger </prop>

</props>

</property>

</bean>

→ To Pass Command line argument Values to MyEclipse IDE appn

Rightclick in the SourceCode of appn → Run as → RunConfigurations

→ Arguments tab →

Program arguments				
db				
val1	val2	val3		valn
args[0]	args[1]	args[2]		args[n-1]

Constructor Injection

→ If Spring Container is using Parameterized Constructor to create Bean class object & to assign Dependent Values to Bean properties then it is called Constructor Injection.

→ Based on `<constructor-arg>` tag appearance count under `<bean>` tag the spring container picks up appropriate parameterized constructor to perform constructor injection.

Example appn Demo.java → same as previous appn

DemoBean.java

```
public class DemoBean implements Demo
```

```
{  
    // DemoBean properties
```

```
    int age;
```

```
    String name;
```

```
    float salary;
```

```
    // parameterized Constructors Supporting
```

Select Bean properties → source → generate constructor using field → select All

→ OK.

```
// implement Business method
```

```
public String sayHello()
```

```
{  
    return "age=" + age + " name=" + name + " salary=" + salary;
```

```
}
```

```
}
```

`<springcfg.xml>`

```
<bean id="db" class="DemoBean">
```

```
    <constructor-arg value="30"/>
```

```
    <constructor-arg value="raja"/>
```

```
    <constructor-arg value="34563.69"/>
```

```
</bean>
```

```
</beans>
```

These values must be specified in the order the parameters are available in constructor.

→ Since `<constructor-arg>` tag is placed for 3 times the Spring Container uses three param constructor for

SpringBean instantiation and for Constructor Injection.

// DemoClient.java

Same as previous app.

NOTE:-

While Working with Constructor injection we need to Configure dependent values in the order the parameters are placed in the constructor.

→ If not interested we can resolve/identify parameters ~~and~~ based on their type of Index.

→ If each Parameter type is ^{in the constructor} unique then we can identify that parameter based on "type" in Spring Configuration file during constructor injection configurations.

Eg:- in SpringBean

```
public DemoBean(int age, String name, float salary)
{
    this.age = age;
    this.name = name;
    this.salary = salary;
}
```

in Spring Configuration file

```
<beans>
<bean id="db" class="DemoBean">
    <constructor-arg type="int" value="30"/>
    <constructor-arg type="float" value="3456.127"/>
    <constructor-arg type="java.lang.String" value="raj a"/>
</bean>
</beans>
```

→ If multiple parameters of constructor contains same type then we can resolve/identify them based on their ~~Index~~ indexes.

Eg:- in SpringBean

```
int a,b,c;
public DemoBean(int a, int b, int c)
{
    this.a = a;
    this.b = b;
    this.c = c;
}
```

in Spring Configuration file

```
<bean id="db" class="DemoBean">
```

```
<constructor-arg index="0" value="10"/> goes to "a"
```

```
<constructor-arg index="2" value="30"/> goes to "c"
```

```
<constructor-arg index="1" value="20"/> goes to "b"
```

```
</bean>
```

```
</beans>
```

- There is no provision to specify BeanProperty name (or) Constructor parameter name in `<constructor-arg>` tag. For this reason we need to identify constructor parameters either through their data ^{types} (or) indexes.



→ There is a provision to activate BeanFactory container based on the already activated BeanFactory container. In this process the 2nd BeanFactory container acts as the child container, and the first BeanFactory container acts as parent container.

Example application:-

Demo.java → Same as previous appn

// DemoBean.java

import java.util.Date;

public class DemoBean implements Demo

{

 Date d;

 public void setD (Date d)

 { this.d = d;

 }

 public String sayHello()

 {

 return "d=" + d;

 }

}

// DemoCfg1.xml (Parent file)

<beans ... >

 <bean id="dt" class="java.util.Date"/>

</beans>

// DemoCfg2.xml (Child file)

<beans ... >

 <bean id="db" class="DemoBean">

 <property name="d" > <ref bean="dt" /> </property>

 </bean>

</beans>

// DemoClient.java

imports

public class DemoClient {

 public static void main (String[] args) {

```

// Parent Bean Factory container
XmlBeanFactory factory1 = new XmlBeanFactory(new ClassPathResource("DemoCfg.xml"));

// child Bean Factory container
XmlBeanFactory factory2 = new XmlBeanFactory(new ClassPathResource("DemoCfg2.xml"),
factory1);

```

```

// get SpringBean class obj

```

```

Demo bobj = (Demo) factory2.getBean("db");

```

```

S.o.p ("Result is" + bobj.sayHello());

```

```

}

```

```

}

```

Q:- What is the diff b/w Bean and local attribute of <ref> tag?

Ans:- "bean" attribute searches for the dependent bean in current Spring Configuration file and also in parent Spring Configuration file.

Eg:- <property name="d"> <ref bean="dt"/> </property>

"local" attribute searches for the dependent bean in current Spring Cfg file

Eg:- <property name="d"> <ref local="dt"/> </property>

<property name="d" ref="dt"/> is same as <property name="d"> <ref bean="dt"/> </property>

→ We can make Spring Containers to keep SpringBean class objects in the following scopes.

① Singleton

② Prototype

③ request

④ session

① Singleton:- Container creates only one object of SpringBean class even though factory.getBean() method is called for multiple times with sameBean id.

Eg:- `<bean id="db" class="DemoBean" scope="singleton">`

`</bean>`

② Prototype:- Container creates multiple objects for SpringBean class for multiple factory. factory.getBean() method calls having sameBean id.

Eg:- `<bean id="db" class="DemoBean" scope="prototype">`

`</bean>`

③ request:- Container keeps SpringBean class object as request attribute value in webapp.

Eg:- `<bean id="db" class="DemoBean" scope="request">`

`</bean>`

④ session:- Container keeps SpringBean class object as session attribute value in webapp.

Eg:- `<bean id="db" class="DemoBean" scope="session">`

`</bean>`

NOTE:- request, session scopes are applicable only in web environment...

The Java class that allows us to create only one Java object per JVM is called

Singleton Java class. Even though java class allows multiple objects creation if we are creating only one object for it then it is not called Singleton java class.

When SpringBean Scope is Singleton our SpringBean class doesn't become Singleton java class.

→ Singleton Scope SpringBeans can be configured as dependents to prototype Scope SpringBean properties and vice versa... In this whole process Container doesn't change scopes of any Bean.

NOTE:- most recommended scope for SpringBean is Singleton Scope.

Q:- Can be placed only parameterized constructors in SpringBean when all properties of SpringBean are configured for SetterInjection?

Ans:- Not possible,

spring
Bcz Container looks to use 0-param constructor to create the above Spring Bean class object.

Application Context Container :-

It is enhancement of BeanFactory container.

This container is available in spring context / JEE module.

It is also light-weight container to activate this container create object for a class that implement `org.springframework.context.ApplicationContext (I)`.

This Interface is subinterface of `org.springframework.beans.factory.BeanFactory (I)`.

There are 3 important classes implementing `ApplicationContext (I)`. we can use these classes to activate Application Context container.

① `FileSystemXmlApplicationContext (org.springframework.context.support)`

→ Activates Spring Container by locating `Spring.cfg` file from the specified path of `FileSystem`.
↳ `ApplicationContext`

`FileSystemXmlApplicationContext ctx = new FileSystemXmlApplicationContext("demo.cfg.xml")`

② `ClassPathXmlApplicationContext (org.springframework.context.support)`

→ Activates Spring Container by locating Spring cfg file from the values added to the Classpath env.... Variable.

`ClassPathXmlApplicationContext ctx = new ClassPathXmlApplicationContext("DemoCfg.xml");`

③ `XmlWebApplicationContext (org.springframework.web.context.support)`

→ Activates ApplicationContext container in Webapplications

~~X~~

25/02/2013

→ Bean property can be taken as static or non-static but the setter method, taken for setter injection must be taken as non-static method.

Using ApplicationContext container we can perform all the activities ^{that are possible} with BeanFactory container but it can be also used to perform additional functionalities like

1) Pre-instantiation of Singleton beans
2) ability to work with multiple Spring Configuration files in single instance of Container activation.

3) Ability to read values from properties file

4) Support for i18n [Internationalization]

5) Support for Event Handling and etc....

NOTE:- all the above features are not there in BeanFactory container.

All these features are exclusive features of ApplicationContext container.

* In IDE Environment multiple Spring Beans developed by multiple programmers will be configured in different Spring Configuration files. we can activate ApplicationContext container by specifying this multiple Spring Configuration files.

// Activate ApplicationContext container

`String cfg[] = {"DemoCfg1.xml", "DemoCfg2.xml"};`

`ClassPathXmlApplicationContext ctx = new ClassPathXmlApplicationContext(cfg);`

NOTE:- in BeanFactory env... to work with multiple configuration files we need to activate multiple instances of BeanFactory container.

→ If SpringBean objects are created by container in the moment SpringContainer is activated then it is called pre-instantiation of SpringBean.

ApplicationContext Container performs pre-instantiation on "Singleton scope" SpringBeans.

Q:- What happens if "prototype" scope SpringBean is taken as dependent value to "Singleton" scope SpringBean class property?

Ans:- Container creates "Prototype" scope SpringBean class object and Singleton scope SpringBean class object through pre-instantiation process if the container is ApplicationContext Container. But it does not change the scope of any Bean.

Q:- When ^{can} SpringBeans are configured in Spring configuration file, how can we enable pre-instantiation (~~using Singleton~~ creating objects when container is activated) only on four SpringBeans.

Ans:- Take four SpringBeans having Singleton scope and take other 6 Beans having prototype scope and also make sure that prototype scope SpringBeans are not dependent to Singleton scope SpringBeans.

Q:- What happens when factory.getBean(-) is called?

Ans:- Creates and returns SpringBean class object if SpringBean scope is "prototype".
Creates/locates and returns SpringBean class object if SpringBean scope is "Singleton".

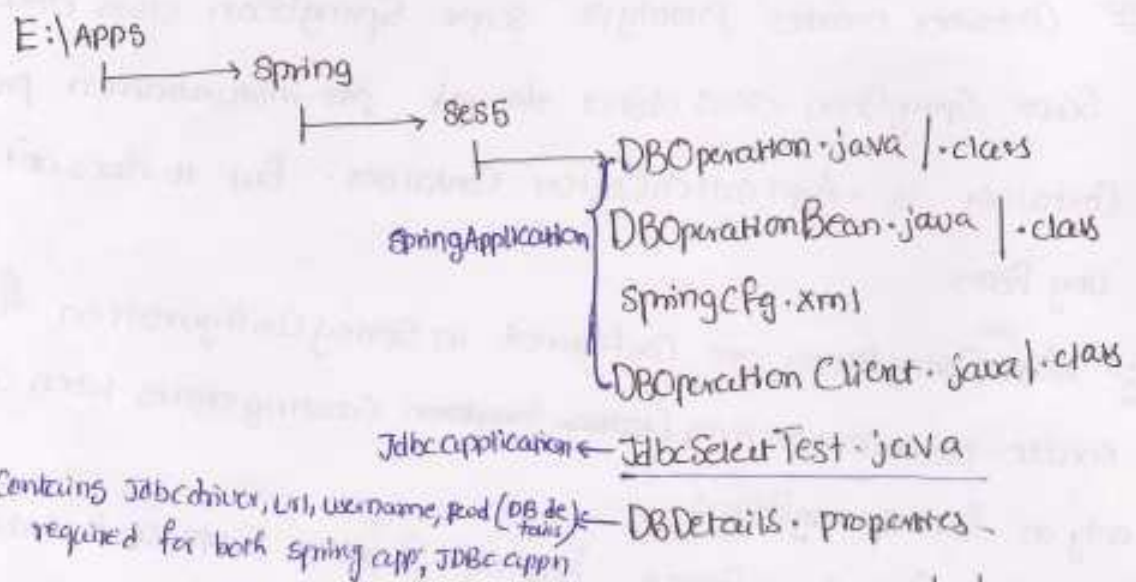
Q:- What happens when ctx.getBean(-) is called?

Ans:- Creates and returns SpringBean class object if SpringBean scope is "prototype".
Locates and returns SpringBean class object if SpringBean scope is "Singleton".

→ The text file that maintains the entries in the form of key=value pairs is called "TextProperties file".

The standard principle of S/W industry is don't hard code any values in our application that are changable in the future. It is recommended to pass these values to application from outside the application by taking the support of TextProperties file / xml file.

→ If Spring Application placed along with other Technology based apps, it is recommended to make Spring Application to use same properties file that other applications are using to centralized common data.



→ To make SpringConfiguration file ~~contains~~ getting certain values from properties file we place placeholders specifying the keys of the properties file (like $\{key\}$)

To recognize these placeholders and to make container gathering values ^{represents placeholder} from properties file based on the placeholders that are specified we need to configure org.springframework.beans.factory.config.PropertyPlaceholderConfigurer as spring bean in springCfg file.

Source code of the above application :

jdbc driver and DB details

// DBDetails.properties

```
my.driver = oracle.jdbc.driver.OracleDriver
my.url = jdbc:oracle:thin:@localhost:1521:orcl
my.dbuser = Scott
my.dbpwd = tiger
```

Spring application

DBOperation.java → Same as 18/02/2013 appm

DBOperationBean.java → Same as "

makes container to recognize placeholders & properties file.

<!-- Spring Cfg.xml -->

<!DOCTYPE ---- ->

<beans>

<bean id="propconfig" class="org.springframework.context.PropertyPlaceholderConfigurer">

<property name="location" value="DBDetails.properties"/>
↳ properties file in current directory.

</bean>

<bean id="dtds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">

<property name="driverClassName" value="\${my.driver}"/>

<property name="url" value="\${my.url}"/>

<property name="username" value="\${my.dbuser}"/>

<property name="password" value="\${my.dbpwd}"/>

</bean>

<bean id="dobj" class="DBOperationBean">

<property name="ds" ref="dtds"/>

</bean>

</beans>

// DBOperationClient.java

```
import org.springframework.support.*;
```

```
public class DBOperationClient
```

```
{
```

```
    public void m(String[] args) throws Exception
```

```
    {
```

```
        // Activate Application Context Container
```

```
        FileSystemXmlApplicationContext ctx = new FileSystemXmlApplicationContext("springcfg.xml");
```



```

// get Spring Bean obj from container
DBOperation bobj = (DBOperation) ctx.getBean("dob");

// call b. methods
S.O.P("Emp salary is" + bobj.fetchSalary(7499));
S.O.P("Emp Name is" + bobj.fetchName(7499));

} // main
} // class
// > javac x.java
// > java DBOperationClient

```

JDBC application :

// JDBC SelectTest.java

import java.sql.*; → query operation

import java.util.*;

import java.io.*; → stream

public class JdbcSelectTest

{ p.s.v.m(String[] args) throws Exception

{

// locate properties file

~~FileInputStream~~ ^{Stream} fis = new FileInputStream("DBDetails.properties");

// load the content of properties file to java.util.Properties class obj

Properties p = new Properties();

p.load(fis);

// read content from Properties class obj

String s1 = p.getProperty("my.driver");

String s2 = p.getProperty("my.url");

String s3 = p.getProperty("my.driver");

String s4 = p.getProperty("my.dbread");

// Write JDBC code

Class.forName(s1);

Connection con = DriverManager.getConnection(s2, s3, s4);

Statement st = con.createStatement();

```

ResultSet rs = st.executeQuery("select * from Student");
while(rs.next())
{
    S.o.p(rs.getInt(1) + " " + rs.getString(2) + " " + rs.getString(3));
}
//close jdbc obj's
rs.close();
st.close();
con.close();
} //main
} //class
1> javac JdbcSelectTest.java

```

- BeanFactoryContainer cannot recognize and use place holders and properties file even though the special Bean "propertyPlaceholderConfigurer" is configured.
- We can configure multiple properties file in SpringEnvironment. If this multiple properties file contain same key with different values then the values collected from lastly configured properties file will be taken and used.

In <!-- springcfg.xml -->

Eg:-

```

<beans>
  <bean id="propConfig" class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
    <property name="locations">
      <list>
        <value>DBDetails.properties</value>
        <value>DBDetails.properties</value>
      </list>
    </property>
  </bean>
  ...
</beans>

```


* Internationalization

→ Country + Language = locale.

java.util.Locale class is given for working with locales.

Eg:- en-US, fr-FR, fr-CA, hi-IN etc....
↳ French as its speak in CANADA.

→ Making our application working for different locales is nothing but enabling i18n on the application. For this we need to take care of the presentation labels, number formats, Date formats, Currency symbols and etc....

→ To enable i18n on our applications we need multiple locales specific properties files including Base properties file. If no matching properties file is found then our application uses labels collected from the Base properties file.

We can use google translator & unicode editor tools to get presentation labels in different languages. ^{are} ~~are~~ to get their unicode numbers.

→ If label can't be typed using English alphabets get their unicode numbers and supplied to the applications. To get Unicode numbers for the content of regional languages like Telugu, Hindi and etc... we can use Unicode editor tool downloaded from "higopi.com"

27/02/2013

Procedure to get Unicode numbers for certain hindi words ~~and~~ by using Unicode editor and ~~native2ascii~~ ~~tool~~ native2ascii tool

Step 1 Download unicode editor tool from www.higopi.com.

Step 2 Launch "unicode editor tool" through index.html and type the following 4 hindi words निकानी, सुरक्षित, रुखी, रेश

Step 3:- Copy above 4 hindi words to Notepad text file.

P1.txt

????

??????

????

????

NOTE:- While saving this file choose ^{the} encoding ^{as} made as unicode.

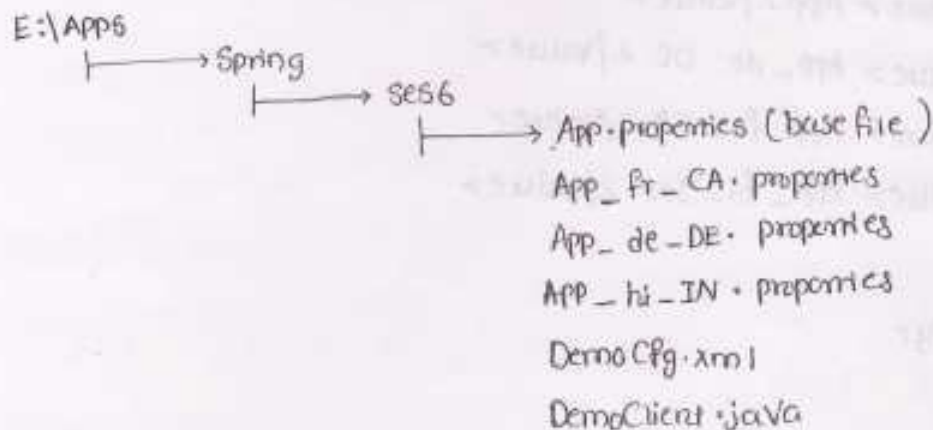
Step 4:- Use ^{the} native2ascii tool to get Unicode numbers

iknContext > nativeRascli -encoding unicode p1.txt p2.txt
↳ built in tool of JDK
↳ contains the generated unicode numbers

You can make ApplicationContext container ready for make i18n by configuring the following SpringBean having properties file names.

org.springframework.context.support.ResourceBundleMessageSource.

Spring based i18n App Example appn



// App-properties (BaseFile)

SH1 = delete

SH2 = save

SH3 = stop

SH4 = cancel

// App_fr_CA.properties (French)

SH1 = EFFACER

SH2 = SAUF

SH3 = ARRÊT

SH4 = ANNULER

// App_de_DE.properties (German)

SH1 = LÖSCHEN

SH2 = DENN

SH3 = ZUG

SH4 = ABSAGEN

// App_hi_IN.properties (Hindi)

SH1 = {u0948,u094f,u0945,u094d} u0942 u094a Unicode representation for अक्षत

SH2 = ----

SH3 = ----

SH4 = ----

} Copy from p2.txt file

NOTE:- The Filename of BaseFile is must be there in Localespecific File names.

→ all properties file must contain same keys having different values

<!-- DemoCfg.xml -->

<beans>

<bean id="messageSource" class="org.springframework.context.support.ResourceBundleMessageSource">

<property name="basenames">

</list>

<value>App</value>

<value>App-de-DE</value>

<value>App-fr-CA</value>

<value>App-hi-IN</value>

</list>

</property>

</bean>

</beans>

// DemoClient.java

import org.springframework.context.support.*;

import java.util.*;

import javax.swing.*;

import java.awt.*;

public class DemoClient {

public static void main(String[] args) throws Exception {

// Locale Object

Locale l = new Locale(args[0], args[1]);

// Activate Application context Container

ClassPathXmlApplicationContext ctx = new ClassPathXmlApplicationContext("DemoCfg.xml");

// get Msgs from properties file

String msg1 = ctx.getMessage("str1", null, "default msg1", l);

String msg2 = ctx.getMessage("str2", null, "default msg2", l);

String msg3 = ctx.getMessage("str3", null, "default msg3", l);

String msg4 = ctx.getMessage("str4", null, "default msg4", l);

// Develop Swing Frame

JFrame jf = new JFrame();

Container cp = jf.getContentPane();

// create btns by getting labels from activated

JButton b1 = new JButton(msg1);

JButton b2 = new JButton(msg2);

JButton b3 = new JButton(msg3);

JButton b4 = new JButton(msg4);

cp.setLayout(new FlowLayout());

cp.add(b1);

cp.add(b2);

cp.add(b3);

cp.add(b4);

jf.pack();

jf.show();

} // main

} // class

//> javac DemoClient.java

//> java DemoClient fr FR

//> java DemoClient de DE

//> java DemoClient x y

(uses APP_fr_FR.properties)

(uses App.properties)

→ ctx.getBean(-) gives SpringBean class object similarly ctx.getMessage(-,-,-) gives Value of the properties file based on the given key.

Can you compare BeanFactory Container and Application Context Container.

Feature	BeanFactory	ApplicationContext
BeanInstantiation / ^{Wiring} Wiring _{Dependency Injection}	yes	yes
Ability to work with multiple spring cty files with single Instance of container	NO	Yes
pre-instantiation ^{spring} of Beans	NO	yes
Recognition of placeholders and properties files	NO	yes
i18N Support	NO	yes
ApplicationEvent handling	NO	yes
Automatic BeanPostProcessor registration	NO	yes
Automatic BeanFactoryPostProcessor registration	NO	yes

Conclusion:- while using Spring in memory sensitive applications like Applets prefer using BeanFactoryContainer (uses less memory). In all other applications prefer Application Context Container.

→ Action Performed on the Component / Object is called "Event". The process of executing logics when Event is raised is called "Event Handling". To perform this Event Handling we need EventListeners. When the Application Context Container is started or stopped the ApplicationEvent will be raised. we can handle this ApplicationEvent we can use ApplicationListener. In this process we can notice the amount of time that the Application Context Container is there in running mode.

Example Application:-

Step-I:- keep 1st appn ready. (ses 1)

Step-II:- Develop separate Listener class implementing org.springframework.context.ApplicationListener Interface.

```
// MyListener.java
import org.springframework.*;
import java.util.*;
```

public class MyListener implements ApplicationListener

{

long stime, endtime;

public void onApplicationEvent(ApplicationEvent ae)

{

S.o.p("onApplicationEvent of MyListener class");

if(ae.toString().indexOf("ContextRefreshedEvent") != -1) // When Container is started

{

stime = System.currentTimeMillis();

S.o.p("The Container is started at " + new Date());

}

else if(ae.toString().indexOf("ContextClosedEvent") != -1) // When Container is stopped

{

endtime = System.currentTimeMillis();

S.o.p("The container is stopped at " + new Date());

S.o.p("The current container is in running mode for " + (endtime - stime) + "msecs");

} //method

} //onApplicationEvent(-)

} //class

Step III:- configure the above Listener class in springcfg file.

<!-- DemoCfg.xml -->

<!DOCTYPE ---->

<beans>

<bean id="mb" class="MyListener"/>

<bean id="db" class="DemoBean"/>

Step IV:- Activate Application Context container in the Client app and stop that container at the end by calling ctx.close();

→ SpringBean can have two user-defined methods as lifecycle methods.

① init lifecycle method

→ Container execute this method right after Bean Instantiation and Dependency Injection.

→ This method is useful to check whether bean properties are assigned with valid values or not. It is also useful to check whether mandatory bean properties are assigned with values or not.

② destroy lifecycle method

→ Container executes this method when it is about to destroy our SpringBean class object.

→ This method is useful to nullify bean properties and release non-Java resources associated with Springbeans (like closing jdbc conn obj).

These two life cycle method must be specified in SpringBean during configuration in

Spring cfg file using init-method, destroy-method attributes of <bean> tag.

Re-prog → Restart →
Application → MyEclipse
Library

Run → Search if we want
to any information

Example Application :-

// Demo.java

```
public interface Demo
{
    public String sayHello();
}
```

// DemoBean.java

public class DemoBean implements Demo

```
{
    // Bean properties
    private String msg;
    private int age;
```

~~private~~ setters & getters

// user-defined init life cycle method

public void myInit() throws Exception

```
{
    S.O.P("DemoBean : myInit()");
```

Validating Bean property value. { if (age <= 0 || msg == null)
throw new Exception("set proper values to Bean property");
}

// user-defined destroy() life cycle method

```
public void destroy()
```

```
{ S.o.p (" DemoBean: my Destroy() ");
```

Validating
Bean Properties
Value

```
{ age = 0; } nullifying  
{ msg = null; } Bean properties
```

```
}
```

// B.method implementation

```
public String sayHello()
```

```
{ return "msg=" + msg + "age=" + age;
```

```
}
```

```
}
```

<!-- Spring Cfg.xml -->

<!DOCTYPE ----->

<beans

<bean id="db"

class="DemoBean"

init-method="myInit"

destroy-method="myDestroy"

<property name="age" value="30"/>

<property name="msg" value="hello"/>

</bean>

</beans>

// DemoClient.java

Same as first application (can use any container)

```
public class DemoClient
```

```
{ p.s.v.m (String[] args)
```

```
{
```

```
// Activate Spring Container
```

```
ClassPathXmlApplicationContext ctx = new ClassPathXmlApplicationContext("Spring Cfg.xml");
```

```
// get Spring Bean class object
```

```
Demo bobj = (Demo) ctx.getBean("db");
```

```
// call B.method
```

```
S.o.p (bobj.sayHello());
```

```
// class container
```

```
ctx.close();
```

x) if container is ApplicationContext Container call ctx.close(); at the end.

A) if the container is BeanFactory container then call factory.destroySingletons();

→ While Configuring pre-defined, third party supplied java classes as SpringBeans in Spring Configuration file Identification of lifecycle methods from huge no. of methods that are available in class is practically not feasible. moreover programmer may forget to configure these lifecycle methods.

To overcome this problem it is recommended to make SpringBean class implementing two special interfaces (InitializingBean, DisposableBean).

and we can use methods of these two interfaces as alternate for user-defined lifecycle methods.

org.springframework.beans.factory.InitializingBean interface gives afterPropertiesSet() method
↳ can be used as alternate to custom-init() method.

org.springframework.beans.factory.DisposableBean interface gives destroy() method
↳ can be used as alternate to custom-destroy() method.

→ While Working with these two methods there is no need of configuring method names in Spring Cfg file.

→ POJO class can act as SpringBean but it is not mandatory that every SpringBean must be a POJO class. When SpringBean class implements the above two interfaces then it acts as non-POJO class.

→ If SpringBean class implements above two interfaces and also contains Custom lifecycle methods then both will be executed. (Custom-init method executes after afterPropertiesSet(), Custom-destroy method executes after destroy()).

Example appn:-

// Demo.java same as previous appn

// DemoBean.java

public class DemoBean implements Demo, InitializingBean, DisposableBean

{
 // Bean properties

 private String msg;

 private int age;

setters & getters

```
public String sayHello() {  
    return "msg=" + msg + "age=" + age;  
}  
public void destroy() throws Exception  
{  
    S.o.p("DemoBean: destroy()");  
    age = 0;  
    msg = null;  
}  
public void afterPropertiesSet() throws Exception  
{  
    S.o.p("DemoBean: afterPropertiesSet()");  
    if (age <= 0 || msg == null)  
        throw new Exception("set proper values to Bean property");  
}  
}
```

<!-- DemoCfg.xml --> Same as previous appn. but don't specify init-method, destroy-method attributes.

// DemoClient.java Same as previous appn.

- If SpringBean class is not implementing InitializingBean, DisposableBean interfaces then only think about configuring certain methods as custom-lifecycle methods. It contains only Spring supplied Java classes.
- Since third party supplied classes, Jdk supplied classes cannot implements Initializing Bean, DisposableBean interfaces. so configure customlifecycle methods while working with ~~spring~~ those spring Beans.

- 1) Setter Injection
- 2) Constructor Injection
- 3) Interface Injection

XxxAware interfaces we can go for Interface Injection.

using interface injection we can inject only special obj to our SpringBean class properties

XxxAware interfaces are

- ① org.springframework.beans.factory.BeanFactoryAware
→ useful to inject BeanFactory container to our SpringBean class
- ② org.springframework.context.ApplicationContextAware
→ useful to inject ApplicationContext container to our SpringBean class
- ③ org.springframework.beans.factory.BeanNameAware
→ useful to inject beanid Spring bean to Springbean class

while performing Interface injection our SpringBean class acts as non-POJO class.

Example Application performing Interface Injection:-

//Demo.java ~~refer as previous app~~

NOTE:- Through Interface Injection we can inject Underlying Containers to SpringBean and we can use these containers to know about other beans being from current SpringBean class.

For example appn on Interface Injection refer Supplementary handout given on 01/03/2013

→ If you want to write BeanProperties Validation logic and Initialization logic outside the SpringBean classes without disturbing the source code of existing SpringBeans then we can use Special java class acting as **BeanPostProcessor**. for this the class must implement `org.springframework.beans.factory.config.BeanPostProcessor` (2)

→ This Interface contains 2 methods

① `public Object postProcessBeforeInitialization(Object bean, String bname)`
↓
represents SpringBean object ↓
represents Bean logical name

- executes after all Dependency Injection
- useful to modify Bean properties after dependency Injection

② `public Object postProcessAfterInitialization(Object bean, String bname)`
↓
represents SpringBean obj ↓
represents Bean logical name

- executes after custom-init method
- useful to modify Bean properties after Complete Initialization.

NOTE:- Custom `init()` method and `afterPropertiesSet()` related logics are specified to one Spring bean whereas the Bean post processor (the above two methods related logics are common for multiple SpringBeans).

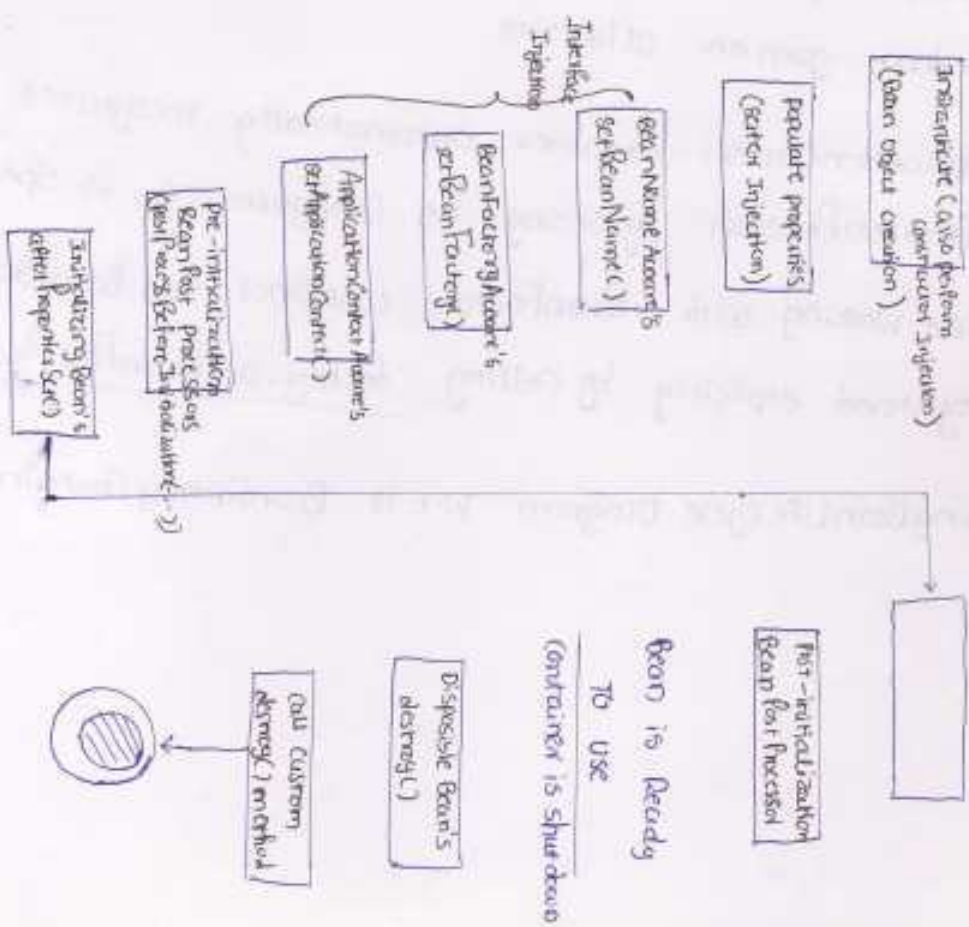
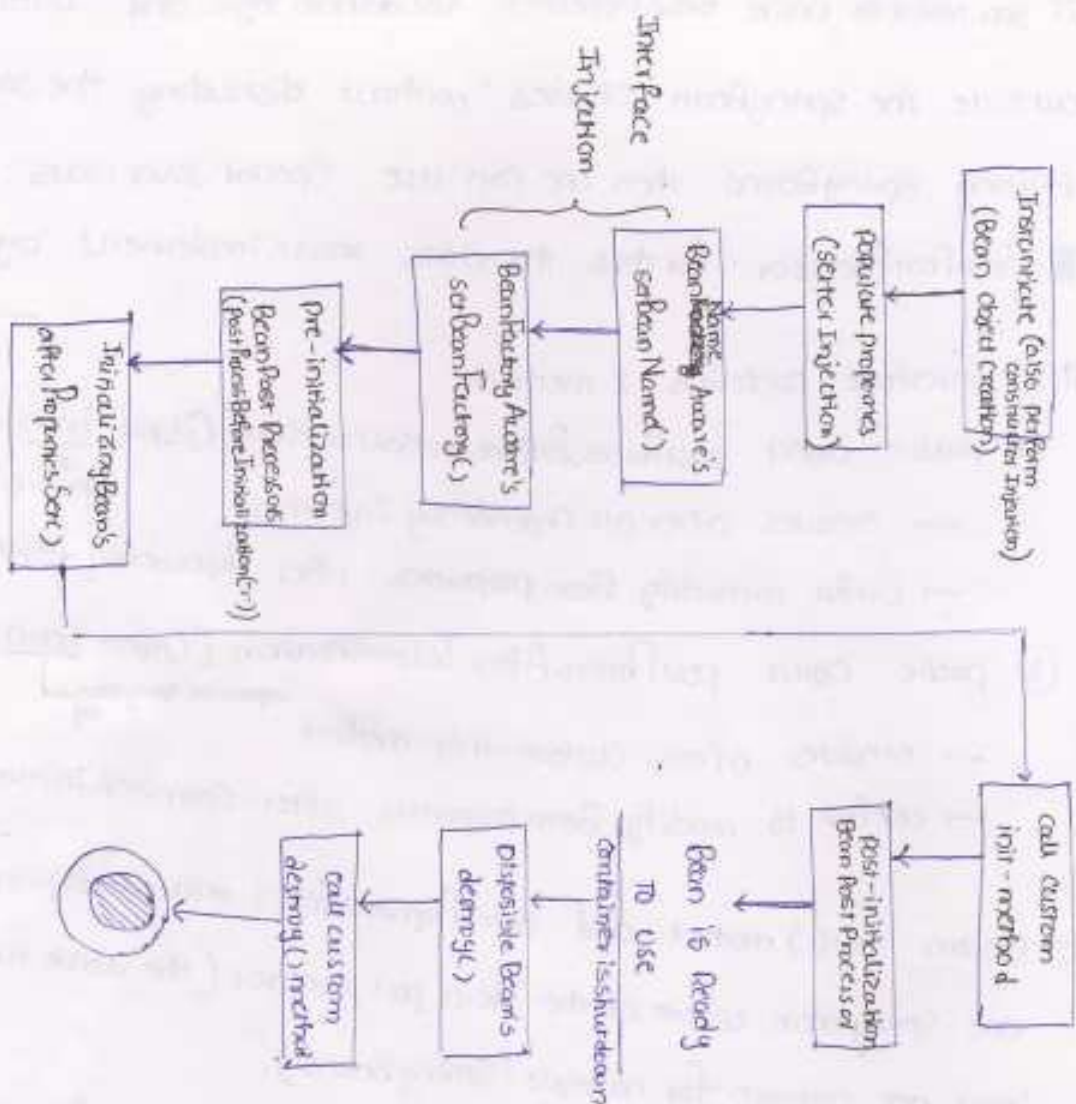
For Example application on BeanPostProcessor implementation refer the supplementary handout given on 01/03/2013

→ `ApplicationContext` container automatically recognizes and registers `BeanPostProcessor` by seeing its Configuration in Spring Configuration file.

While Working with `BeanFactory` container this `BeanPostProcessor` must be registered explicitly by calling `factory.addBeanPostProcessor()` method.

SpringBeanLifeCycle Diagram W.r to BeanFactory Container.

Bean life cycle Diagram w.r to Application Context Container



```

1 Spring App on Interface Injection and Bean Post Processor Impl
2 =====
3 -----Demo.java-----
4 public interface Demo {
5     public String sayHello();
6 }
7 -----DemoBean.java-----
8 import org.springframework.beans.BeansException;
9 import org.springframework.beans.factory.BeanFactory;
10 import org.springframework.beans.factory.BeanFactoryAware;
11 import org.springframework.beans.factory.BeanNameAware;
12 import org.springframework.context.ApplicationContext;
13 import org.springframework.context.ApplicationContextAware;
14
15 public class DemoBean implements Demo, BeanNameAware, BeanFactoryAware, ApplicationContextAware
16 {
17     //Special Bean properties
18     private String bname;
19     private BeanFactory factory;
20     private ApplicationContext ctx;
21
22     String msg;
23
24     //setXxx(-) for setter Injection
25     public void setMsg(String msg) {
26         System.out.println("DemoBean.setMsg(-)");
27         this.msg = msg;
28     }
29
30     //method of ApplicationContextAware()
31     public void setApplicationContext(ApplicationContext ctx)
32     throws BeansException {
33         System.out.println("DemoBean.setApplicationContext(-)");
34         this.ctx=ctx;
35     }
36     //method of BeanFactoryAware()
37     public void setBeanFactory(BeanFactory factory) throws BeansException {
38         System.out.println("DemoBean.setBeanFactory(-)");
39         this.factory=factory;
40     }
41
42     //method of BeanNameAware()
43     public void setBeanName(String name) {
44         System.out.println("DemoBean.setBeanName(-)");
45         this.bname=name;
46     }
47
48     public String sayHello() {
49
50         System.out.println("current bean id"+bname);
51         System.out.println("current bean is singleton?"+"ctx.isSingleton(bname)");
52         System.out.println("current beab is prototype?"+"ctx.isPrototype(bname)");
53         System.out.println("all beans are");
54         String id[]=ctx.getBeanDefinitionNames();
55         for(int i=0;i<id.length;++i)
56         {
57             System.out.println(id[i]+".....");
58         }
59         return "Good morning:msg="+msg;
60     } //method
61
62 } //class
63 -----DemoCfg.xml-----
64 <?xml version="1.0" encoding="UTF-8"?>
65 <beans
66     xmlns="http://www.springframework.org/schema/beans"
67     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
68     xmlns:p="http://www.springframework.org/schema/p"

```

Interfaces for Interface Injection

*methods supporting
Interface Injection*


```

69 xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-bean
70
71 <bean id="mp" class="MyProcessor"/> → BeanPostProcessor Configuration
72
73 <bean id="db" class="DemoBean" >
74   <property name="msg" value="hello"/>
75 </bean>
76
77 <bean id="dt" class="java.util.Date"/>
78 </beans>
79 -----DemoClient.java-----
80 import org.springframework.context.support.FileSystemXmlApplicationContext;
81
82 public class DemoClient {
83
84   public static void main(String[] args) {
85     //Activate Spring container
86     FileSystemXmlApplicationContext ctx=new FileSystemXmlApplicationContext(".\\src\\DemoCfg.xml");
87     // get Spring Bean class obj
88     Demo bob=(Demo)ctx.getBean("db");
89     // call B.method
90     System.out.println(bob.sayHello());
91   } //method
92 } //class
93 -----MyProcessor.java----- → BeanPostProcessor
94 import java.util.Date;
95 import org.springframework.beans.BeansException;
96 import org.springframework.beans.factory.config.BeanPostProcessor;
97 public class MyProcessor implements BeanPostProcessor {
98   @Override
99   public Object postProcessBeforeInitialization(Object bean, String bname) throws BeansException {
100     System.out.println("MyProcessor: postProcessBeforeInitialization()");
101     if(bean instanceof DemoBean )
102     {
103       DemoBean obj1=(DemoBean)bean;
104       obj1.msg="hello1";
105       return obj1;
106     }
107     if(bean instanceof java.util.Date)
108     {
109       Date obj2=(Date)bean;
110       obj2.setYear(130);
111       return obj2;
112     }
113     return bean;
114   }
115   public Object postProcessAfterInitialization(Object bean, String bname) throws BeansException {
116     System.out.println("MyProcessor: postProcessAfterInitialization()");
117     return bean;
118   }
119 }

```

Factory Bean:-

02/03/2013

→ If Normal Bean is configured as Dependent Bean to Bean property then Normal Bean class object will be injected directly to Bean Property.

→ Factory Bean is a bean that generates resultant object always. If Factory Bean is configured as Dependent Bean then Factory Bean class object will not be injected to property but the resultant Bean generated by Factory Bean will be injected.

Eg:-

```
<bean id="tb" class="TestBean"/>
<bean id="db" class="DemoBean">
  <property name="ti" ref="tb"/>
</bean>
```

if "TestBean" normal Spring bean then the "ti" property of DemoBean class will be injected with "TestBean" class obj.

if "TestBean" is Factory Bean then the "ti" property of DemoBean class will be injected with the resultant object given by TestBean class.

Spring Bean becomes Factory Bean when it implements `org.springframework.beans.factory.FactoryBean` (I).

This interface contains 3 methods

- ① `getObject()` → returns the resultant obj
- ② `getObjectType()` → returns the type of resultant obj
- ③ `isSingleton()` → returns whether resultant object is singleton or not

NetBeans

type: IDE slw for java

vendor: Sun/MS (Oracle corp)

version: 6.7.1

opensource

gives GlassFish 2.x as built-in server

to download slw: www.netbeans.org

for docs: www.netbeans.org

Resources of the application

- 1) Demo.java (Spring Interface)
- 2) DemoBean.java (Spring Bean class)
- 3) DemoCfg.xml (Spring of file)
- 4) DemoClient.java (client App)
- 5) TestBean.java (FactoryBean)

x) procedure to deal with Spring appn that deals with userdefined FactoryBean development by using netbeans 6.7.1:-

Step-I:- Create Java project in NetBeans IDE

File Menu → New → Project → java → javaapplication → ^{next} projectName SpringPkg
→ ☐ create main class → Finish

Step-II:- Add Spring Libraries to the project

Right Click ^{Libraries Folder of} on Project → add Library → Spring Framework 2.5

Step-III:- Add the above .java resources to ^{the} Source Pkgs folder of the project

Right click on Source pkg → New → JavaInterface → className: Demo

```
//Demo.java
public interface Demo {
    public String sayHello();
}
```

Step-IV:- Right click on Source pkg → New → Java class → className: DemoBean

```
//DemoBean.java
public class DemoBean implements Demo {
    {
        Object obj;
```

Right click → Insert Code → setters → OK

~~//Setter~~ Setter method supporting setter injection

```
public void setObj(Object obj)
{
    this.obj = obj;
}
```

```
public String sayHello() {
    // return "Good morning" + obj.toString();
    // return "Good morning" + obj.getClass().getName();
    return "Good morning" + obj.toString();
}
```

Step-V:- R.C. on Source ^{New Java} Package → New → Java class → className: TestBean

```
//TestBean.java
public class TestBean implements FactoryBean
{
    public Object getObject() throws Exception {
        return new java.util.Date(); // returns Date class obj as the Resultant obj
    }
}
```

```

public Class getObjectType() {
    return Date.class; → specifies the type of the Resultent Object
}

public boolean isSingleton() {
    return true; → specifies the resultent object scope is Singleton
}

```

Step-VI:- R.C on Source Package → New → Java Class → Name: DemoClient → Finish

```

//DemoClient.java
public class DemoClient
{
    public static void main (String[] args)
    {
        //Activate Spring Container
        ClassPath
        FileSystem Xml Application Context ctx = new
        ClassPath
        FileSystem Xml Application Context ("DemoCfg.xml")

        //get Spring Bean objects
        Demo bobj = (Demo) ctx.getBean("db");
        // bean id of DemoBean

        Object obj = ctx.getBean("tb");
        // bean id of TestBean
        System.out.println ("Obj data is " + obj);
        S.O.P ("Obj class name is " + obj.getClass());
        //Call B-methods
        String result = bobj.sayHello();
        S.O.P ("result is: " + result);
    }
}

```

Called to know the Resultent Obj of Factory Bean

R.C on Main → Properties → Run
 - main class: DemoClient
 Argument: db tb → Ok
 Rightclick on springmms

Step-VII:- Add Configuration file to the project

R.C on Source Pkgs → New → Other → XML → XML Document → Filename: DemoCfg
 → next → ☒ DTD - constraint Document → next →
 DTD public id: -//SPRING/DTD//Bean 2.0/EN
 DTD System id: http://www.springframework.org/dtd/spring-beans-2.0.dtd → Finish
 Document Root: beans

<beans>

<bean id="tb" class="TestBean"/>

<bean id="db" class="DemoBean">

<property name="obj" ref="tb"/>

</bean>

</beans>

Since TestBean is FactoryBean generating Date class object as Resultent Object, so, the obj will be injected with Date class object

Step III:- run the client application

Rightclick in Democlient.java → Run File.

The predefined SpringBean classes that are developed to gather objects from registries, pools and etc.. are designed as FactoryBeans.

Eg:-

① org.springframework.jndi.~~and~~ JndiObjectFactoryBean

TimerFactoryBean

RmiRegistryFactoryBean and etc..

To set Command line arguments to the Application of netBean IDE

RightClick on project → Properties → Run → Mainclass name: Democlient
arguments:

db	tb
args0	args1
args2	args3
args4	args5
args6	args7
args8	args9

Auto wiring:-

→ performing Dependency Injection on BeanProperties is called wiring.

There are two modes to performing this wiring.

① Explicit wiring:-

→ here bean properties will be configured explicitly for wiring using
<property>, <constructor-arg> tags.

② Auto wiring:-

→ here bean properties will not be configured explicitly But container

Automatically detects and injects values to bean properties

→ To enable Autowiring we need to use "autowire" attribute of <bean> tag.

Limitation with Autowiring:-

- ① Autowiring is not possible on Simple, String properties. Autowiring is possible only on reference type bean properties
- ② Autowiring may rise ambiguous Situation towards dependency Injections.
- ③ Autowiring kills the readability of Configuration file and Spring application

Ques We can make spring container performing autowiring in 4 modes.

① byName (autowire = "byName")

→ performs setter injection based on bean properties and dependent bean id's names matching.

② byType (autowire = "byType")

→ performs setter injection based on bean properties and dependent bean id's type matching

③ constructor (autowire = "constructor")

→ performs constructor injection by using parameterized constructors

④ autodetect (autowire = "autodetect")

→ performs either constructor injection or ^{byType} setter injection

→ IF Bean class contains ^{compiler generated (default constructor)} 0-param constructor then it performs ^{byType mode} setter injection otherwise it performs constructor injection

Example application:-

Demo.java → Same as previous appn (spring interface)

DemoBean.java (Spring Bean)

TestBean.java (Dependent Bean)

DemoCfg.xml (spring cfg file)

DemoClient.java (client appn)

// Demo.java

Same as previous

// DemoBean.java

public class DemoBean implements Demo

{

 // bean property

 TestBean tb;

 // 0-param constructor

 DemoBean()

 {

 s.o.p("DemoBean: 0-param constructor");

 }

// 1-Param Constructor

Shift + tab + space

Ctrl + Shift + I

DemoBean (TestBean tb)

```
{
    this.tb = tb;
    S.O.P("DemoBean: 1-Param Constructor");
}
```

// method for Setter Injection

public void setTb(TestBean tb)

```
{
    S.O.P("DemoBean: setTb(-) method");
    this.tb = tb;
}
```

public String sayHello()

```
{
    return "Good morning TestBean - msg = " + tb;
}
```

// TestBean.java

public class TestBean

```
{
    // Bean property
```

String msg;

// setter method for setter Injection

public void setMsg(String msg)

```
{
    this.msg = msg;
}
```

// implement toString() method

public String toString()

```
{
    return msg;
}
```

// DemoApp.xml

<beans>

<bean id="tb" class="TestBean">

<property name="msg" value="hello"/> —> Explicit wiring

</bean>

<bean id="db" class="DemoBean" autowire="byName"/>>

</beans>

↳ for this the DemoBean class Bean property (db) and TestBean class Beanid (tb) must match

// DemoClient.java

```
public class DemoClient {
```

```
    public static void main (String[] args)
```

```
    {
```

```
        // Activate Spring Container
```

```
        ClassPathXmlApplicationContext ctx = new ClassPathXmlApplicationContext("DemoClient.xml");
```

```
        // get Spring Bean obj
```

```
        Demo bobj = (Demo) ctx.getBean("db");
```

```
        // call b. method
```

```
        S.O.P (bobj.sayHello());
```

```
    }
```

```
}
```

→ The Beanid in spring configuration file must be unique id. otherwise SpringContainer may raise ambiguity ^{Publ/Gm} ~~State~~

autowire="byType" coding Springcfg file :-

```
// in DemoCfg.xml
```

```
<!DOCTYPE ....>
```

```
<beans>
```

```
<bean id="t1" class="TestBean">
```

```
    <property name="msg" value="hello"/>
```

```
</bean>
```

```
<bean id="db" class="DemoBean" autowire="byType"/>>
```

```
</beans>
```

↳ Here the Bean property "tb" type TestBean and the DependentBean obj "t1" type (TestBean) must match

Note:- When you Configure Same TestBean class for multiple times with different Beanid's then byType mode of autowiring may raise ambiguity problems

autowire="constructor" coding Springcfg file:-

```
// in DemoCfg.xml
```

```
<!DOCTYPE ....>
```


<beans>

<bean id="t1" class="TestBean">

<property name="msg" value="hello"/>

</bean>

<bean id="db" class="DemoBean" autowire="Constructor"/>

</beans>

↳ For this SpringBean class must have 1-param constructor having TestBean as the param type.

autowire="autodetect" coding Spring Configuration file :-

<beans>

<bean id="t1" class="TestBean">

<property name="msg" value="hello"/>

</bean>

<bean id="db" class="DemoBean" autowire="autodetect"/>

</beans>

- (*) IF DemoBean does not contain any explicit constructor then Spring Container performs "byType" mode setter injection otherwise Spring Container performs "Constructor Injection".

We can configure certain properties for explicit wiring and certain other properties for autowiring in one SpringBean class.

Q:- What happens if we configured both autowiring & explicit wiring on one Bean property.

Ans:- IF both autowiring & explicit wiring performs setter injection then the explicit wiring will effect (applied).

IF both autowiring & explicit wiring performs constructor injection then the explicit wiring will effect.

IF ~~both~~ autowiring performs setter injection and explicit wiring performs constructor injection ~~then~~ or vice versa then the wiring that perform setter injection will be effected.

Both containers (BeanFactory Container & Application Context container) supports autowiring.

- While using Spring with other Java technologies it is recommended to place the logic of activating Spring Container and getting SpringBean class object in one time execution block called constructor (or) static block
- ~~init() method~~ It is recommended to call Business method (or) SpringBean class object from respectively executing block.

<u>Client App</u>	<u>Place to activate Spring Container and to get SpringBean class obj</u>	<u>Place to call B-methods of Spring Bean</u>
AWT / Swing Frame	Constructor / static block	Event handling methods
Applet / JApplet	Constructor / static block / init()	Same
Servlet prg	Constructor / static block / init()	service(-,-) / doXXX(-,-)
JSP prg	<pre><%! public void jspInit() { ----- ----- } %></pre>	<pre><% ----- ----- %></pre>
Struts App	Constructor / static block of Struts Action class	execute(-,-,-,-) of Struts Action class
JSF App	Constructor / static block of Managed Bean	b-method of Managed Bean

- If Java webapplication uses SpringAPI (Third Party API) in the webresource programs then the SpringAPI related main.jar file should be added to CLASSPATH and SpringAPI related main and dependent jar files should be added to WEB-INF/lib folder.
- Jar files added to CLASSPATH will be used by Java compiler during the compilation of webresource programs (server pgs) to recognize Spring API. Jar files added to WEB-INF/lib folder will be used by Servlet container/JSP container to recognize and use Spring API.
- ResultSet object is not a Serializable object. So, we can't send ResultSet object over the network. To solve this problem there are two solutions.

Solution (1):- use RowSet instead of ResultSet

- RowSets are Serializable objects and we can send them over the network.
- Every few JDBC drivers support RowSets.

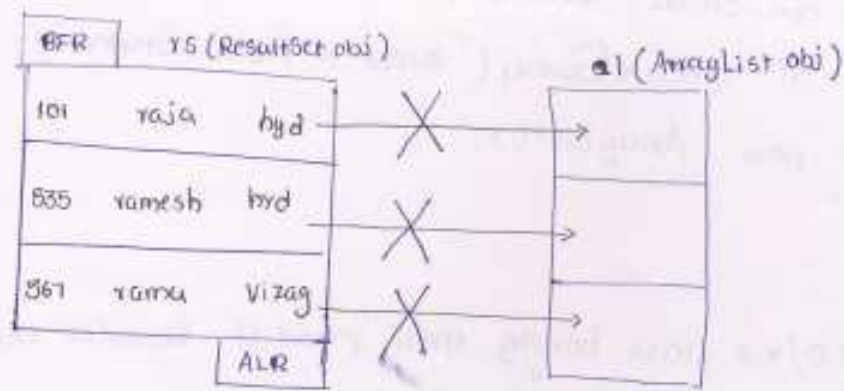
Solution (2):- copy the records of ResultSet obj to CollectionFramework DataStructure

- All Collection Framework DataStructures (like ArrayList and etc...) are Serializable objects. So we can send them over the network.

Solution (2) is good

- If you are looking to perform only Read operation then prefer working with Non-Synchronized Data Structures for Better performance. (Eg:- ArrayList, HashMap and etc...)
- If you are looking to perform both Read and Write operation simultaneously then prefer working with Synchronized DataStructure for Thread Safety. (Eg:- Vector, Hashtable and etc...)
- Even though network is not there it is not recommended to send ResultSet object from Destination Layer to Source Layer directly. It is recommended to keep the records of ResultSet in CollectionFramework DataStructure and send that DataStructure to Destination Layer.

Understanding problem to copy the records of ResultSet obj of ArrayList DataStructure



Each element of ArrayList allows only one object at a time. Each record of ResultSet contains multiple values including multiple objects. So, we can't copy each record of ResultSet to each element of ArrayList directly.

→ To solve the above problem keep each record values in UserDefined Java class object and add that object to each element of ArrayList as shown below

here, this user defined Java class is generally a JavaBean calling and it is called as V.O. class / D.T.O class
(Value Object class) (Data Transfer Object class)

Logic to Transfer ResultSet obj records to ArrayList elements using V.O. class / D.T.O class

StudentBean (D.T.O class / V.O class)

```
public class StudentBean implements java.io.Serializable
{
    // bean properties
    int sno;
    String sname;
    String sadd;
    // write setters & getters
    ...
}
```

NOTE:- Objects added to the elements of Collection Framework DataStructure must be taken as Serializable objects in order to send Collection Framework DataStructure over the network.

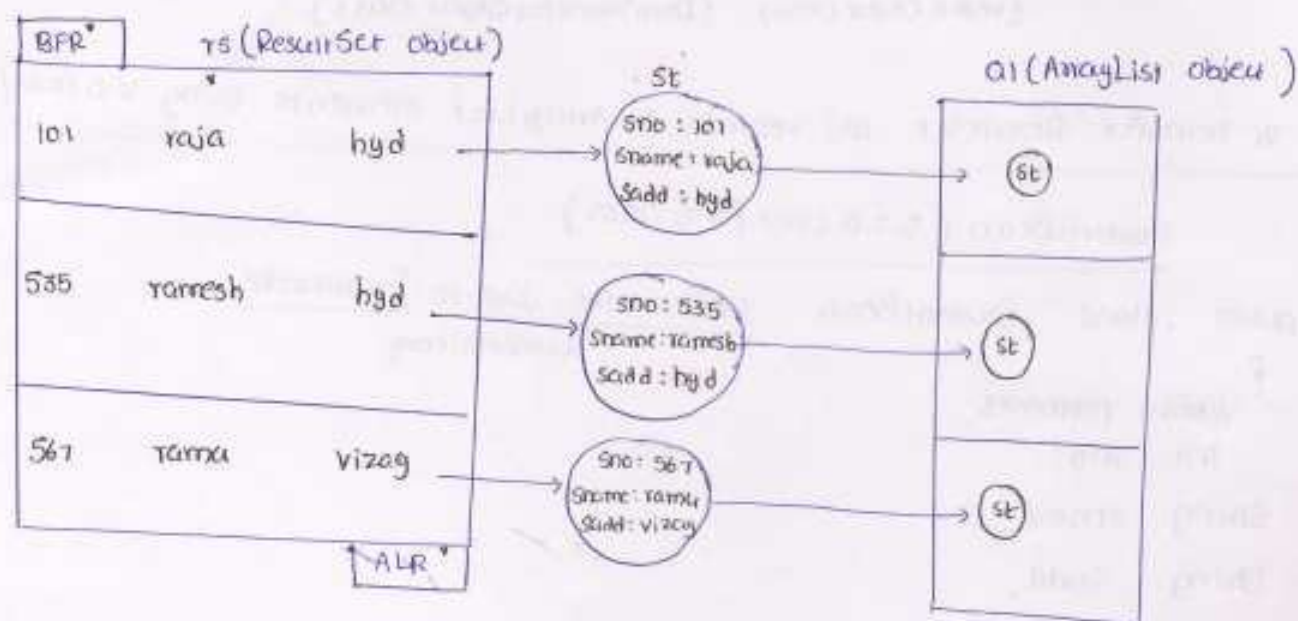
Logic to copy the records of ResultSet obj to ArrayList

```
Statement    st = con.createStatement();
ResultSet    rs = st.executeQuery("select * from Student");
ArrayList    al = new ArrayList();

while (rs.next())
{
    // create obj for D.T.O/V.O class having each record of ResultSet obj
    StudentBean st = new StudentBean();

    st.setSno(rs.getInt(1));
    st.setSname(rs.getString(2));
    st.setSadd(rs.getString(3));

    // add each D.T.O class / V.O class obj to the elements of ArrayList
    al.add(st);
} // while
```



→ Since, 'st' object are representing single big value by combining multiple individual values. ^(multiple simple values & multiple objs) So, it is called value Object and its class is called V.O. class.

Each record of
→ ResultSet Data is transferring from ResultSet to ArrayList in b/w. ^{1st} this record is storing one single big value object. This object is transfer the Data into ArrayList. This "st" object is called Data Transfer object and its class is called D.T.O. class.

Q:- Where did you use CollectionFramework DataStructures in your project?

Ans:- ① To send ResultSet object records over the network as discussed above

② while gathering JDBC properties from properties file we need java.util.Properties

③ To maintain JNDI properties we need Map DataStructures

④ To maintain Mail properties java.util.Properties class

⑤ In SessionTracking to maintain huge amount of data as SingleSession attribute value.

⑥ While developing gaming apps to maintain huge amount of Temporary data.

⑦ To maintain huge amount of data without DBs/DBs

⑧ To construct buffer and cache.

Q:- Where did you use JavaBean in your Projects?

Ans:- ① As V.O class / D.T.O class while copying records of ResultSet to CollectionFramework DataStructure

② As ModelLayer Resource while Developing MVC1 & MVC2 architecture based projects

③ As SpringBean in Spring Apps

As Hibernate persistent class in Hibernate apps

④ As Helper class in SessionTracking. (while maintaining huge amount of data as Single session attribute value).

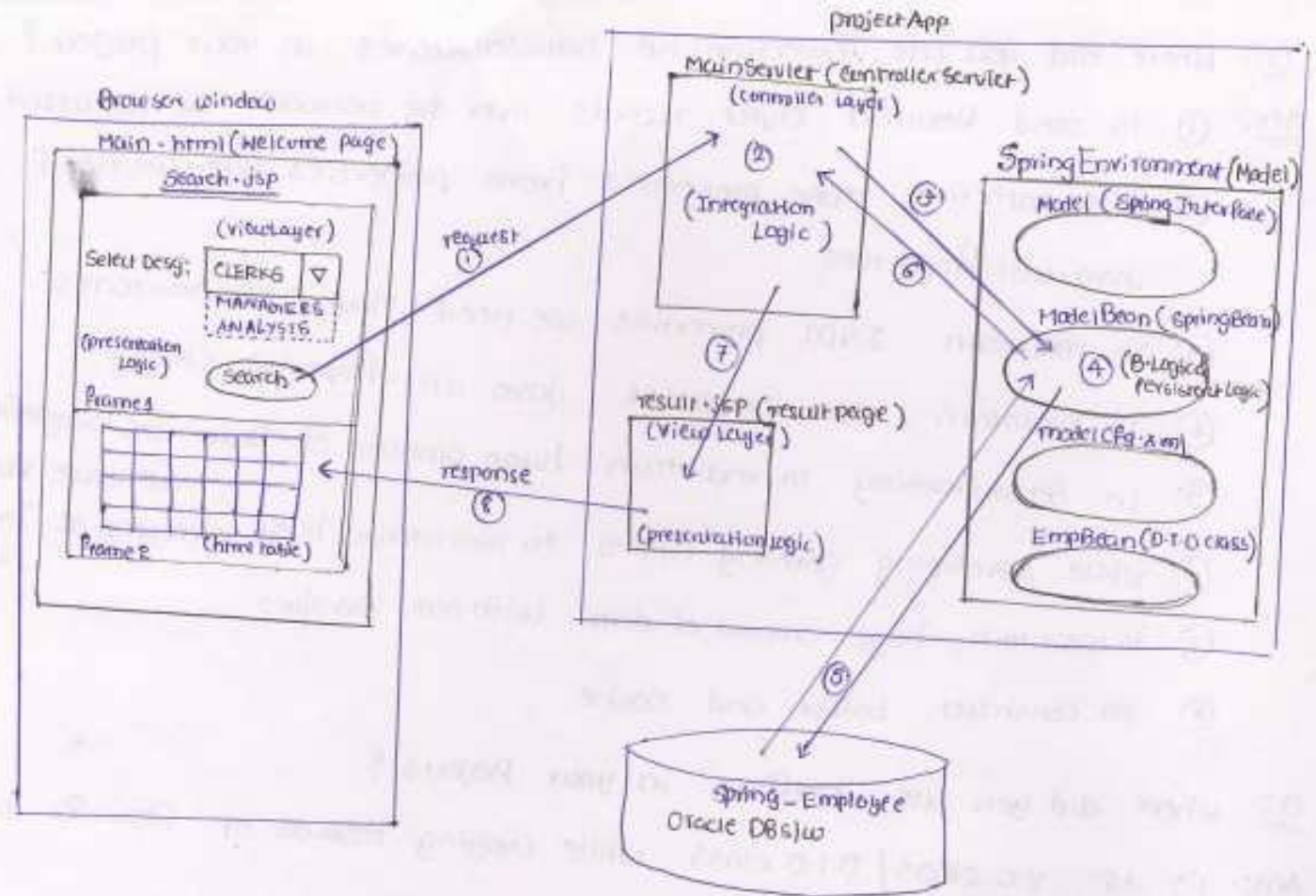
MiniProject info :-

→ It is based on MVC2 architecture

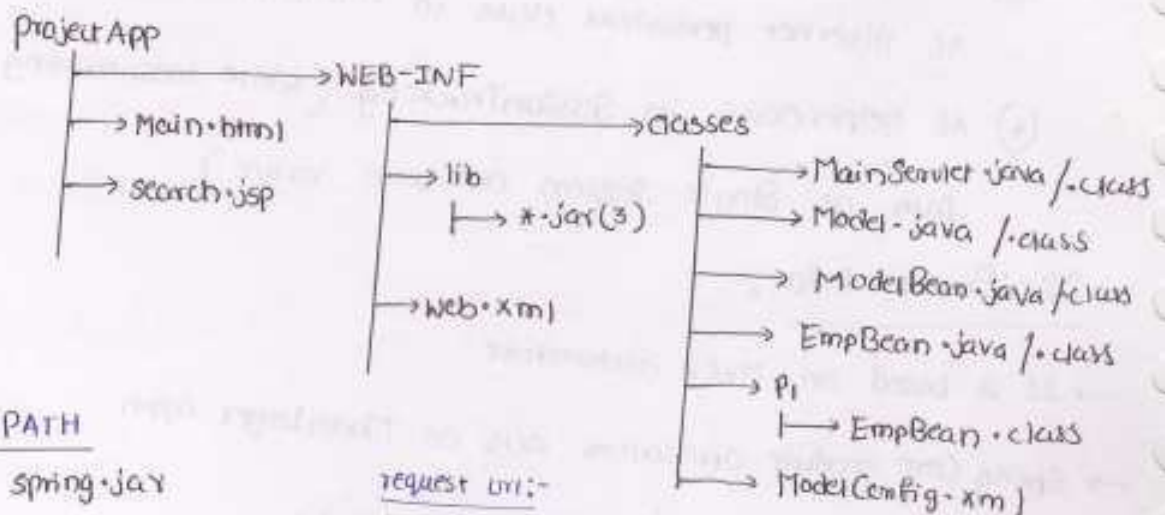
→ Spring Core module application acts as ModelLayer app

→ Jsp programs acts as view layer resources

→ Servlet program acts as Controller Layer resource



- Servlet program passes data to Result.jsp using request attributes.
- The SpringBean class (ModelBean) copies the records ResultSet object to ArrayList elements by using EmpBean class objects.



jar files in CLASSPATH

Servlet-api.jar, spring.jar

jar files in WEB-INF/lib folder

Spring.jar, Commons-logging.jar, ojdbc14.jar

request url:-

http://localhost:2020/ProjectApp/Main.html

- To use Java class in JSP ^{program} it must be placed in the package created in the same folder of JSP (or) in the userdefined package of WEB-INF/classes folder. That means JSP can't use Java class that is there directly in WEB-INF/classes folder.
- For the source code of above diagram refer appn (4) of the page no: (43) to (47)

Understanding Annotations :-

Annotations are java statement and they are alternate for the xml file based Metadata, Resource Configuration operations.

Syntax:-

(a) <Annotation_name> (param1=Val1, param2=Val2,)

↳ It is like xml tag name

↳ Like xml tag attributes

↳ It is like xml tag name

- ↳ like xml tag attributes

two types of annotations

① Annotations for Documentation

→ useful while generating java api documentation

@params @since @version @author @throws and etc...

→ introduced from jdk1.1

② Annotations for programming

→ useful for metadata, resource cfg operations

@Override @Autowired @Resource @Component and etc...

→ introduced from jdk-1.5

Metadata means data about data.

Resource cfg is also some sort of meta data operation

→ XML files give good flexibility of modification but Bad performance. bcz to access xml documents.

→ XML files give good flexibility of modification.
XML parsers are heavy weight slows to process xml documents.

→ Annotations give Bad flexibility of modification but good performance.

→ All Java Technologies that are designed based on Jdk 1.5+ are supporting

Annotations based programming.

Eg:- Spring 2.5+, Hibernate 3.2+, Struts 2.x, EJB 3.x, Servlet 3.x and etc....

→ we can apply Annotations at 3 level

- ① Resource Level (on class/Interface)
- ② Method level (on java method)
- ③ Field level (on Member variable)

To Configure Spring Bean we can use the following annotations :- 07/03/2013

a) @Component

→ makes java class as spring Bean and auto detectable from classpath for dependency injection

b) @Service

→ same as @Component but recommended to use for Service Layer Spring Beans (containing Business Logic)

c) @Repository

→ same as @Component but recommended to use for DAO's type Spring Beans (containing persistence logic)

To perform dependency injection cfgs

@Value → to inject Simple values, array values, List values

@Resource → to inject values to reference type bean properties

@Autowired → to enable autowiring on bean properties

note:- Still annotations not ready to cfg dependent values for Map, set, Properties type bean properties.

*) In spring environment we generally use both Xml files and Annotations together to perform Dependency Injections.

→ For Spring 2.5 based Autowiring refer appn D of the Supplementary handout given on 07/03/2013

→ If @Autowired annotation is placed on the top of setXxx(-) then it is for setter Injection based autowiring

If @Autowired annotation is placed on the top of param Constructor then it is for Constructor Injection based autowiring

If @Autowired annotation is placed on the top of Field then it goes for Field Injection based autowiring.

Spring 2.5 Application on Annotations (IOC) (Application 1)

-----Test.java-----

// Test.java (Spring Interface)

public interface Test

{

public String sayHello();

}

-----TestBean.java-----

//TestBean.java

import org.springframework.stereotype.*; ^{for} @Service annotationimport org.springframework.beans.factory.annotation.*; ^{for} @Autowired annotationimport java.util.*; ^{for} Date

@Service // we can also place @Component (makes the Java class as Spring Bean)

public class TestBean implements Test

{

// Bean property

Date d; ^{reference type Bean property.}

@Autowired //to perform auto wiring(by type) on bean property

public void setD(Date d)

{

this.d=d;

}

public String sayHello()

{

return "good morning "+d.toString();

}

}

-----DemoCfg.xml-----

<beans xmlns="http://www.springframework.org/schema/beans"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xmlns:aop="http://www.springframework.org/schema/aop"

xmlns:context="http://www.springframework.org/schema/context"

xsi:schemaLocation="http://www.springframework.org/schema/beans

http://www.springframework.org/schema/beans/spring-beans-2.5.xsd

http://www.springframework.org/schema/aop

http://www.springframework.org/schema/aop/spring-aop-2.5.xsd

http://www.springframework.org/schema/context

http://www.springframework.org/schema/context/spring-context-2.5.xsd">

<context:annotation-config /> ^{makes the spring container to recognize and use annotations}

<!--<context:component-scan base-package="." />-->

<bean id="tb" class="TestBean"/>

<bean id="dt" class="java.util.Date"/>

</beans>

-----TestClient.java-----

// TestClient.java

import org.springframework.context.support.*;

public class TestClient

{

public static void main(String[] args)

{

FileSystemXmlApplicationContext ctx=new FileSystemXmlApplicationContext("DemoCfg.xml");

Test beanobj=(Test)ctx.getBean("tb");

System.out.println("result is "+beanobj.sayHello());

}

//main

//class

-----Spring 3.x Application on Annotations(IOC)

-----TestInter.java-----

(Spring Interface)

Since we can't add @Service, @Component annotation

on the top of predefined java.util.Date class. so,

we must configure that class in springcfg file

as Spring Bean

Jar files in CLASSPATH :- spring.jar, commons-logging.jar
(Spring 2.5)


```

69 package p1;
70
71 public interface TestInter
72 {
73     public String sayHello();
74 }
75 -----TestBean.java-----
76 package p1;
77 import java.util.Date;
78 import java.util.List;
79 import javax.annotation.Resource;
80 import javax.annotation.Resource;
81 import org.springframework.beans.factory.annotation.Value;
82 import org.springframework.stereotype.Component;
83 import org.springframework.stereotype.Service;
84
85 @Component("tb") -> Bean id
86 public class TestBean implements TestInter
87 {
88     //Bean Property
89     UserBean u1; -> reference type Bean property
90
91     @Value("10") -> 91-92 injects value '10' to property 'no'
92     int no; -> Simple Bean property
93
94     Date d1;
95     @Value("#{T(java.util.Arrays).asList('India','Bharat','Hindustan')}")
96     String nicknames[]; -> Array type property
97     @Value("#{T(java.util.Arrays).asList('Red','Blue','Green')}")
98     List colors; -> Java util List type
99
100     -> performs Dependency Injection reference type Bean property
101     @Resource(name="dfb")
102     public void setD1(Date d) -> refer line no: 151
103     {
104         d1=d;
105     }
106
107     @Resource(name="ub") -> refer line no: 135
108     public void setU1(UserBean u1)
109     {
110         this.u1=u1;
111     }
112
113     public String sayHello()
114     {
115         System.out.println("no="+no);
116         System.out.println("d1="+d1.toString());
117         System.out.println("nicknames");
118
119         for(int i=0;i<nicknames.length;i++)
120             System.out.println(nicknames[i]);
121
122         System.out.println("Colors="+colors.toString());
123         System.out.println("u1="+u1);
124         return "Good Morning";
125     }
126 }
127 -----UserBean.java-----
128 package p1;
129
130 import org.springframework.stereotype.*;
131
132 @Service("ub") -> we can also use @Component annotation
133 public class UserBean

```

```

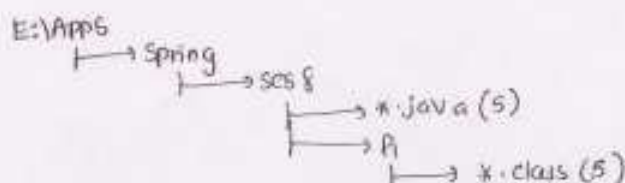
137 {
138     String msg="Hello World";
139
140     public String toString()
141     {
142         return "DemoBean.msg"+msg;
143     }
144 }
145 -----DateFactoryBean.java-----
146 package p1;
147
148 import org.springframework.beans.factory.FactoryBean;
149 import org.springframework.stereotype.Component;
150
151 @Component("dfb")
152 public class DateFactoryBean implements FactoryBean
153 {
154     public Object getObject() throws Exception {
155         return new java.util.Date(); → resultant object
156     }
157     public Class getObjectType() {
158         return java.util.Date.class;
159     }
160     public boolean isSingleton() {
161         return false;
162     }
163 }
164 }
165 -----TestClient.java-----
166 package p1;
167
168 import org.springframework.context.annotation.AnnotationConfigApplicationContext;
169 import org.springframework.context.support.*;
170
171 public class TestClient
172 {
173     public static void main(String s[])
174     {
175         AnnotationConfigApplicationContext ctx = new AnnotationConfigApplicationContext();
176         ctx.scan("p1"); //p1 is the package name
177         ctx.refresh(); → scans the every class of p1 package
178         TestInter test=(TestInter)ctx.getBean("tb");
179         System.out.println("result is"+test.sayHello());
180     }
181 } → To make container loading the classes
182 }

```

User Defined Factory Bean class returning
java.util.Date class object.

Annotations Based Application
Context Container

E:\Apps\Spring\ses8> javac -d . *.java
> java P1-TestClient



jar files in CLASSPATH

- 1) commons-logging.jar (collect from Spring 2.5)
- 2) org.springframework-3.1.0.RC1.jar
- 3) org.springframework-beans-3.1.0.RC1.jar
- 4) org.springframework-context-3.1.0.RC1.jar
- 5) org.springframework-context-support-3.1.0.RC1.jar
- 6) org.springframework-core-3.1.0.RC1.jar
- 7) org.springframework-expression-3.1.0.RC1.jar

import
General
Existing m...
win...

Folder :

→ For Spring 3.x annotations based Dependency Injection application based without using any xml files refer appn (2) of march 07th handout

→ Opening already creating project in MyEclipse IDE

File menu → import → General → Existing projects in workspace
→ ⓐ Select directory → Finish

NOTE:-

While working with AnnotationBased Dependency Injection, if Dependency Injection Configurations are done directly on BeanProperties then there is no necessity of writing setter (-) methods, Constructors related to Injections.

08/03/2013

More annotations of core module:-

- ⓐ Inject → to configure explicit dependency Injection
- ⓐ PostConstruct → to configure custom init method
- ⓐ PreDestroy → to configure custom destroy method
- ⓐ Scope → to specify SpringBean scope (single^{tn}, prototype)
- ⓐ ComponentScan → to recognize SpringBeans and resources that are there in packages
Spring configuration file.

* For Spring 3.0 based Explicit Constructor Injection through Annotations refer appn (1) of supplementary handout given on 08/03/13

x) For Spring 3.0 annotations based example application ^{that uses properties file} refer appn (2) of supplementary handout given on 08/03/13

If you want to know what jar files are added to the project goto .CLASSPATH. ^{an} there is the jar files.

x) For Spring 3.0 annotations based example appn custom-init, custom-destroy methods configuration refer appn (3) of the 08/03/13 handout (^{ⓐ PostConstruct,}
^{ⓐ PreDestroy})

* For Spring 3.0 annotations based FactoryBeanDevelopment and injects it resultant object to another BeanClass property refer appn (4) of the 08/03/13 handout

Same as appn (2) of

in file in CLASSPATH:- previous ④ jar files are

10/Jan / Spring 3 latest / Javax, inject, jar* / 7

here @value annotation injects the values to parameters and these parameter values are assigned to Bean properties.

- Activates the annotations based spring container.

jarfiles in CLASSPATH :- same as appn ①


```

69     private @Value("${db.pwd}")
70     String pwd;
71     @Override
72     public void readDBDetails() {
73         System.out.println("userName="+uname);
74         System.out.println("Password="+pwd);
75     }
76 }

```

↳ refers line no: 51

↳ B-method implemented

```

77 -----Springcfg.xml-----
78 <?xml version="1.0" encoding="UTF-8"?>
79 <beans xmlns="http://www.springframework.org/schema/beans"
80        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
81        xmlns:context="http://www.springframework.org/schema/context"
82        xsi:schemaLocation="http://cxf.apache.org/core http://cxf.apache.org/schemas/core.xsd
83        http://www.springframework.org/schema/beans
84        http://www.springframework.org/schema/beans/spring-beans.xsd
85        http://www.springframework.org/schema/context
86        http://www.springframework.org/schema/context/spring-context.xsd">
87     <context:annotation-config/>
88     <context:component-scan base-package="p1"/>
89     <context:property-placeholder location="/App.properties"/>
90     <!-- <context:property-placeholder location="classpath:/App.properties"/> -->
91 </beans>

```

↳ instructs to recognize annotations

↳ makes the container to look for the resources in 'p1' package

↳ recognizes the properties file of current directory (.)

```

92 -----SelectClient.java-----
93 package p1;
94 import org.springframework.context.support.ClassPathXmlApplicationContext;
95 public class SelectClient
96 {
97     public static void main(String[] args)
98     {
99         System.out.println("main():SelectClient");
100         ClassPathXmlApplicationContext ctx=new ClassPathXmlApplicationContext("SpringCfg.xml");
101         Select bobj=(Select)ctx.getBean("sb");
102         bobj.readDBDetails();
103     }
104 }

```

```

106 App3 (custom init-method and custom-destroy methods cfg)
107 -----Demo.java-----
108 package p1;
109 public interface Demo
110 {
111     public String sayHello();
112 }

```

```

113 -----DemoBean.java-----
114 package p1;
115 import javax.annotation.PostConstruct;
116 import javax.annotation.PreDestroy;
117 import org.springframework.beans.factory.annotation.Value;
118 import org.springframework.stereotype.Component;
119
120 @Component("db")
121 public class DemoBean implements Demo {
122     @Value("satya")
123     String msg;
124
125     @Value("10")
126     int age;
127     DemoBean()
128     {
129         System.out.println("DemoBean():constructor");
130     }
131
132     @PostConstruct
133     public void myInit()
134     {
135         System.out.println("Init-Method");
136         if(age<0){

```

↳ BeanId

↳ makes the myInit() method as custom -init() method

```

137     age=18;
138     System.out.println("Age cannot be -ve, it is set to 18");
139 }
140 }
141 @PreDestroy → make the myDestroy() method as custom-destroy()
142 public void myDestroy()
143 {
144     System.out.println("Destroy-method");
145     age=0;
146 }
147
148 public String sayHello() {
149     return "Hello ." + msg + "\n Your Age ." + age;
150 }
151 }
152 }

```

```

153 -----DemoCfg.xml-----
154 <beans xmlns="http://www.springframework.org/schema/beans"
155     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
156     xmlns:context="http://www.springframework.org/schema/context"
157     xsi:schemaLocation="http://www.springframework.org/schema/beans
158     http://www.springframework.org/schema/beans/spring-beans-2.5.xsd
159     http://www.springframework.org/schema/context
160     http://www.springframework.org/schema/context/spring-context-2.5.xsd">
161     <context:component-scan base-package="p1" />
162 </beans>

```

we can avoid this xml file
by using `AnnotationConfigApplicationContext`
Container and `ctx.scan("p1")` method
(refer march 7th/03/13 2nd appn)

```

163 -----DemoClient.java-----
164 package p1;
165 import org.springframework.context.support.ClassPathXmlApplicationContext;
166 public class DemoClient {
167     public static void main(String s[]) throws Exception {
168         ClassPathXmlApplicationContext ctx=new ClassPathXmlApplicationContext("DemoCfg.xml");
169         // ask spring container to give springbean class object
170         Demo d1 = (Demo) ctx.getBean("db");
171         // calls B.method of SpringBean class
172         String result = d1.sayHello();
173         System.out.println(result);
174
175         ctx.close();
176     }
177 }

```

App4 (on Factory Bean) *Factory Bean generating Date class obj representing the given Date value*

```

178 -----TestBean.java-----
179 package p1;
180 import java.text.SimpleDateFormat;
181 import java.util.Date;
182 import org.springframework.beans.factory.FactoryBean;
183 import org.springframework.beans.factory.annotation.Value;
184 import org.springframework.context.annotation.Scope;
185 import org.springframework.stereotype.Component;
186
187 @Component("tb") → Bean Id
188 @Scope("prototype") → for every Bean on separate Bean obj will be created
189 public class TestBean implements FactoryBean
190 {
191     @Value("21/12/2012") → mandatory
192     private String dt;
193
194     public Object getObject()
195     {
196         Date d=null;
197         try{
198             System.out.println(" getObject():TestBean");
199             SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
200             d = sdf.parse(dt);
201         }catch(Exception e)
202         {
203
204

```

logic to convert Spring Date value to
java.util.Date class object


```

205         System.out.println("Exception while Parsing DT"+e);
206     }
207     return d; // gives java.util.Date class object
208 }
209 public Class getObjectType()
210 {
211     System.out.println(" getObjectType():TestBean");
212     return java.util.Date.class;
213 }
214 public boolean isSingleton()
215 {
216     System.out.println("isSingleton():TestBean");
217     return false;
218 }
219 }
220 -----Demo.java-----
221 package p1;
222
223 public interface Demo
224 {
225     public String sayHello();
226 }
227 -----DemoBean.java-----
228 package p1;
229 import java.util.Date;
230 import javax.annotation.Resource;
231 import org.springframework.stereotype.Component;
232
233 @Component("db")
234 public class DemoBean implements Demo
235 {
236     @Resource(name="tb") // ref's line no : 189
237     Date dt;
238
239     public String sayHello()
240     {
241         return "The time is "+dt;
242     }
243 }
244 -----DemoCfg.xml-----
245 <?xml version="1.0" encoding="UTF-8"?>
246 <beans xmlns="http://www.springframework.org/schema/beans"
247        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
248        xmlns:context="http://www.springframework.org/schema/context"
249        xsi:schemaLocation="http://www.springframework.org/schema/beans
250        http://www.springframework.org/schema/beans/spring-beans-2.5.xsd
251        http://www.springframework.org/schema/context
252        http://www.springframework.org/schema/context/spring-context-2.5.xsd">
253     <context:component-scan base-package="p1" />
254 </beans>
255 -----DemoClient.java-----
256 package p1;
257 import org.springframework.context.support.ClassPathXmlApplicationContext;
258 public class DemoClient
259 {
260     public static void main(String[] args)
261     {
262         System.out.println("Hello");
263         ClassPathXmlApplicationContext ctx=new ClassPathXmlApplicationContext("DemoCfg.xml");
264         Demo bobj=(Demo)ctx.getBean("db");
265         System.out.println(bobj.sayHello());
266     }
267 }

```

JNDI :-

→ To provide global visibility to java objects and their references we keep them in registry slw. Every web server / Application Server gives on registry slw.

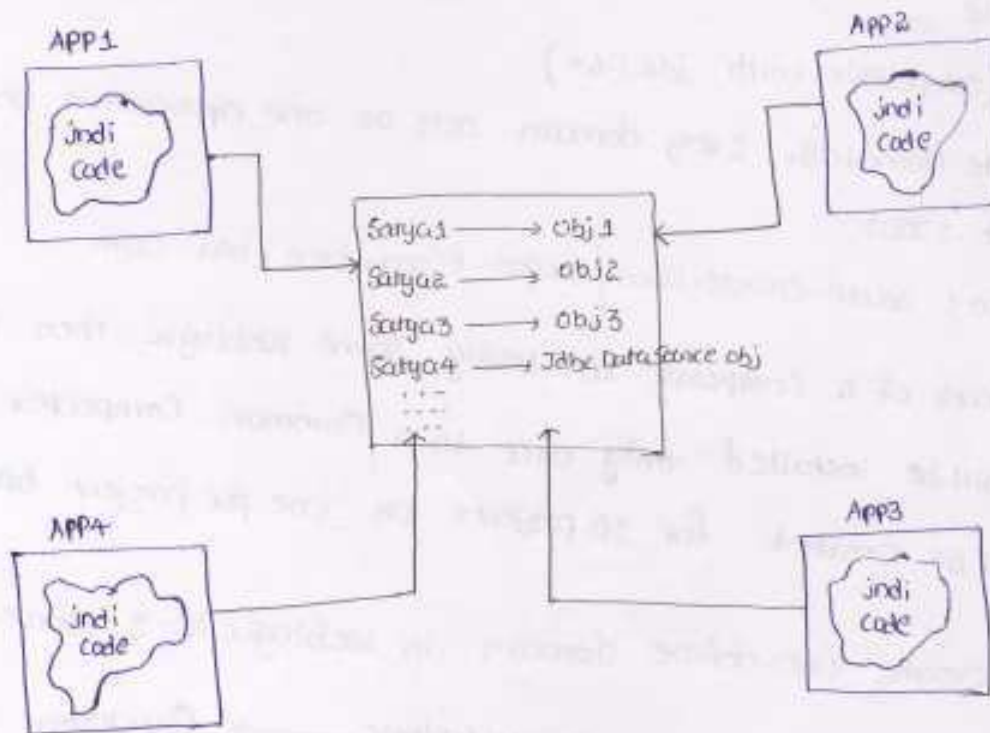
Weblogic gives Weblogic registry

Tomcat gives Tomcat registry

Glassfish gives Glassfish registry

JBoss gives Jnp registry and etc...

Java applications use jdbc api (javax.sql, java.sql pkgs) to interact with DBs.
 Java applications use jndi api (javax.naming and its sub pkgs) to interact with Registry.
 jdbc, jndi api are part of JSE module (jdk slw)



→ There are two types of registry slws

1) Naming Registry slw

→ Maintains info as key values pairs

→ we must know keys to get the values

Eg:- Telephone book (real life)

Rmi registry

Cor registry

and etc...

2) directory registry

→ Maintains info as key-value pairs and values can extra properties/attribute

→ we can use either keys or extra properties to get the values

Eg:- Yellow pages (category wise it give) } (real life)
Windows filesystem

Weblogic registry

GlassFish registry

DNS registry

and etc...

Weblogic

type : Application Server slw

Vendor : BEA Systems (Oracle Corp)

Commercial slw

Version : 10.3 (compatible with jdk 1.6+)

allows to create domains. Every domain acts as one application server

default port : 7001

to download slw : www.oracle.com/technologies/e-commerce/bea.com

If multiple projects of a company are using same weblogic then the weblogic slw will be installed only once in a common computer but 10 domains will be created for 10 projects on one per project basis.

*) procedure to create user define domain in weblogic 10.3 server

Step-I:- Start → programs → OracleWeblogic → Quickstart →

① clicking started with weblogic server → ② create a new weblogic domain

→ ③ generate a domain configured... → Next →

DomainName MyDomain1 → Next →

User Name javaboss
User password javaboss1
Confirm Password javaboss1

→ next → ☒ Administration Server

→ next → Listen port: 7171 → next → create

procedure to see the weblogic registry of Weblogic Server

Weblogic instance
Weblogic Domain

Step I:- Start the above MyDomain1 server

Start → programs → OracleWeblogic → user projects → MyDomain1
→ start admin server for weblogic

Step II:- Open admin Console of the above Domain Server

open Browser window → <http://localhost:7171/console>

Username: javaboss

Password: javaboss1 → Login

Step III:- View the weblogic registry

Admin Console → Environment → Servers → Admin Server → View Jndi tree

→ We can perform insert, Update, delete, ~~update~~ ^{and} Select jdbc operations on DB table by using jdbc code.

→ We can perform bind, rebind, unbind, lookup and list operations on Jndi registry by using jndi code.

bind → keeping obj with nickname/aliasname/jndi name in registry

unbind → removing object from registry

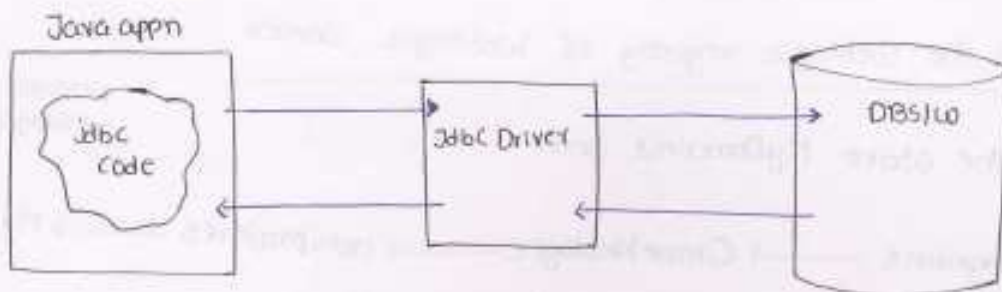
lookup → gathering object from registry with nickname/alias name

list → gathering all bindings from registry (nicknames, objs)

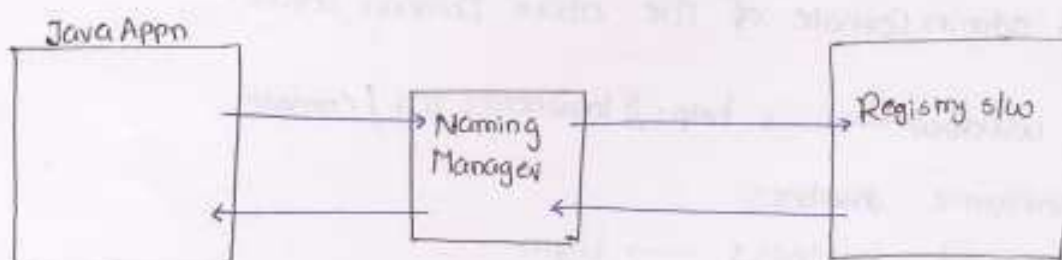
rebind → replacing existing object with new object

→ jdbc Connection Object represents Connectivity b/w java appn and DB s/w.

To create this con object we need jdbc properties (Driver class name, url, dbuser, db pwd) and these jdbc properties will change based on the jdbc driver, db s/w we use



JDBC Driver is the bridge b/w Java appn and DB s/w.



Naming Manager is the bridge b/w Java appn and registry s/w.

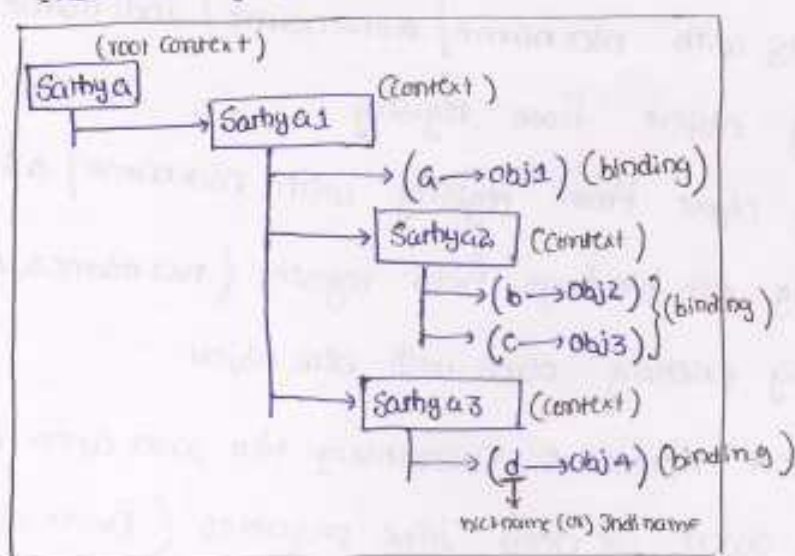
→ InitialContext obj represents connectivity b/w java appn and registry s/w

To create this InitialContext obj we need jndi properties they are

- 1) InitialContextFactory class
- 2) Provider url

these jndi properties will change based on the registry s/w we use.

Understanding Jndi Tree



Sathya1 is subcontext of Sathya context and it is parent context of Sathya2,

Sathya3

NOTE:- In realtime projects the DataSource object that represents JDBC

Connection pool will be placed in Registry also having nickname (or) alias name for global visibility.

Jndi properties of Weblogic Registry

InitialContext class name: weblogic.jndi.WLInitialContextFactory (Weblogic.jar)

provider url : t3://<hostname/ IPaddress> : <port no>

Eg:- t3://localhost:7171

(*) Write a Jndi appn to establish connection b/w Java appn and Weblogic registry.

// JndiConnTest.java

```
import javax.naming.*; // jndi api
import java.util.*;
class
```

public JndiConnTest

{
 public void m(String args[]) throws Exception

{

// Prepare Jndi properties

Hashtable ht = new Hashtable();

ht.put(Context.INITIAL_CONTEXT_FACTORY, "weblogic.jndi.WLInitialContextFactory");

↳ Fixed key

InitialContextFactory class name

ht.put(Context.PROVIDER_URL, "t3://localhost:7171");

// Create Initial Context object

InitialContext ic = new InitialContext(ht);

if (ic == null)

S.o.p("Connection is not established");

else

S.o.p("Connection is established");

} // main

} // class

// add Weblogic.jar (<weblogic_home>/middleware/wlserver_10.3/server/lib/weblogic.jar) in class path

javax.naming.InitialContext class gives the following method to perform jndi operations

bind(-,-)

unbind(-)

rebind(-,-)

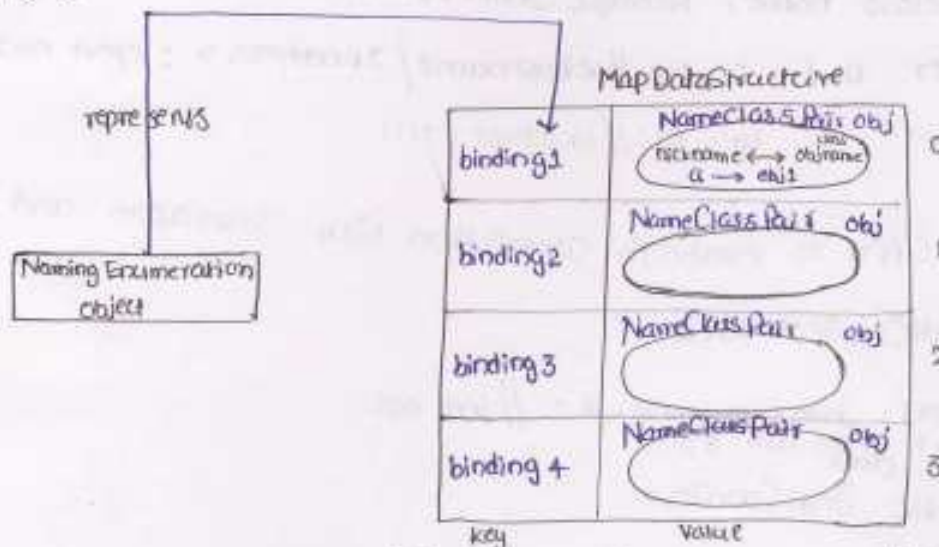
list(-)

lookup(-)

11/03/2013

JNDI API is

related to J2SE module
API.



Each NameClassPair class obj represents one binding (nickname, obj)
javax.naming.NamingEnumeration(I) is sub interface of java.util.Enumeration(I).

(*) Example application on JndiList operation.

// JndiList.java

import javax.naming.*;

import java.util.*;

public class JndiList

{
 p s v m (String[] args) throws Exception

{

 // Prepare Jndi properties

 Hashtable<String, String> ht = new Hashtable<String, String>();

 ht.put(Context.INITIAL_CONTEXT_FACTORY, "woblogic.jndi.WLInitialContextFactory");

 ht.put(Context.PROVIDER_URL, "t3://localhost:7171");

 ht.put(Context.SECURITY_PRINCIPAL, "javaboss");

 ht.put(Context.SECURITY_CREDENTIALS, "javaboss1");

} optional
while working
with root context

InitialContext ic = new InitialContext(ht); // points root context by default

//performing list operation

//Naming Enumeration ne = ic.list(""); //gives all the binding of root context

Naming Enumeration ne = ic.list("javax/jms"); //gives all bindings of javax/jms context

while (ne.hasMore())

{

NameClassPair ncp = (NameClassPair) ne.next(); //gives one binding at a time.

S.o.p (ncp.getName() + " ↔ " + ncp.getClassName());

}

gives nickname

gives class name of the object

} //main

} //class

- (*) For example application on Jndi operations refer TestJndi.java, TestJndi1.java of Page no's (57) to (59)

GlassFish

type: Application Server slw

Vendor: Sun/MS (Oracle corp)

Version: 2.x (Compatible with JDK 1.5+)

OpenSource

allows to create domains. default domain is domain1

default port no's : To access admin console 4343

To access webapplication: 8080

GlassFish slw that comes with NetBeans IDE slw installation can be used with or without NetBeans IDE.

Procedure to create User Defined Domain in GlassFish.

D:\sun\appserver\bin> asadmin create-domain --adminport=4343 --user=testuser mydomain2

D:\sun\appserver\bin> asadmin create-domain --adminport=4646 --user=testuser mydomain2

Procedure to see GlassFish registry of mydomain2 server

Step-I:- Start mydomain2 server

D:\sun\Appserver\bin> asadmin start-domain mydomain2

Step-II:- Open the admin console of mydomain2 server

Open Browser window → Type the following url

http://localhost:4846

Username: testuser

Password: testuser → Login

Step-III:- Give the glassFish registry.

Admin Console → Application Server → JNDI Browsing →

*) GlassFish server gives GlassFish registry as the built-in registry s/w.

JNDI properties of GlassFish registry

InitialContextFactory class name : com.sun.enterprise.naming^(*).SerialInitContextFactory

provider url : iiop://<host>:<port no>

eg:- iiop://localhost:4846

(*) This class is available in appserv-rt.jar and this jar file having multiple dependent jar files. They are

appserv-admin.jar
appserv-ee.jar
appserv-ext.jar
javaee.jar

available in

D:\sun\Appserver\lib folder

implmstr.jar → D:\sun\Appserver\lib\install\application\jmsra

(*) By changing the JNDI properties of TestJndi.java, TestJndi1.java files

we can perform JNDI operations on other registry s/w's (refer page no: 57 to 59)

iiop → inter internet ORB protocol

↳ Object Request Broker

JBoss

- Type: Application server slw
- Version: 5.x (Compatible with jdk 1.5)
- Vendor: Apache (Redhat)
- gives 5 default domains
 - default (default)
 - web
 - Standard
 - all
 - minimal
- to download slw: AS zip file from www.apache.org website
 Jboss - ~~5.1.0~~ 5.1.0.GA - jdk 1.5
- to install slw: extract zip file to a folder
- default port no: 8080 (to access the webapplications)
 1099 (to access the jnp registry)
- to start the server: use <Jboss-home>\bin\run.bat file
 (if problems are raised then remove findstr word from run.bat file content)

(*) procedure to see the Jnp registry of JBOSS 5.x server.

Step-I:- Start JBoss server

Step-II:- open admin console of JBOSS

open browser window → write following url
<http://localhost:8080/jmx-console>

Step-III:- Give the jnp registered related Jndi tree

admin console → Service = Jndiview → select "list" operation → invoke

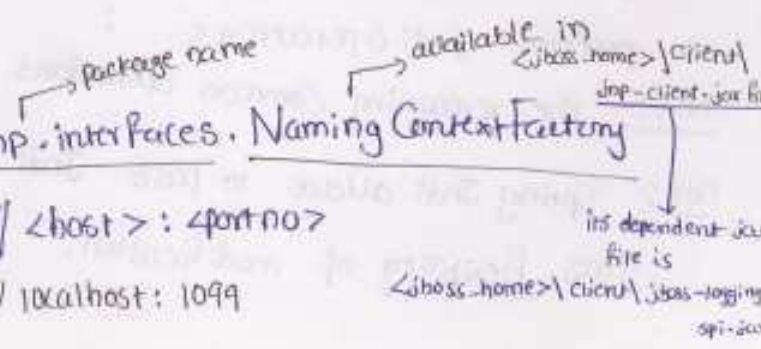
→ look at Global Jndi Name space

the Jndi properties of JNP registry:

InitialContextFactory class name: org.jnp.interfaces.NamingContextFactory

Provider url: jnp://<host>:<port no>

Eg: jnp://localhost:1099



jar files required in classpath to interact with Weblogic registry : weblogic.jar

jar files required in classpath to interact with GlassFish registry :

appserv-rt.jar

javaee.jar

appserv-admin.jar

appserv-ee.jar

appserv-ext.jar

imgjmsra.jar

jar files required in classpath to interact with JBoss registry :

jnp-client.jar

jboss-logging-spi.jar

Spring Jndi :-

- Spring Jndi provides abstraction layer on plain Jndi programming by giving `Org.springframework.jndi.JndiTemplate` class.
- This `JndiTemplate` class internally uses plain Jndi and simplify the process of performing Jndi operations on Registry s/w.

plain jndi programming

- prepare Jndi properties
- create InitialContext obj
- perform Jndi operations
- take care of Exception handling
- close InitialContext object

Spring Jndi programming

- get `JndiTemplate` class obj
- perform jndi operations

note:- the remaining common operations will be taken care by JndiTemplate class.

note:- Spring Jndi allows to pass Jndi properties from Spring Configuration file gives flexibility of modification.

→ SpringJndi internally uses plainJndi but it never makes programmer to bother about plainJndi (This is nothing but getting abstractionLayer). Client application of Spring application is java class. So that it can be taken as Spring Bean and its properties can be configured for Dependency Injection.

→ JndiTemplate class gives bind(-), rebind(-), unbind(-) and lookup(-) methods but it is not giving list().

⊗ for example appn on SpringJndi that uses JndiTemplate class refer page no: (59) & (60) appn no: (12)

→ Spring Technologies provides abstraction Layer on plain Technologies.

In this process if Spring API fails to support certain operation then we can use Spring supplied Callback Interfaces to implement that operation by using plain Technology API.

→ The interface whose methods implemented in implementation class called by underlying container automatically is called Callback interface.

Example Scenario:-

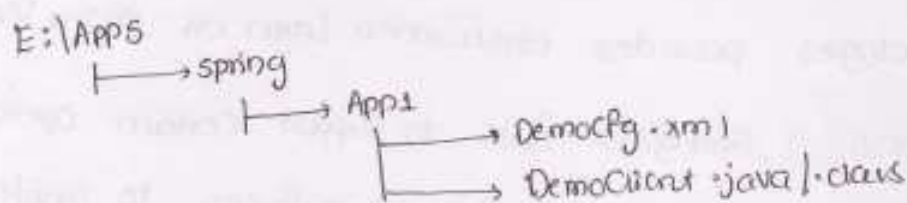
The SpringJndi supplied JndiTemplate class does provide methods to perform list operation on registry. This can be achieved by using JndiCallback interface. JndiCallback (I) gives doInContext(-) method. In this method we can write plain jndi code to perform list operation.

```
addWindowListener(new WindowAdapter() {  
    public void windowClosing(WindowEvent we)  
    {  
        System.exit(0)  
    }  
})  
// inner class  
// method call
```


In the above code:

- `AddWindowListner ( )` method is called, having one object as the argument value.
- That object is the Object of anonymous Inner class that extends from `WindowAdapter` class.
- This Anonymous Inner class is Overriding `WindowClosing ( )` method having some logic

(*) Example Application on to perform `IndiCallback` Interface to implement the list operation functionality.



`DemoCfg.xml` same as app(2) page no: 59

// `DemoClient.java`

```
import org.springframework.context.support.*;
import org.springframework.jndi.*;
import javax.naming.*; → plainJndi to work with Callback(I)
```

```
public class DemoClient
{
```

```
    static JndiTemplate template;
```

```
    // setter method for setter Injection
```

```
    public void setTemplate(JndiTemplate template)
```

```
    {
        this.template = template;
```

```
    }
```

```
    public static void main (String[] args) throws Exception
```

```
    {
        FileSystemXmlApplicationContext ctx = new FileSystemXmlApplicationContext(
            "DemoCfg.xml");
        // implement list operation... logic
    }
```

```
template <execute (new JndiCallback()) {
```

```
public Object doInContext (Context ctx) throws NamingException  
    { // use plain Jndi api to implement list operation logic
```

```
    NamingEnumeration ne = ctx.list("");
```

```
    while (ne.hasMore())
```

```
    {
```

```
        NameClassPair np = (NameClassPair) ne.next();
```

```
        S.o.p (np.getName() + " —————> " + np.getClassName());
```

```
    } // while
```

```
    return null;
```

```
    } // doInContext (-)
```

```
    } // inner class
```

```
}; // execute (-)
```

```
    } // main
```

```
}; // class
```

```
// > javac *.java
```

```
// > java DemoClient
```

```
// make sure that weblogic server is there in running mode.
```

→ In spring environment if ~~case~~ spring doesn't contain any method manually then we use Spring JndiCallback Interface to achieve that method

(*) template.execute () method is called by keeping Object of anonymous inner class implementing JndiCallback Interface

This anonymous inner class is defining doInContext (-) method having logic to perform List operation.

In Spring Application we can use 3 types of Jdbc con pools

① Spring's Built-in Connection (Working with DriverManager DataSource class)

→ It is not recommended to use because it does pool the con obj's

② Third party JDBC con pool

→ Apache DBCP

→ C3P0

(Recommended to use in standalone spring Apps (Applications that run outside the server)).

③ WebServer / Application Server Managed JDBC con pool

(Recommended to use the Spring apps are deployable in the server)

④ For example App spring's Built-in con pool utilization refer Feb 18th appn.

→ Apache DBCP

→ Type: Third Party

→ jar file: <spring-home> \lib\ jakarta-commons\ commons-dbcp.jar

→ class name: org.apache.commons.dbcp.BasicDataSource

important properties of BasicDataSource class

driverClassName

initialSize

password

url

username

⑤ Example application

E:\Apps

└── spring

└── Apache DBCP App

└── DBOperation.java

└── DBOperationBean.java

└── DBOperationClient.java

└── SpringCfg.xml

} same as
18/02/2013 appn

<!DOCTYPE >

<beans>

<bean id="dbcp" class="org.apache.commons.dbcp.BasicDataSource">

<property name="driverClassName" value="Oracle.jdbc.driver.OracleDriver"

<property name="url" value="jdbc:oracle:thin:@localhost:1521:orcl"

<property name="username" value="scott"/>

<property name="password" value="tiger"/>

<property name="initialSize" value="5"/>

</bean>

<bean id="dob" class="DBOperationBean">

<property name="ds" ref="dbcp"/>

</bean>

</beans>

Jar files in CLASSPATH:- jar files in classpath

① commons-dbcj.jar → <spring-home> \ lib \ commons-dbcj - folder

② spring.jar

③ commons-logging.jar

> javac *.java

> java DBOperationClient

C3P0

type: third party jdbc connection pool

jar file: <spring-home> \ lib \ C3P0 \ C3P0-0.9.1.2.jar

Class name that gives

important properties

password

user

minPoolSize

maxPoolSize

jdbcUrl

driverClass

Example application :-

Same as previous appn but perform the following operations.

① Add following entries in Spring Cfg.xml file.

"com.mchange.v2.c3p0.ComboPooledDataSource"

```
<beans>
  <bean id="dbcp" class="com.mchange.v2.c3p0.ComboPooledDataSource">
    <property name="driverClass" value="Oracle.jdbc.driver.OracleDriver"/>
    <property name="jdbcUrl" value="jdbc:oracle:thin:@localhost:1521:orcl"/>
    <property name="user" value="scott"/>
    <property name="password" value="tiger"/>
    <property name="minPoolSize" value="5"/>
    <property name="maxPoolSize" value="15"/>
  </bean>
  <bean id="db" class="DBOperationBean">
    <property name="db" ref="dbcp"/>
  </bean>
</beans>
```

add <Spring-home>\lib\c3p0\c3p0-0.9.1.2.jar file to CLASSPATH environment Variable as additional jar file.

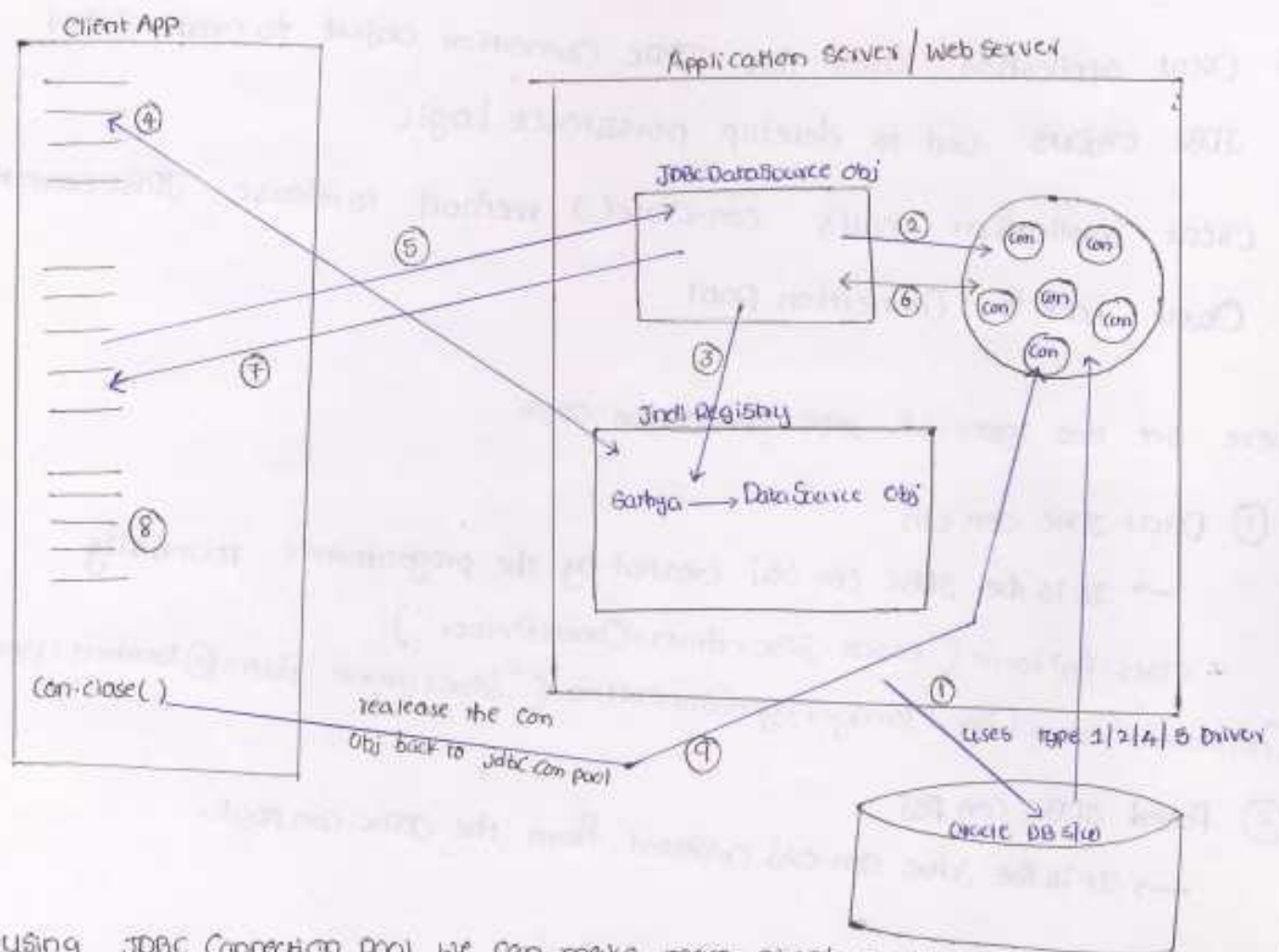
Q:- What is the JDBC connection pool that it have used in your project?

Ans:- If your spring project is standalone (or) Desktop project then use third party managed Connectionpool. Apache DBCP {c3p0 and etc..}

If spring project is Deployable webapplication then use Server managed JDBC Connectionpool

Understanding the process of working with ServerManaged JDBC ConnectionPool

DataSource object represents jdbc con pool and this DataSource obj will be placed in Registry slw for global visibility having nickname or alias name



using JDBC Connection pool we can make more client applications interacting with DB slw by using minimum no. of JDBC con obj's.

All Connection Obj's in Connection pool represents connectivity with same DB slw. Eg:- JDBC ConnectionPool for Oracle means all Connection obj's in that Connection pool represents connectivity with same Oracle DB slw.

W.r.to the Diagram,

- ① Server uses type 1/2/4/5 driver to interact with DB slw and to create JDBC Connection pool having JDBC Connection obj's.
- ② & ③ Server creates JDBC DataSource object representing JDBC Connection pool and keeps that Object in Jndi registry having nickname (or) alias name

- ④ Client application uses Jndi code to get DataSource object from registry.
- ⑤⑥⑦ client application gets JDBC Connection object from ConnectionPool through DataSource object.
- ⑧ Client application uses this JDBC Connection object to create other JDBC objects and to develop persistence Logic.
- ⑨ Client application calls con.close() method to release JDBC connection object back to Connection pool.

There are two types of jdbc Connection obj's

① Direct JDBC con obj

→ It is the JDBC con obj created by the programmer manually

```

class.forName("oracle.jdbc.driver.OracleDriver");
Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:orcl");
  
```

② Pooled JDBC con obj

→ It is the jdbc con obj collected from the jdbc con pool.

⊗ procedure to create JDBC Connection pool for oracle in mydomain1 server of Weblogic.

Step-I:- Start mydomain1 server and open its admin console.

Step-II:- create JdbcDataSource representing JDBC Connection pool for oracle.

Admin Console Screen → Services → JDBC → DataSources →

new → Name: → Jndiname: → Database type:

DB driver: → next → next →

Database Name: Hostname: port: Database Username:

Password: Conf Password: → next → Test Configuration

→ next → ☒ AdminServer → Finish.

Specify the JDBC Connection parameters

Admin Console → Services → JDBC → DataSources → myds1 →

Connection Pool tab → Initial capacity → Max capacity

Capacity increment: → advanced → Shrink frequency Seconds

→ Save.
no. of connect objects that should be created when there is a need of creating new JDBC Connection objects.

destroys Idle JDBC Connection objects after every 900 seconds.

NOTE:- in the above steps when Finish button is clicked the JDBC DataSource object reference will be placed automatically in Registry s/w.

* Org.sf.jndi.JndiObjectFactoryBean is the FactoryBean that gathers object from registry s/w through Jndi look-up operation and injects that object to specific Bean property. This class is very useful to gather DataSource object from Registry s/w and to inject that object to our BeanClass property.

Example Spring application to use the above weblogic server managed JDBC Connection pool

Step 1:- Keep mydomain1 Server of weblogic in running mode.

Step 2:- Keep Feb 18th appn ready and perform the following modifications in SpringCg.xml file.

// SpringCg.xml

<!DOCTYPE ... >

<beans>

<bean id="jofb" class="org.sf.jndi.JndiObjectFactoryBean">

<property name="jndiName" value="sathya"/>
↳ jndiname

Indi Properties to Establish the Connection with Registry.

```

<property name="IndiEnvironment">
  <props>
    <prop key="java.naming.factory.initial">weblogic.indi.WLInitialContextFactory</prop>
    <prop key="java.naming.provider.url">t3://localhost:7171</prop>
  </props>
</property>
</bean>

```

```

<bean id="dob" class="DBOperationBean">
  <property name="ds" ref="jofb"/>
</bean>
</beans>

```

Injects JDBC DataSource Object gathered from Indi registry by using IndiObjectFactoryBean, its resultant object will be injected

Step-3:- Run the client application

```
> java DBOperationClient
```

Jar files in classpath:-

spring.jar

commons-logging.jar

weblogic.jar

(*) procedure to create JDBC Connection Pool for oracle in mydomain2

Server of GlassFish 2.x.
Step-0:- keep oracle thin driver required ojdbc4.jar file <GlassFish-home>/application/domains/mydomain2/lib/external folder

Step-1:- Start mydomain2 server & open its admin console. (asadmin Start-domain mydomain2)

Step-2:- Create JDBC Connection Pool for oracle.

Admin Console Screen → Resources → JDBC → Connection Pools →
 → New → Name: mypool1 → resource type: javax.sql.DataSource
 Vendor: oracle → next → Initial and min pool size: 4 →
 Maximum pool size: 20 → pool resize quantity: 2 → 30s

☒ DataBase name orcl
☒ Driver TYPE thin
☒ Password tiger
☒ Port Number 1521
☒ Server Name localhost
☒ Service Name localhost
☒ URL jdbc:oracle:thin:@localhost:1521:orcl
☒ User scott

→ Finish → select mypool1 → Ping → save

Step-III:- Create JDBC DataSource pointing to the above JDBC Connection pool.

AdminConsole Screen → resources → JDBC ^{Resources} → ~~Get~~ JDBC Resources

New → JNDI Name: Sathya ^{MyDsJndi} → poolname: mypool1 → OK
Keeps DataSource obj in Jndi Registry
 AppServer → Jndi Registry
 → see Jndi tree

* Step-IV Refer appn on 18/02/2013 modify SpringCfy.xml

to use GlassFish server managed Connection pool in Spring appn

Step①:- keep mydomain2 ^{server of} GlassFish in Running mode

Step②:- keep feb 18th appn ready as shown in previous application but change the ~~var~~ following values in SpringCfy.xml file.

- (a) change Jndiname to MyDsJndi from Sathya
- (b) change weblogic Registry properties with GlassFish Registry properties.
refer 11th march discussions.

Step③:- execute the client appn by adding following jar files

2 → Spring.jar, commons-logging.jar

6 → GlassFish registry jar file refer march 12th

Spring DAO (also called as Spring JDBC)

15/03/2013

- The Java class that separates persistence logic from other logics of the application and makes that logic as reusable logic is called DAO.
- DAO Design pattern is not related with Spring DAO module.
- Spring DAO module gives JDBC Template class to provide abstraction layer on plain JDBC programming that means simplifies the process of developing JDBC persistence logic.

Plain JDBC programming

- ① register jdbc driver with DriverManager service
 - ② Establish the connection with DB sql
 - ③ create JDBC statement obj
 - ④ send and execute SQL query in DB sql
 - ⑤ Gather results and process the results
 - ⑥ Close JDBC obj's
 - ⑦ performs Exception Handling
- Common logics of the application (same in all JDBC apps)
- Application specific logic (changes in every app)
- Common logics of the app (same in all JDBC apps)

In the above application programmer should take care of Common and Application specific logics.

Spring JDBC programming

- ① get JdbcTemplate class obj by supplying jdbc DataSource obj
 - ② send and execute SQL queries in DB sql
 - ③ Gather results and process the results
- Application specific logics

In the above app JdbcTemplate class will take care of Common logics so programmer just need to take care of only Application specific logics

→ to create JdbcTemplate class obj Jdbc DataSource obj is dependent obj.

→ dataSource is the important property of JdbcTemplate class (org.springframework.jdbc.core package)

The important methods of JdbcTemplate class to send and execute the SQL Queries :-

To execute non-select Queries

update(String qry)

To execute select SQL Queries

queryForInt(String qry) } When select query return numeric values
queryForLong(String qry) } Eg:- select count(*) from student.

queryForObject(String qry) } when select query returns single record.

Eg:- select * from student where sno=101

queryForList(String qry) } when select query returns multiple records.

Eg:- select * from student.

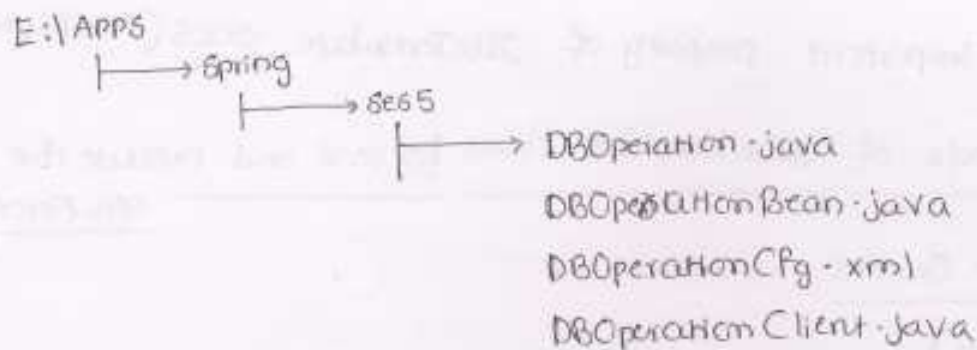
To perform Batch Processing :-

batchUpdate(String[] queries)

→ When all these methods are having single argument then Simple statement obj will be used internally.

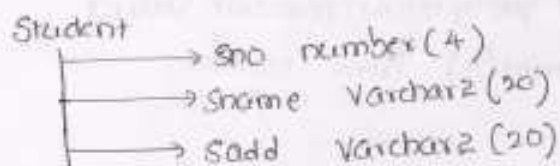
→ When all these methods are called having two arguments or three arguments then PreparedStatement obj will be used internally.

Example Application:-



Prefer using Third party jdbc con pool like Apache DBCP.

DB table in create



```
// DBOperation.java
import java.util.*;
public interface DBOperation
```

```
{
    public int insertInfo(int no, String name, String add);
    public int updateDetails(int no, String newname, String newadd);
    public int deleteDetails(int no);
    public int countStudents();
    public Map getStudentDetails(int no);
    public List getStudentDetails(String city);
}
```

→ BMAP to execute select query for single record.

→ List to execute select query for multiple records.

```
// DBOperationBean.java
```

```
import org.springframework.jdbc.core.*;
import java.util.*;
public class DBOperationBean implements DBOperation
```

```
{
    // bean property
    JdbcTemplate jt;
```

```
    // method for constructor injection
```

```
    public DBOperationBean(JdbcTemplate template)
```

```
{
    this.jt = template;
}
```

// implement B-methods

```
public int insertInfo (int no, String name, String address)
```

```
{
```

```
int result = jt.update ("insert into Student values(?,?,?)",  
                        new Object[] {no, name, address});
```

Supplies the query parameter values

```
return result;
```

```
}
```

```
public int updateDetails (int no, String new name, String new address)
```

```
{
```

```
int result = jt.update ("update student set sname=? sname sadd=? where  
sno=?", new Object[] {newname, newaddress, no});
```

Using PreparedStatement objects.

```
return result;
```

```
}
```

```
public int deleteDetails (int no)
```

```
{
```

```
int result = jt.update ("delete from student where sno=?", new Object[] {no});
```

```
return result;
```

```
}
```

```
public int countStudents ()
```

```
{
```

```
int result = jt.queryForInt ("select count(*) from student", new Object[] {});
```

```
return result;
```

```
}
```

```
public Map getStudDetails (int no)
```

```
{
```

```
Map m = jt.queryForMap ("select * from student where sno=?", new  
Object[] {no});
```

```
return m;
```

```
}
```

```
public List getStudentDetails (String city)
```

```
{
```

JDBC Template through
DataAccess Exception.

This is an unchecked
exception. So, exception
handling in DAO module
is optional.

AutoUnboxing
concept

Map Structure

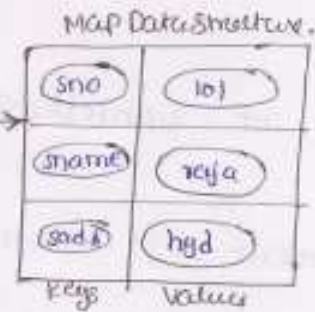
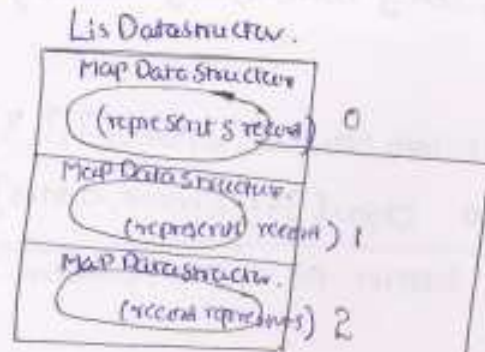
Column names	int	0	column values
	String	1	
	String	2	
keys	sno	id	
values	1234	5678	


```
List l = jt.queryForList("select * From Student where scdd = ?", new
Object[] {city});
```

```
return l;
```

```
}
```

```
}
```



```
// DBOperation Cfg.xml
```

```
<beans>
```

```
<bean id="dbcp" class="org.apache.commons.dbcp.BasicDataSource">
```

```
<property name="driverClassName"
```

```
.....
```

```
</bean>
```

```
<bean id="template" class="org.sf.jdbc.core.JdbcTemplate">
```

```
<property name="dataSource" ref="dbcp"/>
```

```
</bean>
```

```
<bean id="dob" class="DBOperation Bean">
```

```
<constructor-arg ref="template"/>
```

```
</bean>
```

```
</beans>
```

```
//DBOperation Client.java
```

```
import Org.sf.Context.Support.*;
```

```
public class DBOperationClient
```

```
{
```

```
    p s v m(String[] args)
```

```
{
```

// Activate Container

FileSystemXm Application Context ctx = new FileSystem^{Xm}ApplicationContext("DBOperation
cfg.xml");

// get Bean class obj

DBOperation bobj = (DBOperation) ctx.getBean("dob");

// call Methods

S.O.P("no. of Records that are inserted" + bobj.insertInfo(101, "rajä", "hyd"));

S.O.P("no. of Records that are Updated" + bobj.updateDetails(101, "neeraja", "neerajä"));

S.O.P("no. of Records that are deleted" + bobj.deleteDetails(101));

S.O.P("no. of students count" + bobj.~~count~~countStudents());

S.O.P("101 student details" + bobj.getStudDetails(101));

S.O.P("101 hyd student details" + bobj.getStudentDetails("hyd"));

} // main

} // class

jar files in CLASSPATH :-

2 Spring jars → spring-jar, commons-logging-jar

commons-dbcp-jar

OJdbc14.jar

Q:- What is the diff b/w executing select query in plain JDBC and in Spring JDBC?

Ans:- plain JDBC gives ResultSet object. and it is not Serializable object to send over the network.

Spring JDBC select query gives Results in List, Map Datastructures and we can send this Datastructures directly over the network (All collection framework Datastructures are Serializable objects by default)

Spring JDBC internally uses plain JDBC even though plain JDBC throws checkedException called SQLException. The Spring JDBC converts that one to Unchecked Exception called DataAccessException.

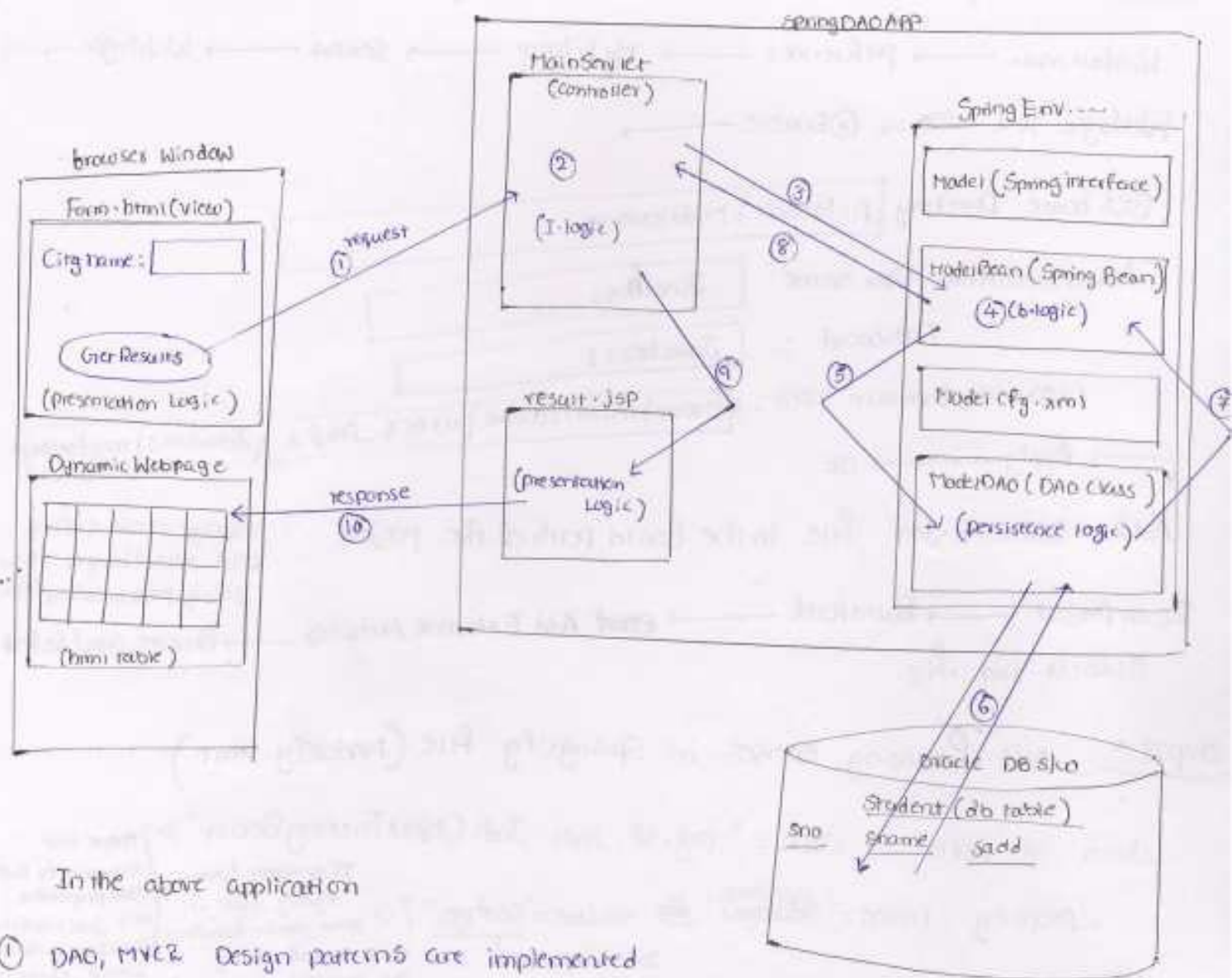
Eg:-

```
public int queryForInt(String qry) throws DataAccessException
{
    try
    {
        // plain jdbc code
    }
    catch (SQLException e)
    {
        throw new DataAccessException();
    }
}
```

Exception Rethrowing

The Java class that separates persistence logic from other logics of the application is called DAO. DAO makes the persistence logic as reusable logic and the flexible logic to modify.

Example webapplication based on mvc2 architecture and implementing DAO Design pattern



In the above application

- ① DAO, MVC2 Design patterns are implemented
- ② DAO class contain Spring JDBC based persistence Logic.
- ③ Use Server managed Connection Pool in the above application.

Design Pattern is a Best solution for recurring problems of application Development

Design Patterns are the Best practices to develop s/w applications by using s/w Technology

eg. (x) procedure to develop the above webapplication by using MyEclipse 10.x Server

Step-I:- Create web project

File → New → WebProject

Proj. Name: TestProj1 → Finish

Step-II:- Add spring capabilities to the project

Right click on project → MyEclipse → Add Spring capabilities → @Spring 3.0

→ ☒ Spring 3.0 Core, persistence Core, persistence JDBC Libraries → Spring Cfg

ModelCfg.xml → File → Finish

Step III:- Configure MyDomain1 server of weblogic server with MyEclipse JPE.

Window menu → Preferences → MyEclipse → Servers → Weblogic →

Weblogic 10.x → ☒ Enable →

BEA home Directory

Administration User name:

Password:

Execution Domain Root:

→ Apply → OK.

Add Osdbecl4.jar file to the Build path of the project.

Always start coding with Model layer or then going presentation layer.

RC on Project → Build path → ~~add~~ Add External Archives → Browse and select Osdbecl4.jar file.

Step IV:- Add following entries in SpringCfg file (ModelCfg.xml)

```
<bean id="jofb" class="org.sf.indi.IndiObjectFactoryBean">  
  <property name="indName" value="scutycu" />  
</bean>
```

Indi Name

If Application and registry reside in same server then there is no need of giving indi properties to talk with registry.

There are no need to supply indi properties. bcz indi properties and indi class with in the server same location.

```
<bean id="template" class="org.sf.jdbc.core.JdbcTemplate">  
  <property name="dataSource" ref="jofb" />  
</bean>
```

```
<bean id="mydao" class="ModelDAO">  
  <property name="jt" ref="template" />  
</bean>
```

```
<bean id="mb" class="ModelBean">  
  <property name="mdao" ref="mydao" />  
</bean>
```

Step-V:- Develop the DAO class as shown below

→ including only persistence logic

PC on Src → class → classname: ModelDAO → Finish.

```
public class ModelDAO {
    // bean property
    JdbcTemplate jt;
    // Write setXxx(-) method for setter Injection
    public void setJt(JdbcTemplate jt)
    {
        this.jt = jt;
    }
    public MapList getDetails(String city)
    {
        ListMapMap l = jt.queryForMapList("select * from student where saddr=?",
            new Object[] {city});
        return l;
    } // method
} // class
```

Step-VI:- Add SpringInterface, SpringBean class to the project

↳ Model1 ↳ ModelBean

PC on Src → New → Interface → Name: Model → Finish.

// Model.java

```
public interface Model {
    public MapList searchStudents(String city);
}
```

PC on Src → New → class → Name: ModelBean → add: Model → Finish.

```
public class ModelBean implements Model {
```

// bean property

ModelDAO mdao;

// setter method for setter Injection

```
public void setMdao(ModelDAO mdao)
```

```
{
    this.mdao = mdao;
}
```

```
public MapList searchStudents(String city) {
    // b.logic
}
```



```

if (city == null || city.equals("") City.equals(""))
{
    City = "hyd";
}

```

//use persistence logic of DAO class

```

List Map m = mdao.getDetails(city)
return m;
} //method
} //class

```

Step-III:- add Form.html to the webroot folder of the project

R.C ~~to~~ webroot $\rightarrow$ new $\rightarrow$ html $\rightarrow$ Name: Form.html

```

<form action="controller" method="get">
    City name: <input type="text" name="city" />
    <input type="submit" value="search details" />
</form>

```

Step-IV:- add servlet program to the project

R.C ^{on IDE} $\rightarrow$ new $\rightarrow$ Servlet $\rightarrow$ Name: MainServlet $\rightarrow$ ☒ init & destroy, ☒ doGet(), doPost()
 $\rightarrow$ next $\rightarrow$ Servlet/JSP mapping URI: /controller $\rightarrow$ Finish.

```

public class MainServlet extends HttpServlet {
    Model bobj;
    public void init() {
        //Activate SpringContainer

```

ClassPathXmlApplicationContext ctx = classPathXmlApplicationContext("ModelConfig.xml");
 //get SpringBean class object from Container

```

Model bobj = (Model) ctx.getBean("mb");
} //init()

```

```

Public void doGet (HSR request, HSR response) throws ServletException {
    //read form data
    String s1 = request.getParameter("city");
    //call B-method
}

```

```
List Map m = bobj.searchStudents(s1);
```

```
// keep result in Request attribute
```

```
request.setAttribute("attr1", m);
```

```
// Forward the Request to result page
```

```
RequestDispatcher rd = request.getRequestDispatcher("result.jsp");
```

```
rd.forward(request, response)
```

```
} // doGet(--)
```

```
public void doPost (HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException
```

```
{
```

```
doGet(request, response);
```

```
}
```

```
public void destroy() {
```

```
bobj = null;
```

```
}
```

```
}
```

Step-8i: add result.jsp to Webroot folder

Rec on Webroot → new → JSP → name: result.jsp

```
<%@page import="java.util.*"%>
```

```
<% List l = (List) request.getAttribute("attr1"); %>
```

```
<table>
```

```
<tr>
```

```
<th> sno </th>
```

```
<th> sname </th>
```

```
<th> sadd </th>
```

```
</tr>
```

```
<% for (int i=0; i<l.size(); ++i) %>
```

```
{
```

```
Map m = (Map) l.get(i); %>
```

```
<tr>
```

```
<td> <%= m.get("sno") %> </td>
```

```
<td> <%= m.get("sname") %> </td>
```

```
<td> <%= m.get("sadd") %> </td>
```

```
</tr>
```

```
<% %>
```

```
</table>
```


Step I:- Run the project

Run on Project → Run as → MyEclipse server application → Weblogic10.x

Test the project.

Open Browser window → <http://localhost:7171/TestProj1/Form.html>

Procedure to Configure glassFish 2.x Domain Server with MyEclipse IDE.

Window menu → preferences → MyEclipse → Servers → GlassFish →

GlassFish 2.x → ☒ Enable → Home directory: D:\sun\AppServer →

Configuration directory: D:\sun\AppServer\config. → Domain name: mydomain2

→ Apply → OK

→ If In the configuration file to change Jndiname to Sathya to mydsind;

If underlying server does not provide Server managed Connection pool then programmer can use Third party Connection pools in that Spring based web application.

→ Plain JDBC supports only positional parameters (?) in the query
Spring JDBC supports both position (?) and named parameters (:name) in the query.

→ Name Parameters are self descriptive, to work positional parameters we need JDBC Template class, to work with Named Parameter we need NamedParameterJdbcTemplate class.

* For Example application on NamedParameters refer page No: 54/55 Application ⑧

if certain persistent operations are not directly possible with the methods of JdbcTemplate class then we can implement those operations with callback interfaces support.

RowMapper → To process one record of the ResultSet obj

ResultSetExtractor → To process the multiple records of ResultSet obj

PreparedStatementCreator → gives an object to create and use JDBC PreparedStatement obj

PreparedStatementSetter → gives PreparedStatement obj to use it.

* For example application on SpringDAO Base Select operations and working with various callback interfaces refer application: (7) of the page no (51) to (54)

19/03/2013

→ In order to make certain Business Logic as reusable logic and centralized logic we can work with PL/SQL procedures and functions of Database s/w

→ PL/SQL procedures and Functions are visible in multiple applications of a module and in multiple modules of a project.

→ Using Spring JDBC we can call PL/SQL procedure or Function. For this we need to take a Java class extending from org.springframework.jdbc.core.StoredProcedureCreator and we use execute() to call the PL/SQL procedure (or) Function.

* For example application on calling PL/SQL procedure of Oracle DB s/w refer page no: (5) 64 (57) App no: (10)

→ plain jdbc and spring jdbc persistence Logic will be developed by using SQL Queries and these SQL Queries are Database s/w dependent queries so, ~~these~~ these persistence Logic are Database s/w dependent persistence Logics

→ to develop Objects based Database s/w independent persistence Logic without using SQL Queries then we need to work with ORM s/w's like Hibernate, JBosses, topink and etc...

→ Spring ORM module does not supply its own ORM s/w. but it provides abstraction layer on existing ORM s/w and simplifies the process of working with ORM s/w's.

For this it supplies multiple Template classes.

ORM 3/co

Hibernate
ibatis
Jdo
Jpa
TopLink

itsTemplate class

org.sf.orm.hibernate3.HibernateTemplate
org.sf.orm.ibatis.SqlMapClientTemplate
org.sf.orm.jdo.JdoTemplate
org.sf.orm.jpa.JpaTemplate
org.sf.orm.toplink.TopLinkTemplate

Org.sf.orm.hibernate3.LocalSessionFactoryBean gives SessionFactory obj. For this JDBCDataSource obj is required.

SessionFactory obj is required to create HibernateTemplate class obj.

Configuration in Spring cfg file

- Bean that gives JDBC DataSource obj
- Bean that gives HibernateSessionFactory obj (LocalSessionFactory)
- Bean ~~that~~ ^{of} gives HibernateTemplate class

important methods of HibernateTemplate class

for Single record operations:

save(-) / persist(-) → for record insertion
update(-) → for record updation
load(-) / get(-) → for selecting a record
delete(-) → for deleting record
saveOrUpdate(-) / merge(-) → for inserting / updating record.

for bulk operations:

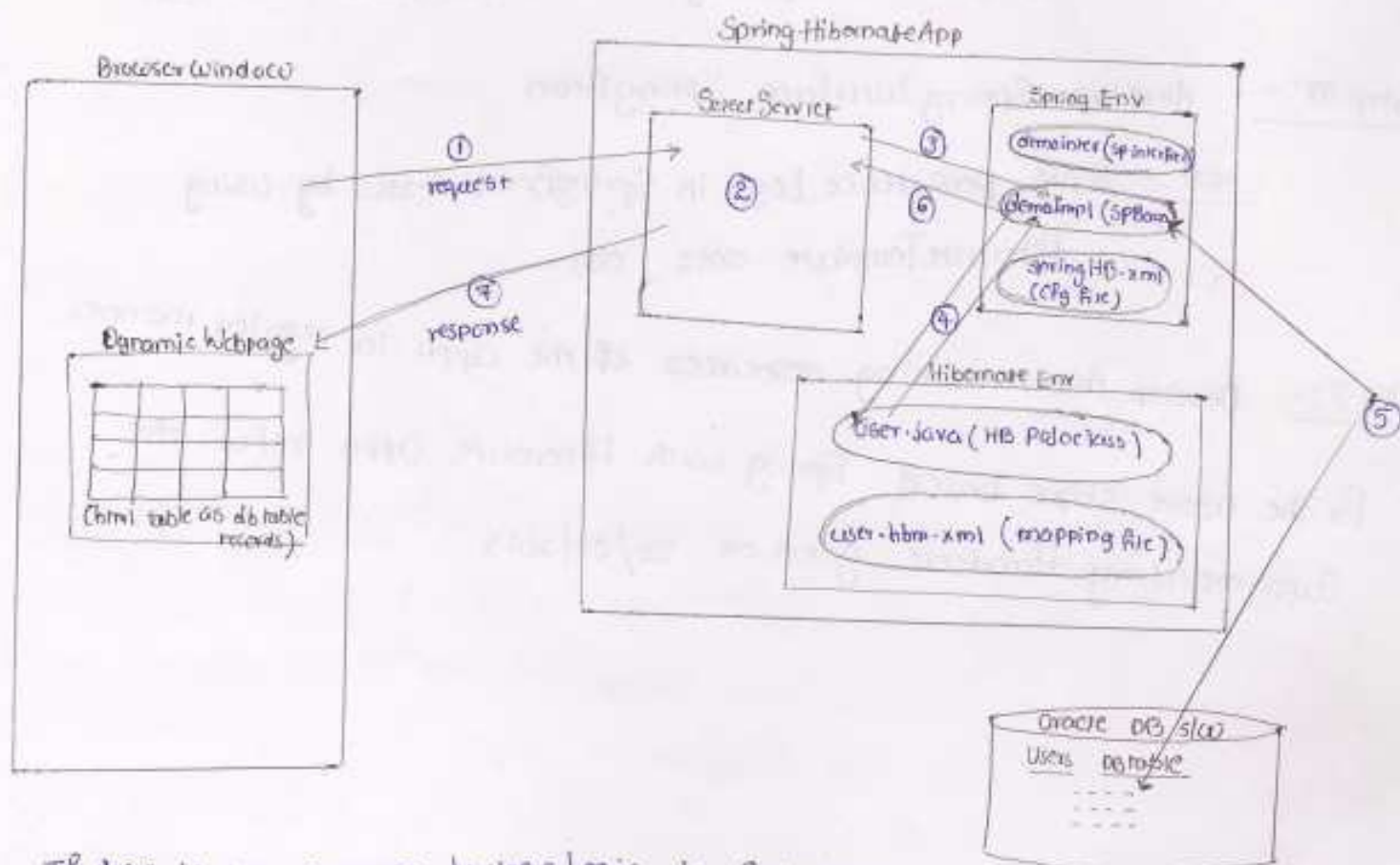
bulkUpdate(-) → for HQL Based non-select operation
find(-), findXxx(-) → for HQL Based select operation
iterate(), ~~iterate~~ → for HQL based select operations.

Module - III

Spring ORM Module

- In Spring with Hibernate Application the Spring Appn represents B-Logic and Hibernate Appn represents persistence Logic.
- Spring and Hibernate Integration is nothing but adding hibernate persistence logic in Spring appn.
- To write Hibernate ~~appn~~ persistence logic in SpringBean class by using Spring ORM module we need HibernateTemplate class Obj.

Spring with Hibernate Appn (using Hibernate Template class Obj)



If there is no separate business Logic in SpringBean then persistence logic itself acts as B-Logic of the SpringBean

In Spring with Hibernate application we can take separate Spring, Hibernate configuration files (or) we can take only Spring configuration file also maintaining the entries of Hibernate configuration.

Procedure to develop the above Spring with Hibernate application

Step-I:- Develop Hibernate pojo class, mapping file

Step-II:- Keep Spring and Hibernate related jar files in CLASSPATH and WEB-INF/lib folder

jar files in CLASSPATH: spring.jar, hibernate3.jar

jar files in WEB-INF/lib folder: 2 → spring jar files
8 → hibernate jar files

Step-III:- Cfg the following beans in Spring cfg file

- Bean giving JDBCDataSource object
- Local SessionFactory Bean giving Hibernate SessionFactory obj
- HibernateTemplate class as Spring bean
- Our Spring Bean injecting HibernateTemplate class obj.

Step-IV:- develop SpringInterface, SpringBean

Note:- Write persistence Logic in SpringBean class by using HibernateTemplate class obj.

Step-V:- Develop the remaining resources of the appn in regular manner.

For the above steps based Spring with Hibernate appn refer the
Supplimentary Handout given on 20/03/2013

The HQL Query that is placed in mapping file having logical name is called "Named HQL query".

and by Named Param.

* Executing Named Native ^{SQL} queries with positional parameters

Step-I:- write Native SQL query with positional parameters

```
<h-m>
<class --->
</class>
<sql-query name="test2">
    <return class="User"/>
    <![CDATA[ select * from users where userid >=? and userid <=? ]]>
</sql-query>
</h-m>
```

Step-II:- execute the above Native SQL query.

```
public Iterator getData()
{
    List l = ht.findByNamedQuery("test2", new Object[] {100, 300});
    Iterator it = l.iterator();
    return it;
}
```

* Executing Named Native SQL query with Named parameters.

Step-I:- prepare ^{Named} Native SQL query with Named Parameters in mapping file

```
<h-m>
<class --->
</class>
<sql-query name="test2">
    <return class="User"/>
    select * from users where role=:P1
</sql-query>
</h-m>
```

Step-II:- execute the above query

```
public Iterator getData()
{
    List l = ht.findByNamedQuery And Named Param ("test2", new String[] { "P1" }, new Object[] { "P1" });
    Iterator it = l.iterator();
}
```


// to insert single record

```
User u1 = new User();
```

```
u1.setUId(1001);
```

```
u1.setUname("Rajesh");
```

```
u1.setRole("P.L");
```

```
ht.save(u1);
```

```
return it;
```

```
}
```

While working with plain Hibernate all non-select persistent operations must be executed as Transactional statements. But in Spring Hibernate that is not required.

*)

```
public Iterator getData()  
List l = ht.find("findByNamedQuery And NamedParam", "test2", new String[]{"P.L"}, new Object[]  
Iterator it = l.iterator();
```

// to perform bulk modification using HQL

```
int res = ht.bulkUpdate("update User u1 set u1.role=? where u1.role=?",  
new Object[]{"P.L", "t.L"});
```

```
S.o.p("no. of records that are updated" + res);
```

```
return it;
```

→ HibernateTemplate class provides abstraction layer on plain hibernate and takes care the following internal operations.

- 1) Session obj creation
- 2) Transaction mgmt
- 3) Closing of SessionFactory, Session objs and etc....

*) The following operations are not possible directly with HibernateTemplate class.

- ① Non-select HQL queries with NamedParameters
- ② Non-select Native SQL queries with Named (or) positional parameters.
- ③ HQL select query with ~~Named parameters~~ Criteria API based logics execution
- ④ Native SQL select query with Both Named, positional parameters.

To implement all these facilities we need to work with HibernateCallback interface
org.springframework.orm.hibernate3.
Supplied by Spring ORM module. This interface gives doInHibernate(-) exposing the Session object.

```

1 -----
2 Title: Spring with Hibernate App (approach 2)
3 -----
4 ----- User.java ----- Hibernate persistent class (or) Entity class
5 public class User
6 {
7     private int uid;
8     private String uname, role;
9     public void setUid(int n)
10    {
11        uid = n;
12    }
13    public int getUid() { return uid; }
14    public void setUsername(String s) { uname = s; }
15    public String getUsername() { return uname; }
16    public void setRole(String r) { role = r; }
17    public String getRole() { return role; }
18 }
19
20 /*
21 create table users
22 ( userid number(5) primary key,
23   uname varchar2(20),
24   role varchar2(20)
25 );
26 */
27 ----- User.hbm.xml ----- Hibernate mapping file
28 <?xml version="1.0"?>
29 <!DOCTYPE hibernate-mapping PUBLIC
30   "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
31   "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
32 <hibernate-mapping>
33     <class name="User" table="users">
34         <id name="uid" column="userid" />
35         <property name="uname" />
36         <property name="role" />
37     </class>
38 </hibernate-mapping>
39 ----- demointer.java ----- Spring Interface
40 import java.util.*;
41 public interface demointer
42 {
43     public Iterator getData();
44 }
45 ----- demoimpl.java ----- Spring Bean class
46 import java.util.*;
47 import org.springframework.orm.hibernate3.*;
48
49 public class demoimpl implements demointer
50 {
51     private HibernateTemplate ht;
52     public void setHt(HibernateTemplate ht)
53     {
54         this.ht = ht;
55     }
56     public Iterator getData()
57     {
58         List l = ht.find("from User");
59         Iterator it = l.iterator();
60         return it;
61     }
62 }
63 ----- SpringHB.xml -----
64 <?xml version="1.0" encoding="UTF-8"?>
65 <!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"

```

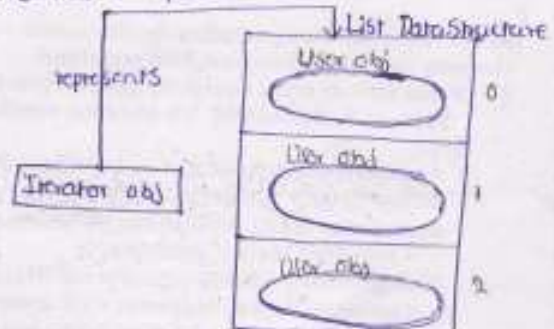


logic to inject

HibernateTemplate class object

represents

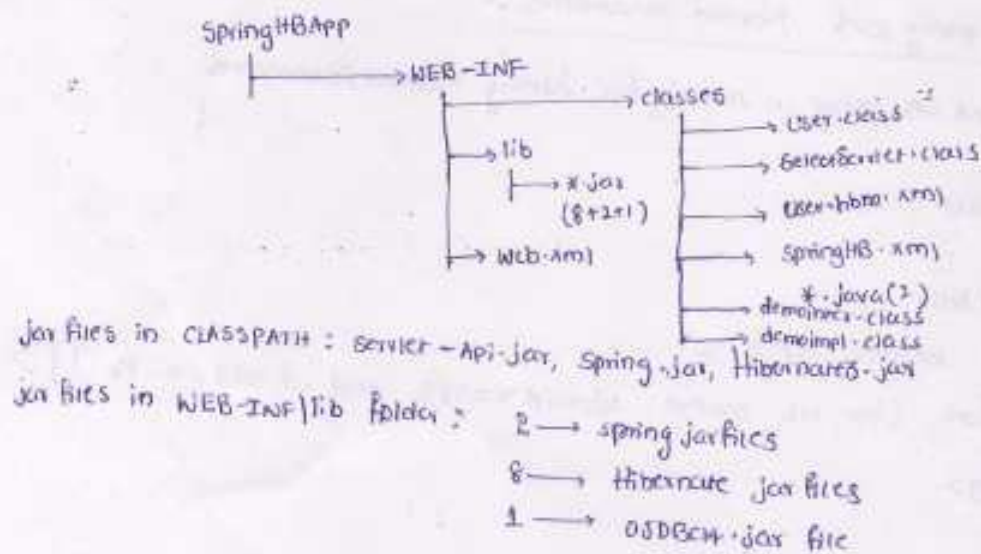
List Data Structure




```

139         User u1=(User)i1.next();
140         out.println("<tr><td>"+u1.getId()+" <td>"+u1.getUserName()+"
141                 <td>"+u1.getRole()+"</tr>");
142     }
143     out.println("</table>");
144 }
145 catch (HibernateException e)
146 {
147     out.println(e);
148     e.printStackTrace();
149 }
150 }
151 }
152 -----web.xml-----
153 <?xml version="1.0" encoding="ISO-8859-1"?>
154 <web-app xmlns="http://java.sun.com/xml/ns/j2ee"
155         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
156         xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
157         http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd"
158         version="2.4">
159     <servlet>
160         <servlet-name>select</servlet-name>
161         <servlet-class>selectservlet</servlet-class>
162     </servlet>
163     <servlet-mapping>
164         <servlet-name>select</servlet-name>
165         <url-pattern>/selectaction</url-pattern>
166     </servlet-mapping>
167 </web-app>
168

```



Make persistence operations by using HibernateTemplate class.

1) Executing HQL select query with positional parameters

```

public Iterator getData()
{
    List l = ht.find("from User where uid >=? and uid <=?", new Object[] {100, 200});
    Iterator it = l.iterator();
    return it;
}

```

positional parameters

parameters values

2) Executing HQL select query with Named parameters

```

public Iterator getData()
{
    List l = ht.findByNameParam("from User where uid >=:P1 and uid <=:P2", new String[] { "1", "200" });
    Iterator it = l.iterator();
    return it;
}

```

named parameters

parameters values

Executing Named HQL query with Positional parameters :-

Step-I:- Write Named HQL query in Hibernate mapping file

```
<h-m>
  <class --->
    ...
  </class>
  <query name="test1">
    from User ub where ub.role like ?
  </query>
</h-m>
```

Step-II:- execute the above HQL query.

```
public Iterator getData()
{
    List l = ht.findByNamedQuery("test1", new Object[] {"P1"});
    Iterator it = l.iterator();
    return it;
}
```

Executing Named HQL query with Named parameters :-

Step-I:- Prepare Named HQL query in mapping file having Named Parameters.

```
<h-m>
  <class --->
    ...
  </class>
  <query name="test1">
    <![CDATA[ from User ub where ub.uid >=:P1 and ub.wid <=:P2 ]]>
  </query>
</h-m>
```

Step-II:- execute the above HQL query.

```
public Iterator getData()
{
    List l = ht.findByNamedQueryAndNamedParam("test1", new String[] {"P1", "P2"},
                                                new Object[] {100, 300});
    Iterator it = l.iterator();
    return it;
}
```

Executing Hibernate Persistence Logic of Criteria API by using HibernateCallback (I) of Spring ORM module.

```
public Iterator getData()
{
    List l = (List) ht.execute(new HibernateCallback() {
        public Object doInHibernate(Session ses) {
            // Criteria API based persistence Logic
            Criteria ct = ses.createCriteria(User.class);
            // add Condition
            Criterion cond = Restrictions.like("role", "%P.L.");
            ct.add(cond);
            // execute logic
            List ll = ct.list();
            return ll;
        } // method
    }); // inner class
    Iterator it = l.iterator();
    return it;
}
```

NOTE:- In the above code ht.execute() method is called having anonymous inner class object that implements HibernateCallback (I)

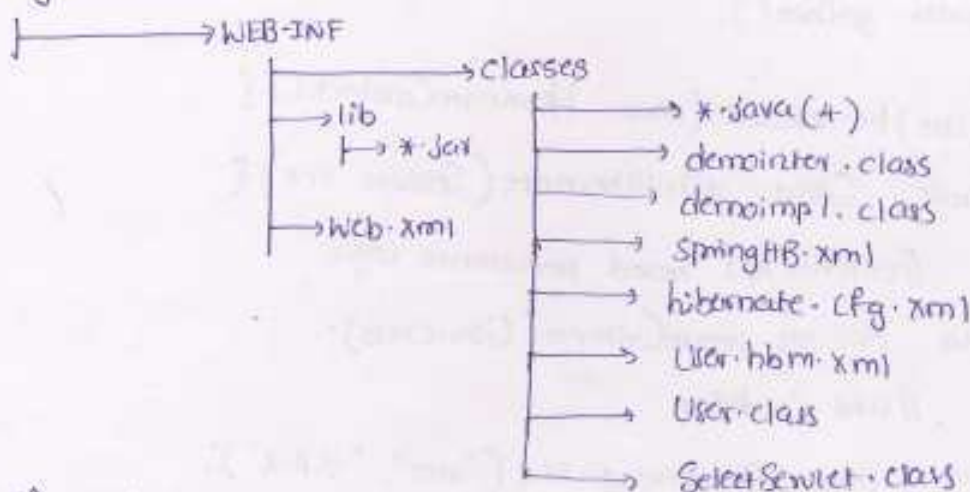
To execute this code place the following jar files in the classpath.

- { Spring.jar
- { hibernate3.jar
- { Servlet-api.jar

→ In Spring with Hibernate applications we can take Hibernate Configuration file and Spring Configuration file separately. then Hibernate Configuration file must be linked with Spring cnfg file by using configure Location property of LocalSessionFactoryBean. In this situation there is no need of configuring bean that gives data source object in Spring Configuration file.

Revised Spring with Hibernate App of previous class

SpringHBAApp



hibernate.cfg.xml :-

Same old hibernate configuration file.

SpringHB.xml

<!DOCTYPE ----- >

<beans>

<bean id="mySessionFactory" class="org.springframework.orm.hibernate3.LocalSessionFactoryBean">

<property name="configLocation" value="classpath:hibernate.cfg.xml"/>

</bean>

== // Same old HibernateTemplate, Demoimpl classes Configuration
== refer previous app.
==

</beans>

(*) For example application on annotations based spring, hibernate integration (spring 3, hibernate 4).
refer the Supplementary Handout given on 23/03/2013

Module - IV

Spring JEE Module

- 1) gives ApplicationContext Container
 - 2) provides abstraction Layer on java mail api for mailing operations.
 - 3) provides abstraction Layer on jms for messaging operations.
 - 4) provides abstraction Layer on rmi, webservice to develop distributed applications.
 - 5) gives direct distributed technology called HttpInvoker.
 - 6) Supports Timer based scheduling operations.
 - 7) provides abstraction Layer on Jndi to interact with Registry softwares.
- and etc....

→ Making our application executing on certain Date and time either only for one time or for multiple times is nothing but enabling scheduling on the applications. While dealing with Time critical jobs this scheduling is Very important.

Examples:-

Sending reminders on the 1st of every month.

cleaning Temporary files every day at 12'o'clock.

We can use TimerTask, Timer Class of JDK environment for these operations.

→ The Java class that extends from TimerTask class defines the job on which we are planning to enable service.

— The Scheduled method of Timer class can enable scheduling on the job

for JDK level Timer Service application refer Application (8) of the page no's (67) and (68)

lines:

1475-1482 The logic on which we are looking to enable Timer Service

1492-1493 Initial delaying milliseconds

3000 Specifies initial delay.

3000 specifies the time gap b/w two successive executions.

1494-1496 execute the job on specified Date and time

Advantages of Spring ^{scheduling} ~~Abstract~~ Layer on Jdk level scheduling and gives the following benefits.

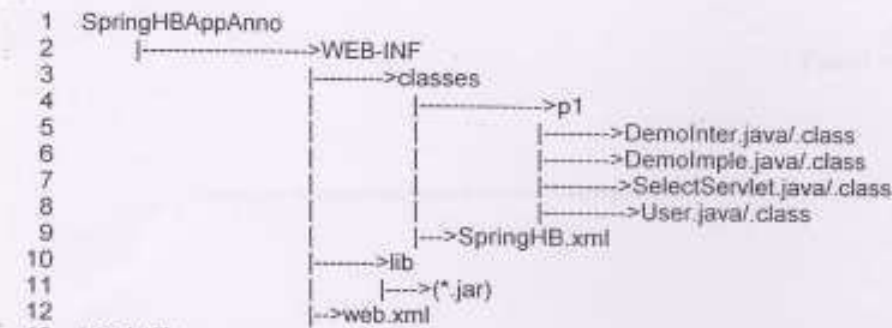
- ① allows to perform scheduling on multiple jobs.
- ② allows to specify scheduling related parameters declaratively using xml files.
- ③ allows to perform scheduling on the user defined methods of user defined classes. that means classes need not to be extended from TimerTask class

Three important beans of Spring Based Timer operations.

- ① org.springframework.scheduling.timer.TimerFactoryBean
(Starts ~~Factory~~ Timer service When container is started)
- ② org.springframework.scheduling.timer.SchedulingTimerTask
→ represents the job on which we want to enable Timer service
→ This job must be there in a java class that extends from TimerTask class.
- ③ org.springframework.scheduling.timer.MethodInvokingTimerTaskFactoryBean
→ represents the job on which we want to enable Timer service
→ This job can be there in a user-defined method of user-defined java class.

For Spring based scheduling example applications refer application 19, 20 of the

Page no's 68 to 70



jars in lib

15	antlr-2.7.7.jar	15-23	collected from hibernate-4-x
16	commons-logging.jar	24-25	collected from jackson-der.com
17	dom4j-1.6.1.jar	27-28	collected from spring 3.1
18	hibernate-commons-annotations-4.0.1.Final.jar		
19	hibernate-core-4.1.8.Final.jar		
20	hibernate-ehcache-4.1.8.Final.jar		
21	hibernate-jpa-2.0-api-1.0.1.Final.jar		
22	javassist-3.15.0-GA.jar		
23	javax.persistence.jar		
24	jboss-logging-3.1.0.GA.jar		
25	jboss-transaction-api_1.1_spec-1.0.0.Final.jar		
26	ojdbc14.jar		
27	org.springframework.asm-3.1.1.RELEASE.jar		
28	org.springframework.aspects-3.1.1.RELEASE.jar		
29	org.springframework.beans-3.1.1.RELEASE.jar		
30	org.springframework.context.support-3.1.1.RELEASE.jar		
31	org.springframework.context-3.1.1.RELEASE.jar		
32	org.springframework.core-3.1.1.RELEASE.jar		
33	org.springframework.expression-3.1.1.RELEASE.jar		
34	org.springframework.jdbc-3.1.1.RELEASE.jar		
35	org.springframework.orm-3.1.1.RELEASE.jar		
36	org.springframework.transaction-3.1.1.RELEASE.jar		

Jar Files in CLASSPATH

javax.persistence.jar
 org.springframework.context.support-3.1.1.RELEASE.jar
 org.springframework.context-3.1.1.RELEASE.jar
 org.springframework.beans-3.1.1.RELEASE.jar
 org.springframework.core-3.1.1.RELEASE.jar
 hibernate-core-4.1.8.Final.jar

```

39  DemoInter.java
40  -----
41  package p1;
42  import java.util.*;
43  public interface DemoInter
44  {
45      public Iterator getData()throws Exception;
46  }
47  =====
48  DemoImpl.java
49  -----
50  package p1;
51  import org.hibernate.*;
52  import java.util.*;
53  import org.springframework.stereotype.*;
54  import org.springframework.beans.factory.annotation.Autowired;
55
56  @Component("d1")
57  public class DemoImpl implements DemoInter
58  {
59      @Autowired
60      private SessionFactory sesfact;
61
62      public void setSesfact(SessionFactory f1)
63      {
64          sesfact=f1;
65      }
66
67      public Iterator getData()throws Exception
68      {

```



```

69         Session ses=sesfact.openSession();
70         Query query=ses.createQuery("from User");
71         Iterator i1=query.iterate();
72         return i1;
73     }
74 }
=====
75
76 User.java
77 -----
78 package p1;
79 import javax.persistence.*;
80
81 @Entity
82 @Table (name= "users")
83 public class User
84 {
85     @Id
86     @GeneratedValue
87     @Column(name= "userid")
88     private int uid;
89
90     @Column(name= "uname")
91     private String uname;
92
93     @Column (name= "role")
94     private String role;
95
96     public void setUid(int n)
97     {
98         uid=n;
99     }
100
101     public int getUid(){ return uid; }
102     public void setUname(String s){ uname=s; }
103     public String getUname(){ return uname; }
104     public void setRole(String r){ role=r; }
105     public String getRole(){ return role; }
106 }
=====
107
108 DBTable
109 -----
110 create table users
111 ( userid number(5) primary key,
112   uname varchar2(20),
113   role varchar2(20)
114 );
=====
115
116 SpringHB.xml
117 -----
118 <beans xmlns="http://www.springframework.org/schema/beans"
119       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
120       xmlns:context="http://www.springframework.org/schema/context"
121       xsi:schemaLocation="http://www.springframework.org/schema/beans
122       http://www.springframework.org/schema/beans/spring-beans-2.5.xsd
123       http://www.springframework.org/schema/context
124       http://www.springframework.org/schema/context/spring-context-2.5.xsd">
125
126 <context:component-scan base-package="p1" />
127
128 <bean id="myds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
129   <property name="driverClassName"><value>oracle.jdbc.driver.OracleDriver</value> </property>
130   <property name="url"> <value>jdbc:oracle:thin:@localhost:1521:orcl</value> </property>
131   <property name="username"> <value>scott</value> </property>
132   <property name="password"> <value>tiger</value>
133   </property>
134 </bean>
135
136 <bean id="sesfact" class="org.springframework.orm.hibernate4.LocalSessionFactoryBean">

```

Recommended to
Configure either
Native DBCP (or)
C3P0 (or) Spring
managed ConnectionPool

```

137 <property name="dataSource" ref="mysds"/>
138 <!-- use this to avoid configuring mapping file-->
139 <property name="packagesToScan" value="p1"></property>
140 </bean>
141
142 </beans>
143 =====
144 SelectServlet.java
145 -----
146 package p1;
147
148 import javax.servlet.*;
149 import javax.servlet.http.*;
150 import java.io.*;
151 import org.hibernate.*;
152 import java.util.*;
153 import org.springframework.beans.factory.*;
154 import org.springframework.context.*;
155 import org.springframework.context.support.*;
156 public class SelectServlet extends HttpServlet
157 {
158     public void service(HttpServletRequest request, HttpServletResponse response)
159         throws ServletException, IOException
160     {
161
162         PrintWriter out=response.getWriter();
163         System.out.println("In service method of Servlet");
164         try{
165             ApplicationContext ctx=new ClassPathXmlApplicationContext("SpringHB.xml");
166             demointer d1=(demointer)ctx.getBean("d1");
167             Iterator i1=d1.getData();
168
169             out.println("<body bgcolor=#ffffcc text=red>");
170             out.println("<h1><center>all users</h1><hr><br><h3>");
171             out.println("<table width=80% border=2>");
172             while(i1.hasNext())
173             {
174                 User u1=(User)i1.next();
175                 out.println("<tr><td>"+u1.getId()+" <td>"+u1.getUsername()+" <td>"+u1.getRole()+"</tr>");
176             }
177             out.println("</table>");
178         }
179         catch(Exception e)
180         {
181             out.println(e);
182             e.printStackTrace();
183         }
184     }
185 }
186 =====
187 web.xml
188 -----
189 <web-app>
190 <servlet>
191     <servlet-name>select</servlet-name>
192     <servlet-class>p1.SelectServlet</servlet-class>
193 </servlet>
194
195 <servlet-mapping>
196     <servlet-name>select</servlet-name>
197     <url-pattern>/selectaction</url-pattern>
198 </servlet-mapping>
199
200 </web-app>
201 =====
202

```

Handwritten note: makes the Spring container recognizing all the ^{classes} ~~packages~~ that are there in 'P1' Pac

In spring 3.1 HibernateTemplate class has been deprecated and removed. So, inject
Hibernate SessionFactory object to SpringBean while working with Spring 3 {hibernate 4
Combination.

Java Mail

→ Database s/w maintains records

registry s/w maintains Objects

mail Server maintains email accounts and messages.

→ Java application uses uses jdbc api → DB s/w (Oracle, Sybase and etc...)

→ Java appn Jndi api → Registry s/w (~~Webapp~~^{logic} jboss registry and etc...)

→ Java appn Java mail api → Mail server s/w (James Mailserver, Exchange server, Lotus ^{notes} and etc...)

java mail api means javax.mail, javax.mail.internet pkg (part of JEE module)

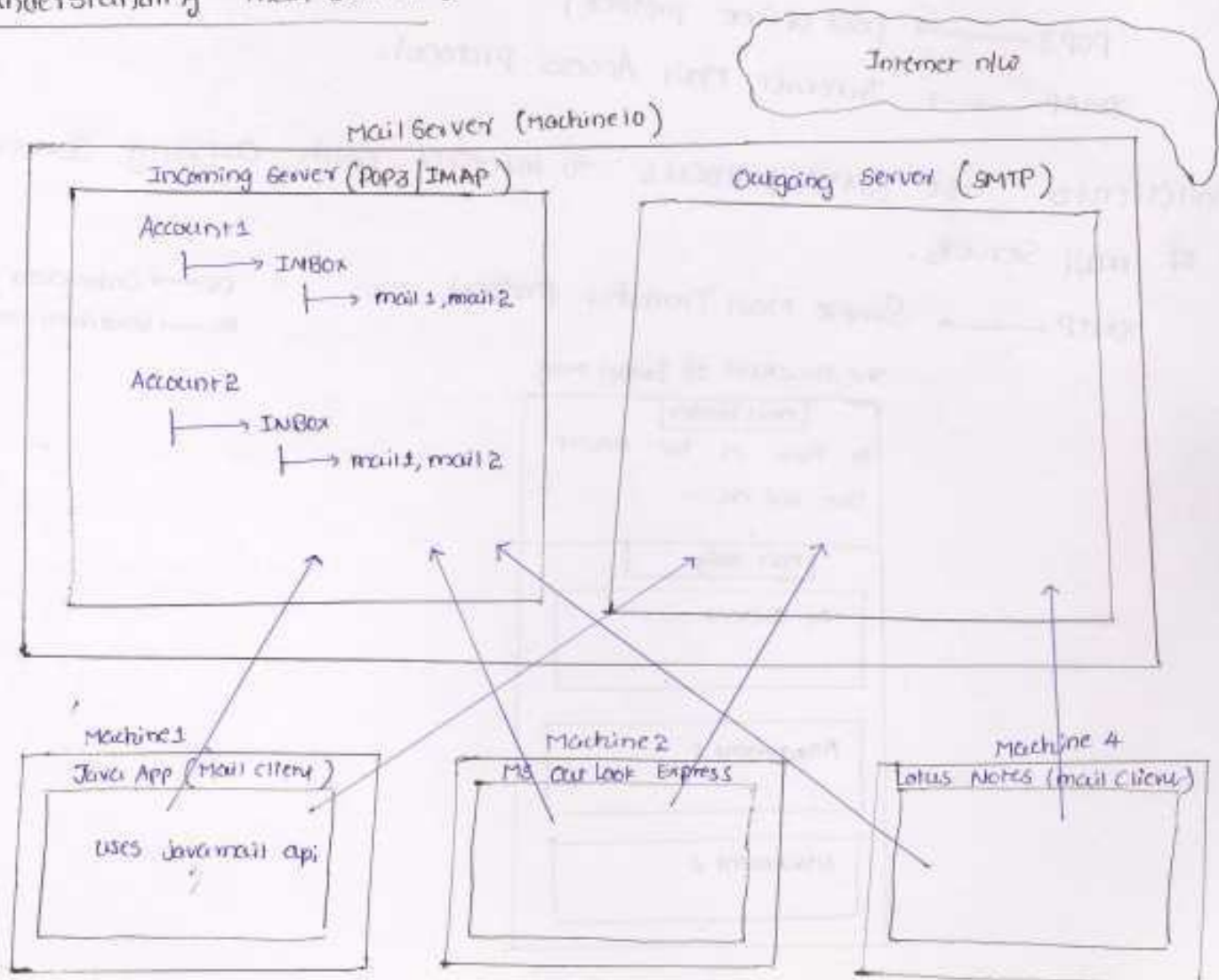
DB s/w, Web Server s/ws are not responsible to email accounts and email

→ they are two types mail clients

1) Ready Made mail clients (MS outlook, MS outlook express, Lotus notes)

2) Programmable Mail clients (App with Java mail api utilization)

Understanding mail Server :-



- Incoming Server is responsible to receive and manage email messages in the inboxes of email accounts.
- Outgoing Server is responsible to send email messages from one email account to another email account.
- POP3 incoming Server deletes the email messages from inbox once they are read by mail clients. So, it is useful for intranet mailing system.
- The IMAP incoming Server maintains the email messages in the inbox given after ^{they are} read from inbox. So, it is suitable for Internet-based mailing operations.

Eg:- like WWW.GMail.com

- Mail clients use POP3/IMAP protocols to interact with incoming Servers of mail Servers.

POP3 → post office protocol

IMAP → Internet Mail Access protocol.

Mailclients use SMTP protocols to interact with Outgoing Servers of mail Servers.

SMTP → Simple Mail Transfer protocol.

cc → Carbon client

bcc → blind Carbon client.

The structure of Email msg



James

→ Type : MailServer

Version: 2.1

Vendor: Apache

default port no's

For admin console : 4555

For POP3 incoming server : 110

For SMTP outgoing server : 25

→ Installation : Extract the zip for installation of slw.

To start the mail server : <James_home>\bin\run.bat file.

Procedure to create email accounts manually in James mail server :-

Step 1 :- Start the James server

Step 2 :- use telnet^{tool} to connect to^{the} admin console of James server.

Start → run → telnet → open localhost 4555

login id:

root

password:

root

Welcome root.

help

	username	password
adduser	Srikanth	Srikanth
adduser	mani	mani
listusers		

countusers

deluser ravi
User ravi deleted

→ Session object of Java mail API represents the connectivity between Java app and mail server. To create this session obj we need mail properties and these property names are fixed but values will vary based on the mail server we use.

(*) For example application on Java mail API 60 & 61 Page no's appn no:

→ In real weblevel mail applications the Javamail API code will be written in webresource programs like Servlet, JSP. to perform mail operations on email accounts.

Details of Gmail mail Server :-

Outgoing Server name : SMTP.Gmail.Com

~~Outgoing~~ port number of outgoing server : 465

Incoming Server name : POP.Gmail.Com

Incoming Server port number : 995

for plain Java mail API based apps to interact with mail server of Gmail refer application ② to appn ④ of the Supplementary Handout given on 26/03/2013

```

1 Java mail Apps
2 -----App1-----SendAttachment.java (using plain java mail API)
3 /*
4  Java Application to send an E-mail with attachment
5  */
6 import javax.mail.*;
7 import javax.mail.internet.*;
8 import javax.activation.*;
9 import java.util.*;
10 public class SendAttachment
11 {
12     public static void main(String []args) throws Exception
13     {
14         //Creating Properties Object
15         Properties properties=new Properties();
16
17         //adding protocol,mailserver address & the port number of the mailserver
18         properties.put("mail.transport.protocol","smtp");
19         properties.put("mail.smtp.host","localhost");
20         properties.put("mail.smtp.port","25");
21
22         //Creating the Session Object
23         Session session=Session.getInstance(properties);
24
25         //Creating and configuring the Message object
26         Message message=new MimeMessage(session);
27
28         message.setFrom(new InternetAddress("sachin1"));
29         InternetAddress address[]={new InternetAddress("dhoni1")};
30         message.setRecipients(Message.RecipientType.TO,address);
31         message.setSentDate(new Date());
32         message.setSubject("MAIL WITH ATTACHMENT");
33
34         // mail body obj
35         Multipart mailbody=new MimeMultipart(); → represents the empty body of the Email message
36
37         // mail body part1 obj (text content)
38         MimeBodyPart part1=new MimeBodyPart();
39         part1.setText("hello see my resume");
40         mailbody.addBodyPart(part1);
41
42         //mail body part2(attachment)
43         FileDataSource fds=new FileDataSource(args[0]); //represents attachment file
44         MimeBodyPart part2=new MimeBodyPart();
45         part2.setDataHandler(new DataHandler(fds));
46         part2.setFileName(fds.getName());
47         mailbody.addBodyPart(part2);
48
49         message.setContent(mailbody); → Setting body to message content
50
51         //Sending the mail
52         Transport.send(message);
53
54     } //main
55 } //class
56 //> javac SendAttachment.java
57 //> java SendAttachment one.txt
58
59 // Prgs on plain java mail api (with Gmail Env....)
60 -----App2-----SendMail.java-----
61 //Mail.java
62 import java.util.*;
63 import javax.mail.*;
64 import javax.mail.internet.*;
65
66 public class SendMail
67 {
68     public SendMail() {

```

add mail.jar file to the CLASSPATH.


```

69 // mail properties outgoing server (gmail.com)
70 Properties props = new Properties();
71 props.put("mail.smtp.host", "smtp.gmail.com"); //hostname
72 props.put("mail.smtp.port", "465"); //port number
73 props.put("mail.smtp.auth", "true");
74 props.put("mail.smtp.socketFactory.class", "javax.net.ssl.SSLSocketFactory");
75
76 try {
77 // create Session obj
78 Authenticator auth = new SMTPAuthenticator(); // Object creation for inner class
79
80 Session session = Session.getInstance(props, auth);
81
82 //prepare mail msg
83 MimeMessage msg = new MimeMessage(session);
84 //set header values
85 msg.setSubject("open to it know it"); testjava @ gmail . com
86 msg.setFrom(new InternetAddress("satyaemail123@gmail.com"));
87 msg.addRecipient(Message.RecipientType.TO, new InternetAddress("nataraz@gmail.com"));
88 //msg text
89 msg.setText("mail from satya");
90
91 Transport.send(msg);
92 } //try
93 catch (Exception ex) {
94 ex.printStackTrace();
95 } //catch
96 } //constructor
97
98 private class SMTPAuthenticator extends javax.mail.Authenticator {
99     public PasswordAuthentication getPasswordAuthentication() {
100         return new PasswordAuthentication("satyaemail123@gmail.com", "javaj2ee");
101     } //method
102 } //inner class
103 public static void main(String[] args) {
104     SendMail mail = new SendMail();
105     System.out.println("mail has been delivered");
106 } //main
107 } // Mail class
108 -----App3-----GetMail.java----->java SendMail.java
109 import java.util.*;
110 import javax.mail.*;
111
112 public class GetMail {
113
114     String host="pop.gmail.com"; testjava @ gmail . com
115     String emailfrom="satyaemail123@gmail.com";
116     String password="javaj2ee";
117     String port="995"; testjava
118
119     GetMail()
120     {
121         Properties props = new Properties();
122
123         props.put("mail.pop3.host", host);
124         props.put("mail.pop3.port", port);
125         props.put("mail.pop3.auth", "true");
126         props.put("mail.pop3.socketFactory.class", "javax.net.ssl.SSLSocketFactory");
127
128         try {
129             Authenticator auth = new SMTPAuthenticator();
130             Session session = Session.getInstance(props, auth);
131
132             Store store = session.getStore("pop3s"); // Connecting to pop3 incoming server
133
134             store.connect(host, emailfrom, password); // Connects to Email I d
135
136             Folder folder = store.getFolder("inbox");

```

```

137
138 folder.open(Folder.READ_ONLY); → Inbox open in READ-ONLY mode
139 Message[] msg = folder.getMessages(); → reading all the email messages from the Inbox.
140
141 //try
142 {
143     System.out.println((msg.length)+" message found");
144     for (int i = 1; i <= msg.length; i++) {
145         Message mess=folder.getMessage(i);
146         mess.writeTo(System.out);
147     } //for
148 } //try
149 catch(Exception e)
150 {e.printStackTrace();}
151 //constructor
152
153 private class SMTPAuthenticator extends javax.mail.Authenticator {
154     public PasswordAuthentication getPasswordAuthentication() {
155         return new PasswordAuthentication(emailfrom, password);
156     }
157 } //private class
158
159 public static void main(String args[])
160 {
161     new GetMail();
162 } //main
163 } //class
164 -----App4-----DeleteMail.java-----
165 import java.util.*;
166 import javax.mail.*;
167
168 public class GetMail {
169
170     String host="pop.gmail.com"; //satya@gmail.com
171     String emailfrom="satyaemail123@gmail.com";
172     String password="java2ee";
173     String port="995"; //terjava
174
175     Delete GetMail()
176     {
177         Properties props = new Properties();
178
179         props.put("mail.pop3.host", host);
180         props.put("mail.pop3.port", port);
181         props.put("mail.pop3.auth", "true");
182         props.put("mail.pop3.socketFactory.class", "javax.net.ssl.SSLSocketFactory");
183
184         try {
185             Authenticator auth = new SMTPAuthenticator();
186             Session session = Session.getInstance(props, auth);
187
188             Store store = session.getStore("pop3s");
189
190             store.connect(host,emailfrom,password);
191
192             Folder folder = store.getFolder("inbox");
193
194             folder.open(Folder.READ_WRITE);
195             Message[] msg = folder.getMessages();
196
197             System.out.println((msg.length+1)+" message found");
198             for (int i = 1; i <= msg.length; i++) {
199                 Message mess=folder.getMessage(i);
200                 mess.writeTo(System.out);
201             } //for
202         } //try
203         catch(Exception e)
204         {e.printStackTrace();}
205         //constructor
206

```

Handwritten notes:
 - "msg[0].setFlag(Flags.Flag.DELETED, true); //mark them for deletion"
 - "folder.close(true); //close the inbox with flag true"
 - "store.close();"

Handwritten annotations:
 - "L3 standard output stream (write msgs to monitor)"
 - "L3 standard output stream (write msgs to monitor)"
 - "marks the email messages for deletion"


```

205 private class SMTPAuthenticator extends javax.mail.Authenticator {
206     public PasswordAuthentication getPasswordAuthentication() {
207         return new PasswordAuthentication(emailfrom, password);
208     }
209 } //private class
210
211 public static void main(String args[])
212 {
213     new GetMail();
214 } //main
215 } //class
216
217 Application sending sms.....
218 -----App5-----SendSMS.java-----
219 import java.io.*;
220 import java.net.InetAddress;
221 import java.util.Properties;
222 import java.util.Date;
223 import javax.mail.*;
224 import javax.mail.internet.*;
225 import javax.activation.*;
226
227 public class SendSMS {
228
229     public SendSMS() {
230     }
231
232     //create an account on ipipi.com with the given username and password
233     public void msgsend() {
234         String username = "sathyaSMS"; //Your Credentials
235         String password = "sathya123";
236         String smtpHost = "ipipi.com"; //Name of Server
237         String compression = "None"; //I dont want any compression
238         String from = "sathyaSMS@ipipi.com"; //u r userid@ipipi.com
239         //This mobile number need not be registered with ipipi.com
240         String to = "919861098610@sms.ipipi.com"; //mobile number where u want to send sms
241         String body = "Hi This Msg is sent through Java Code";
242         Transport tr = null;
243
244         try {
245             Properties props = System.getProperties();
246             props.put("mail.smtp.auth", "true");
247
248             // Get a Session object
249             Session mailSession = Session.getDefaultInstance(props, null);
250
251             // construct the message
252             Message msg = new MimeMessage(mailSession);
253
254             //Set message attributes
255             msg.setFrom(new InternetAddress(from));
256             InternetAddress[] address = (new InternetAddress(to));
257             msg.setRecipients(Message.RecipientType.TO, address);
258             msg.setSubject(compression);
259             msg.setText(body);
260             msg.setSentDate(new Date());
261
262             tr = mailSession.getTransport("smtp");
263             tr.connect(smtpHost, username, password);
264             msg.saveChanges();
265             //send msg to all recipients
266             tr.sendMessage(msg, msg.getAllRecipients());
267             tr.close();
268         } catch (Exception e) {
269             e.printStackTrace();
270         }
271     }
272
273     public static void main(String[] argv) {

```

Handwritten notes:

- Line 224: *javaMail API*
- Line 245: *adding new mail properties to the existing mail properties*
- Line 252: *empty email msg*
- Line 255: *ipipi.com* (pointing to from)
- Line 256: *mobile number* (pointing to to)
- Line 257: *preparing email msg*
- Line 262: *this transport object points to the outgoing server of ipipi*
- Line 263: *Connects to outgoing server of ipipi*
- Line 266: *sends the above created msg to the specified recipients*

[illegible]


```
341 {  
342     public static void main(String args[])  
343     {  
344         ApplicationContext ctx=new FileSystemXmlApplicationContext("mailcfg.xml");  
345         Student beanref=(Student)ctx.getBean("st");  
346         beanref.sendEmailMsg();  
347         System.out.println("mail has been delivered");  
348     }  
349 }  
350  
351
```

Spring Mail :-

telnet services in windows 7

- Provides abstraction layer on plain javaMail API.
- As of now SpringMail is supporting only Send mail operations.
- Allows cfg mail msg properties and mail properties in Spring cfg file Declaratively.
- gives callback interfaces to send email msgs with attachments.

→ org.springframework.mail package, } represents Springmail API.
org.springframework.mail.javamail packages }

→ org.springframework.mail represents Spring^{supplied} direct ~~info~~ facilities to perform mail operations (useful to send email messages)

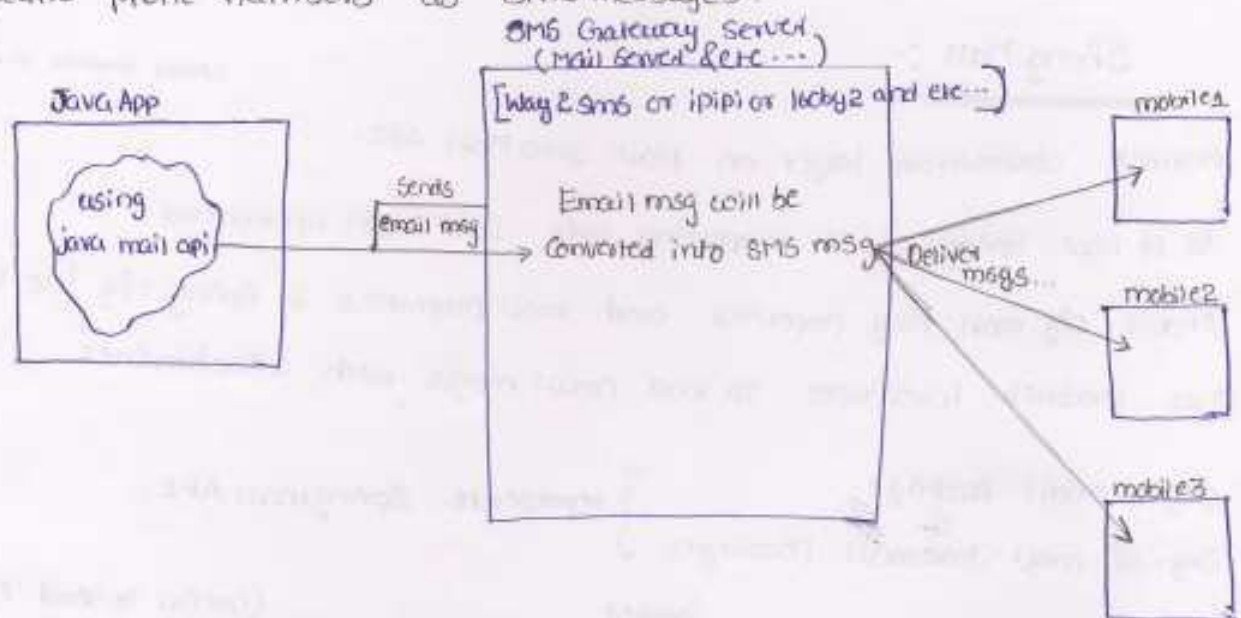
→ org.springframework.mail.javamail represents Spring's abstraction Layer on plain java mail api. (useful to send email messages with attachments)

(*) For example application on Sending email message by using Springmail refer appn no: (14) of the page no's 61 & 62

→ To send ~~email~~ email msgs with attachment by using Springmail API we need to work with the callback interface MimeMessagePreparator() & one concrete class called MimeMessageHelper

(*) For example application on Springmail based send mail operation with attachment refer appn (16) of the given ^{given on} handout 26/03/2013

Way2Sms, IPIPI, 160by2 and etc... are the Sms Gateways. These Gateways can receive messages as email messages and delivers these messages to specific phone numbers as Sms messages.



③ procedure to Send Sms msgs to mobile numbers by using Javamail Api app

Step 1:- Create one Account in IPIPI.com.

Step 2:- Username : zavaBoss
password : zavaBoss1

Step 3:- Run the Send Sms application (refer : supplementary handout given on 26/03/13)

Procedure to access the webapplication of TomcatServer with domain name.

Step 1:- specify domain name in built-in ~~reg~~ DNS registry of windows Xp/7

Go to C:\Windows\system32\drivers\etc\hosts file and replace 127.0.0.1 localhost to 127.0.0.1 www.nataraz.in

Step 2:- change ^{the} Tomcat Server port number to "80"

If port number is not Specified in Http request url then the protocol http tries to send the request to a Service whose port number is "80".

add following code in server.xml file under <Engine> tag specifying the domain name and webapplication name

```

<Host name="www.nataraz.in" appBase="webapps"
    unpackWARs="true" autoDeploy="true" >
    <Context path="" docBase="VoteApp" />
    <!-- webapplication -->
</Host>

```

Step 3:- Deploy the above specify VoteApp webapps in tomcat server.
make sure that WelcomeFile is configured in that webapps.

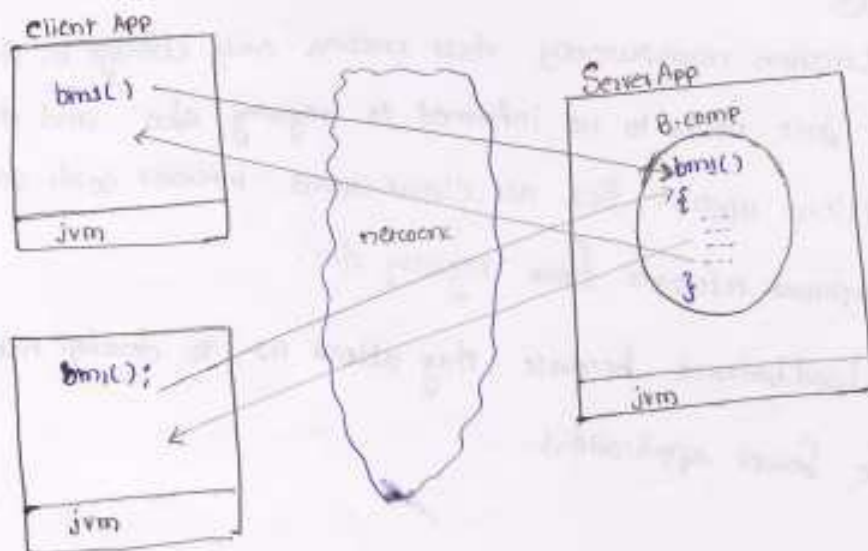
Step 4:- Restart the Server once.

Step 5:- Test the application

Open browser window and type this url <http://www.nataraz.in/>

Understanding the need of Distributed application Development :-

Traditional client-server appn

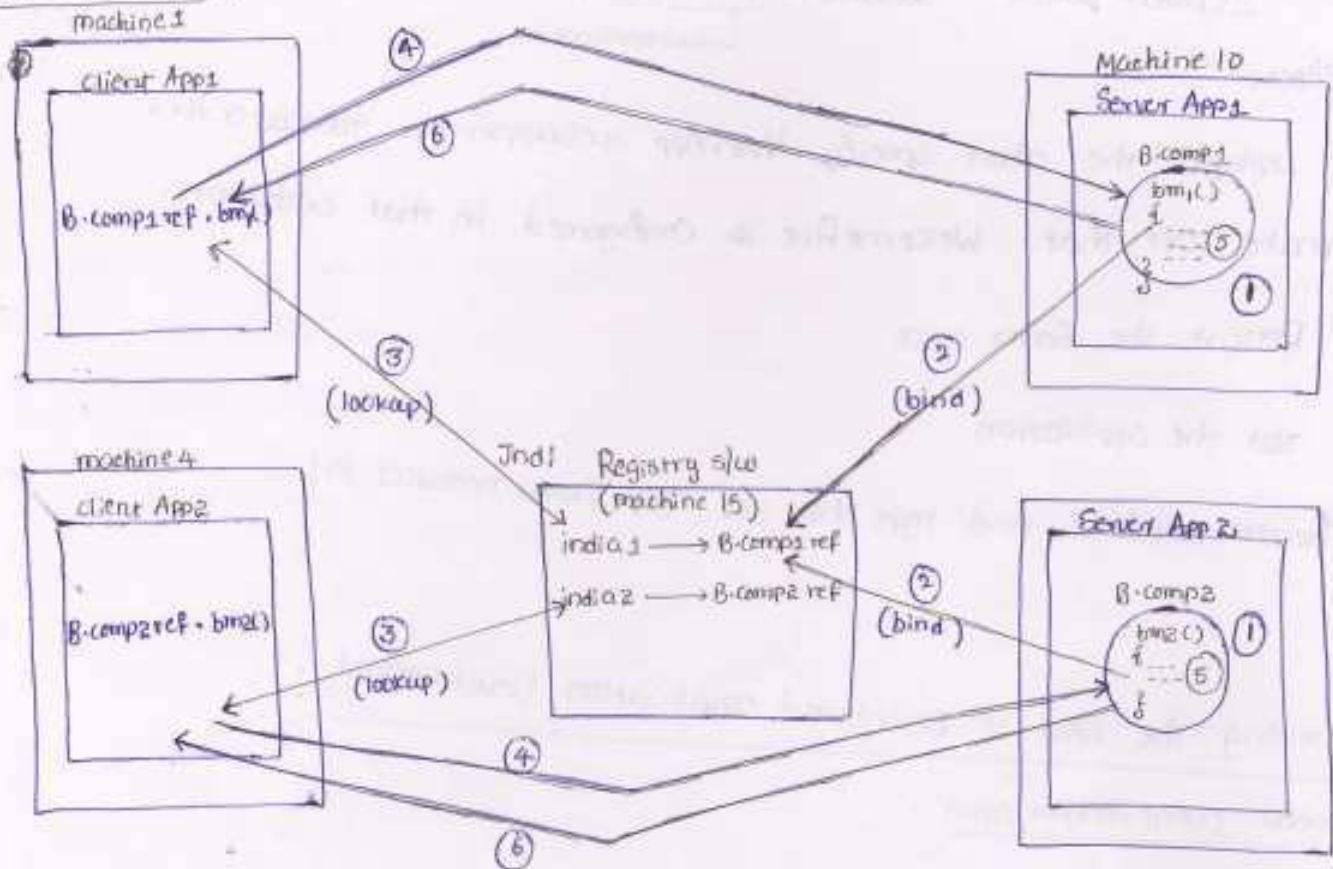


Traditional client-server applications (Socket programming Based) are location dependent that means any change in the location of Server application we must modify code in the client apps. This indicates client and Server apps are tightly-coupled apps (hardly-coupled) here.

If degree of dependency is more b/w two apps then they are called tightly coupled apps. Other wise they are called loosely-coupled apps.

To develop the ~~apps~~ ^{client-server} Apps as Location transparent Apps then develop them as distributed applications.

Distributed App:-



Distributed apps support Location transparency that means any change in the location of Server application just needs to be informed to registry s/w. and there is no need of change in coding client apps. Bcz all client apps interact with server apps by getting Business Component reference from registry s/w.

Distributed apps support Load balance because they allow us to develop multiple Business Components in multiple Server applications.

W.r to the diagram:-

- multiple Server applications will be developed having multiple B-components.
- These multiple B-components references will be registered with registry s/w for global visibility.
- Client apps gather B-component references from registry s/w
- (4), (5), (6) client appn calls B-methods of B-component and these B-methods of Server appn will executed & final result goes back to client appn.

To develop Distributed apps we need the Support of distributed technologies.

Eg:- RMI, EJB, web services, HttpInvoker, CORBA and etc.....

→ Spring JEE provides abstraction layer on existing distributed technologies like RMI, EJB, webservices and also provides its own distributed technology called HttpInvoker.

→ In Distributed App Development 4 parties are involved.

① Service Interface / Business Interface → (as a common understand document b/w Service provider Apps and Server client apps)
→ Declaration of b-methods

② Server Provider App

(The Server App that contains b-methods in B-component)

③ Registry slw

(makes b-comp references as globally visible obj's and also makes Distributed Apps as location transparent apps)

④ Service Client App

(Calls the b-methods of B-components.)

Plain RMI :-

→ It is built-in Distributed Technology of JDK

→ uses Rmi registry as Registry slw

→ Core pkgs of rmi api are: java.rmi, java.rmi.server pkgs

Q:- What is the Difference b/w Local client and Remote client?

Ans:- If the appn and its client resides on same JVM then it is called Local client to appn.

If appn and its client resides in two different JVMs of same or different Computers then it is called Remote client to appn.

Q:- What is the Diff b/w Local object and Remote Object?

Ans:- If ^{the} methods of the object can be called only from local clients then it is called Local object.

If the methods of the object can be called from both Local, Remote clients then it is called Remote Object. Object becomes Remote Object only when its

class implements java.rmi.Remote (I).

→ while developing distributed apps all the business objects must be developed as Remote object.

(*) Example application on plain rmi

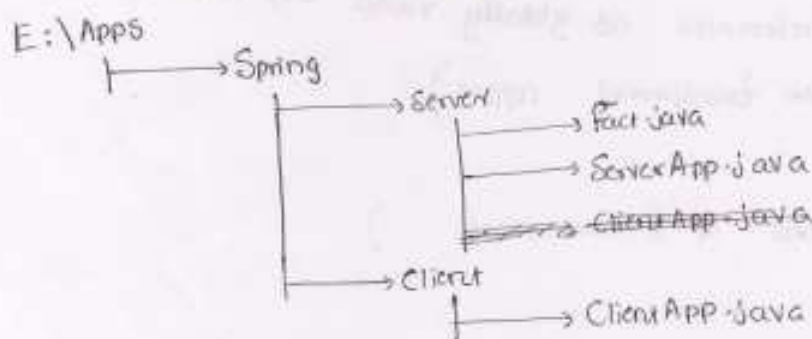
Fact.java (Service Interface)

ServerApp.java (Service Provider App)

ClientApp.java (Service Client App)

Rmi registry (Registry slo).

In one computer multiple JVMs can be there to run multiple java apps Simultaneously (or) Concurrently



// Fact.java (Service Interface)

```
import java.rmi.*;

public interface Fact extends Remote
{
    // declare B-methods
    public long findFactorial(int val) throws RemoteException;
}
```

// ServerApp.java

```
import java.rmi.*;
import java.rmi.server.*;
//impl class that gives B-comp obj
class ServerApp FactImpl extends UnicastRemoteObject implements Fact
{
    // to make this Business Object registrable in Registry slo
    public FactImpl() throws RemoteException
    {
        S.O.P("FactImpl : 0-param Constructor");
    }
    // implement b-method
    public long findFactorial(int val)
    {

```

```

        long res = 1;
        for (int i = 1; i <= val; ++i)
        {
            res = res * i;
        }
        return res;
    } // find Factorial
} // class

```

```

public class ServerApp
{
    public static void main (String args[]) throws Exception
    {
        FactImpl obj = new FactImpl(); // Business Obj
        // bind B-obj with registry s/w
        Naming.bind ("india", obj); // impossible also Naming.bind ("rmi://localhost:1099/india", obj)
        S.o.p ("Server App is ready....");
    }
}

```

// ClientApp.java

```

import java.rmi.*;

public class ClientApp
{
    public static void main (String[] args) throws Exception
    {
        // get B-obj ref from registry through lookup operation
        Fact bobj = (Fact) Naming.lookup ("india"); // Fact bobj = (Fact) Naming.lookup ("rmi://localhost:1099/india")
        // call B-methods
        S.o.p ("Client App: " + bobj.findFactorial (5));
    } // main
} // class

```

Compile .java files.

E:\Apps\spring\server> javac *.java

Copy Fact.class file to Client folder.

E:\Apps\spring\client> javac *.java

Generate Stub file

E:\Apps\spring\server> rmic FactImpl

gives FaceImpl - Stub - class

Copy Stub file to client folder

Start rmi registry from Server folder

E:\Apps\Spring\Server > rmiregistry

Run the Server application

E:\Apps\Spring\Server > java ServerApp

Run the Client application

E:\Apps\Spring\Client > java ClientApp

The default port number of
rmi registry is 1099

To change Rmi registry port number :-

Syntax :-

> rmiregistry <newport no>

Eg:- > rmiregistry 2000

- The process of converting Java notation Data to Network notation data is called marshalling. and reverse is called Unmarshalling.
- The Stub at client side & at Serverside performs these marshalling & unmarshalling operations.
- Stub acts as client side proxy for client appn & Serverside proxy for Server appn.
- Bcz, these two proxies will communicate with each other on behalf of on rmi client & Server side appns.

∴ Spring rmi :-

- provides abstraction layer on plain rmi
- allows to develop Server & client appns as Spring appns.
- There is no need of working with plain rmi api.
- org.springframework.remoting.rmi pkg represents Spring rmi api.
- There is no need of performing exception handling in Spring rmi appns.
- There is no need of starting rmi registry explicitly
- can work with Spring Container started rmi registry or can work with explicitly started rmi registry.

→ There is no need of
generating stub files

Explicitly

→ in Spring rmi appn the B.obj
that is registered with rmi registry is
identified through two items
(1) rmiName / alias name (2) ServiceInterface name

→ allocates to keep business object reference in rmi registry through dependency injection process and allocates to get B.obj ref from rmi registry through dependency injection process.

→ allocates to develop the references as poji's & POJO's

→ allocates to specify rmi registry related cfgs in xmi file declaratively.

org.springframework.rmi.RmiServiceExporter can expose given Business object to Rmi having specified name

org.springframework.rmi.RmiProxyFactoryBean → gives Business obj ref gathered from rmi registry through lookup operation

(*) For example application on SpringRmi based Distributed apps refer page no: 62-66 app (15)

→ While developing plain RMI client app we need to gather following details from Server application.

a) nickname given to rmi B-object.

b) a copy of service interface, stub files

c) Uri to perform lookup operation on rmi registry like (rmi registry host name, port number)

d) Documentation about Business methods

→ While developing SpringRMI client app we need following details from Server app.

Same as above details but Stub file is not required.

Plain rmi Server App ← (yes) → plain rmi client App

Spring rmi Server App ← (yes) → Spring rmi client app

Plain rmi Server App ← (yes) → Spring rmi client app

[Stub file should be copied to Client app side]

Spring rmi Server App ← (no) → plain rmi client app

[plain rmi client App need stub file but Spring rmi Server app does not generate this stub file explicitly]

→ In real world all Credit card and Debit card component processing will be developed as Distributed Components / Applications.

for example application on SpringRmi using Spring ~~CRM~~^{DAO} module refer appn (6) of the page no's 64 & 65

While developing logics in SpringRMI Server application we can take the support of Spring CRM, DAO modules while developing BusinessLogics.

Drawbacks of RMI :-

- ① RMI is language dependent (That means both client and Server appn must be developed in Java Language.).
 - ② RMI appns can't use Internet networks as the Communication channel.
 - ③ Applying middleware Services on the BusinessLogics of Rmi appns is Very difficult.
 - ④ RMI registry is a weak registry to handle huge no. of bind & lookup operations. To overcome these problems use Other Distributed Technologies like EJB, HTTPInvoker, Webservices and etc... EJB, HttpInvoker are language dependent. Webservices is language independent.
- SpringEJB provides abstraction Layer on plain EJB. but abstraction layer can not be used in the development of EJB component that is just useful in the development of EJB client App development (so Spring developers never thinks about Spring EJB).
- Spring Webservices provides abstraction Layer on both server and client App development.

HTTP INVOKER

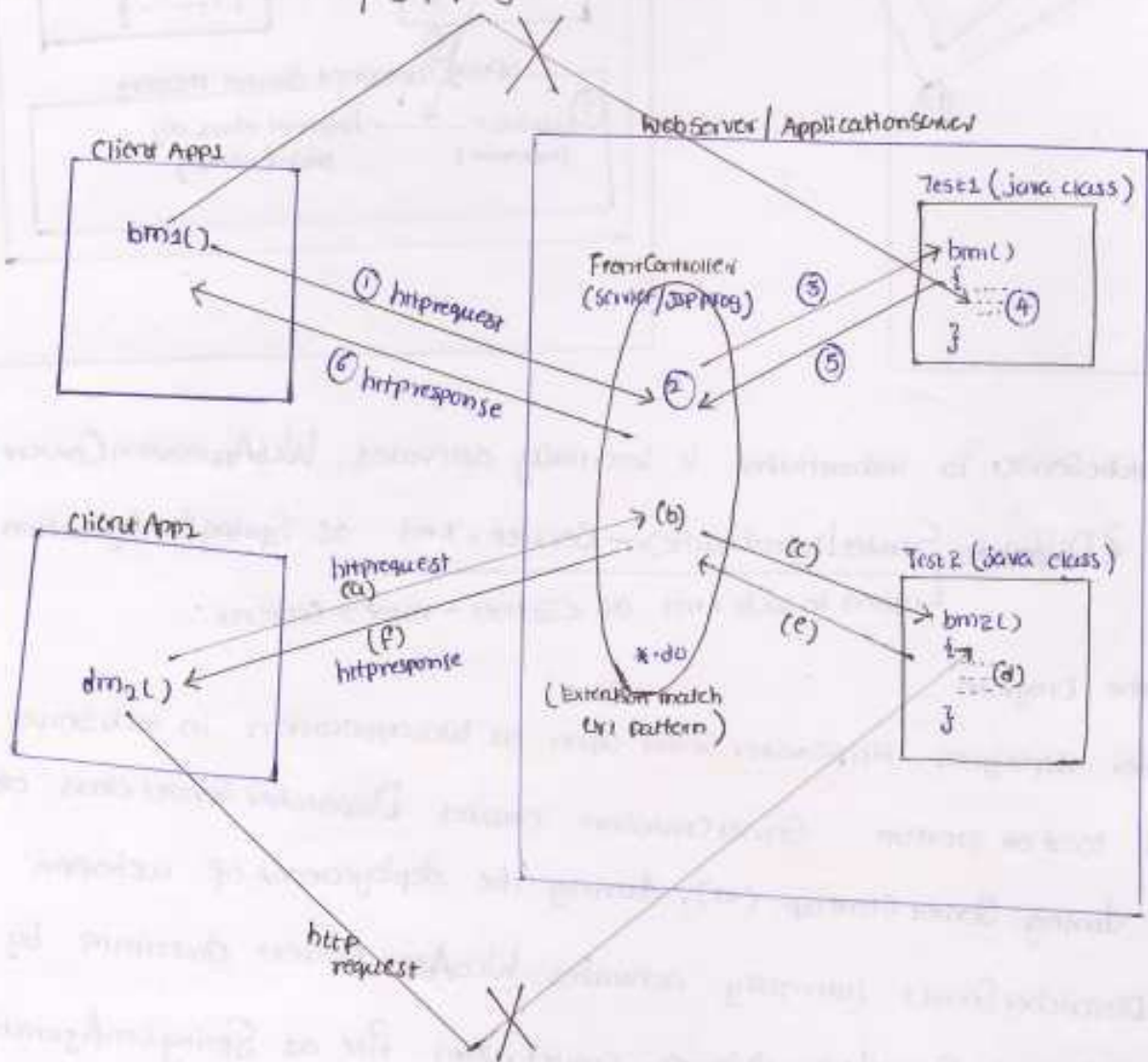
- It is spring's own web based distributed Technology
- Spring Container acts as Registry also
- Here Server appn is nothing but web application deployed in the Server
- Here client appn can be stand alone appn or web appn
- org. sf. remoting. httpinvoker. HttpInvokerServiceExporter keeps B-objs in the registry (Spring Container) having Service interface name as nick name (or) alias name.
- org. sf. remoting. httpinvoker. HttpInvokerProxyFactoryBean gets B-objs reference from registry

- This technology is also language dependent. both server and applications must be written in Java language.

→ The Java classes of webapplication can't take `HttpRequest` direct from client and they can't generate `HttpResponse` direct to client.

To overcome this problem we take one special Servlet program/JSP program to trap and take all `HttpRequest` coming to webappn from clients and to pass the requests to appropriate Java classes.

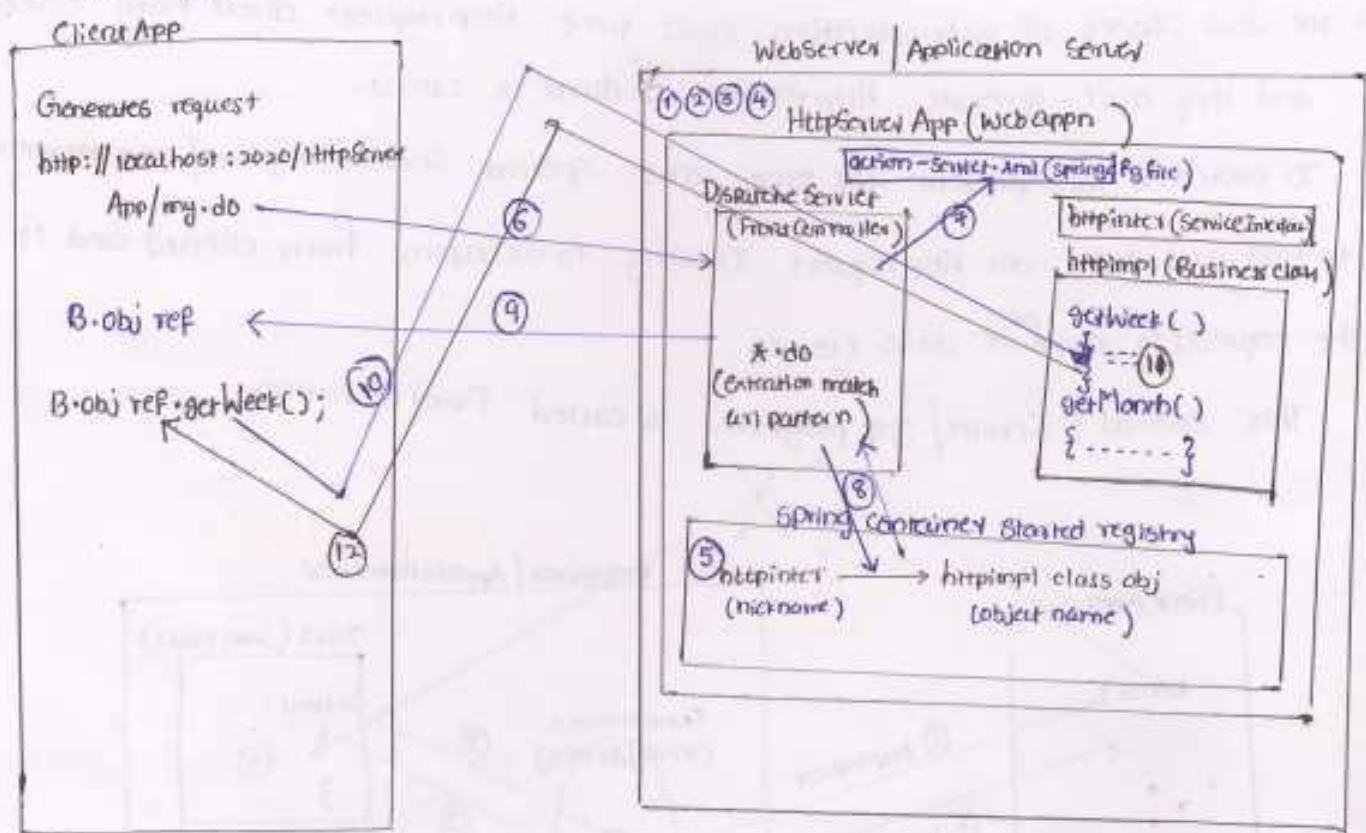
This special Servlet/JSP program is called Front Controller.



→ FrontController is a special webresource program of a that acts as entry & exit point handling all the `HttpRequest` & `HttpResponses` of ordinary Java classes of web appn (refer above diagram)

→ In struts appn `ActionServlet` is Front Controller Servlet. In JSP application `FacesServlet` is `FrontControllerServlet`. In spring appns `org.springframework.web.servlet.DispatcherServlet` is Front Controller Servlet.

→ a Front Controller Servlet must be configured in web.xml file having extension match or Directory match url pattern to trap and take multiple request.



When DispatcherServlet is instantiated it internally activates WebApplicationContext Container by taking <DispatcherServletLogicalName>-Servlet.xml as Spring Configuration file name
 ↳ given in web.xml as <Servlet-name> context

W.r to the Diagram

- ① programmer deploys HttpInvoker Server app as Web application in web server
- ② Bcz of load on startup ServletContainer creates DispatcherServlet class object either during Server startup (or) during the deployment of webapp.
- ③ This DispatcherServlet internally activates WebAppContext Container by taking <DispatcherServletLogicalName>-Servlet.xml file as Spring Configuration file.
- ④ This Spring Container performs pre-instantiation on all the Spring Beans of Spring file.
- ⑤ In the process of preInstantiation the HttpInvokerServiceExporter keeps Business class object in registry slw having ServiceInterface name as nickname/
alias name.
- ⑥ Client appn Uses some request url and HttpInvoker Proxy Factory Bean to send request to Server application.

⑦ As a FrontController the DispatcherServlet traps and takes the request and passes the request to SpringContainer. and this Container uses SimpleUrlHandlerMapping class to bind the request with B.object.

⑧ Based on the above activities the DispatcherServlet gathers B.object reference from Registry also.

⑨ client appn gets B.object reference from Server appn.

⑩ client appn calls business method on Business Object reference.

⑪ Logic of B-method executes in Server appn

⑫ The results generated by B-method comes to client application.

* For example appn on HttpInvoker appn no: ⑪ page no 65, 66 & 67
based Distributed appn

→ A Distributed application can be a Webappn or Non-Webappn.

→ Webappn that uses registry supporting Location Transparency and allows programmable client appns to call the B-methods. is called Distributed appn.

→ HttpInvoker, Webservices are the Webbased distributed appns.

→ Normal Servlet, Jsp webappns are not at all Distributed appns.

The client appn developer of HttpInvoker should gather following details from the Server appn of HttpInvoker.

a) a copy of ServiceInterface / BusinessInterface

b) request url to get Business Object reference

c) Documentation about Business method.

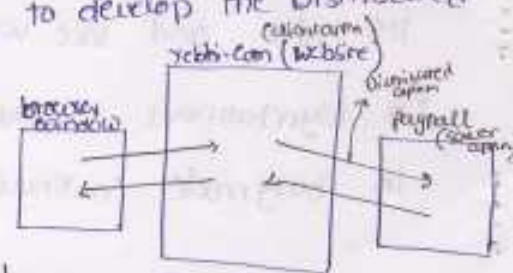
Server appn developer can send this details to client APP Developer through mail or as a content to download.

→ Industry prefers webservices (or) SpringWebServices to develop the Distributed

appn. Otherwise they prefer using HttpInvoker to develop

Springbased Distributed appns. Spring RMI is not industry

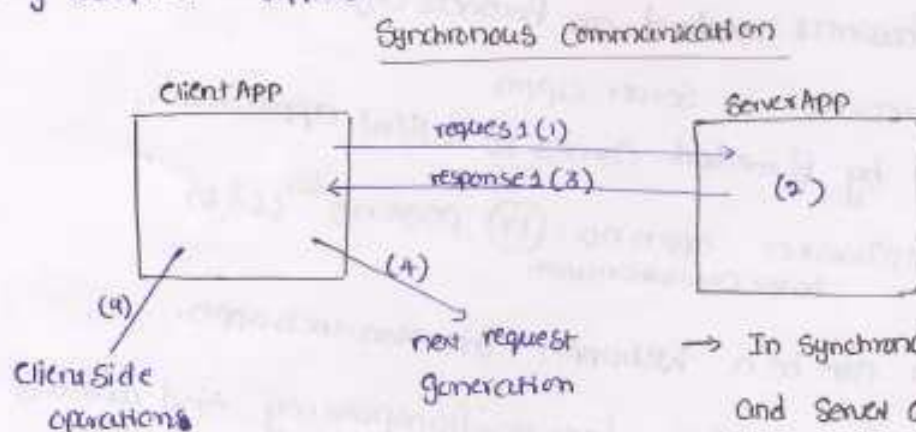
Standard Distributed Technology to develop the Distributed appns.



JMS (Java Message Service) :-

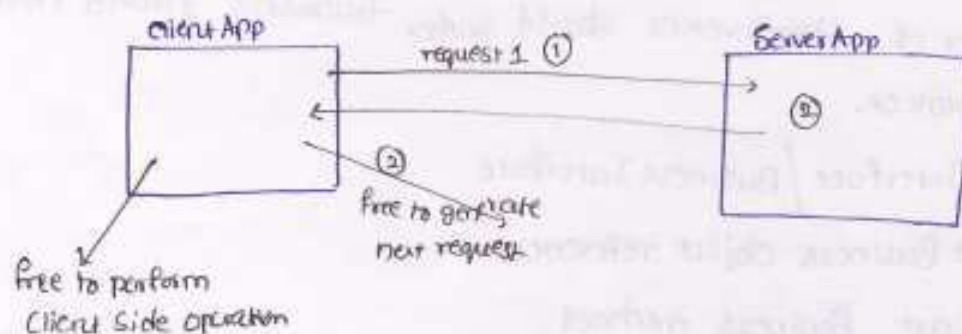
→ If client is blocked or not free to generate next request (or) to perform client side operations until given request related response comes to client from Server, then it is called Synchronous Communication.

→ In Synchronous Communication client apps and Server apps are tightly coupled apps.



→ In Synchronous Communication client and Server apps are tightly coupled apps.

→ If client app needs to perform client side operations and to generate next request without waiting for given request related response from Server app, then it is called "Asynchronous Communication".



→ In Asynchronous Communication client and Server apps are loosely coupled apps.

→ In Synchronous Communication if client app is busy the Server app is idle and vice versa.

In Asynchronous Communication both client and Server apps reside in busy mode continuously.

What Is Messaging?

Messaging is a method of communication between software components or applications. A messaging system is a peer-to-peer facility. A messaging client can send messages to, and receive messages from, any other client. Each client connects to a messaging agent that provides facilities for creating, sending, receiving, and reading messages.

Messaging enables distributed communication that is *loosely coupled*. A component sends a message to a destination, and the recipient can retrieve the message from the destination. However, the sender and the receiver do not have to be available at the same time in order to communicate. In fact, the sender does not need to know anything about the receiver, nor does the receiver need to know anything about the sender. The sender and the receiver need to know only what message format and what destination to use. In this respect, messaging differs from tightly coupled technologies, such as Remote Method Invocation (RMI), which require an application to know a remote application's methods.

Messaging also differs from electronic mail (e-mail), which is a method of communication between people or between software applications and people. Messaging is used for communication between software applications or software components.

What Is the JMS API?

The Java Message Service is a Java API that allows applications to create, send, receive, and read messages. Designed by Sun and several partner companies, the JMS API defines a common set of interfaces and associated semantics that allow programs written in the Java programming language to communicate with other messaging implementations.

The JMS API minimizes the set of concepts a programmer must learn to use messaging products but provides enough features to support sophisticated messaging applications. It also strives to maximize the portability of JMS applications across JMS providers in the same messaging domain. The JMS API enables communication that is not only loosely coupled but also

Asynchronous. A JMS provider can deliver messages to a client as they arrive; a client does not have to request messages in order to receive them.

Reliable. The JMS API can ensure that a message is delivered once and only once. Lower levels of reliability are available for applications that can afford to miss messages or to receive duplicate messages.

The JMS Specification was first published in August 1998. The latest version of the JMS Specification is Version 1.0.2b, which was released in August 2001. You can download a copy of the Specification from the JMS Web site, <http://java.sun.com/products/jms/>.

The JMS API in the J2EE platform has the following features.

Application clients, Enterprise JavaBeans (EJB) components, and web components can send or asynchronously receive a JMS message. Application clients can in addition receive JMS messages asynchronously. (Applets, however, are not required to support the JMS API.)

Message-driven beans, which are a kind of enterprise bean, enable the asynchronous consumption of messages. A JMS provider can optionally implement concurrent processing of messages by message-driven beans.

Message send and receive operations can participate in distributed transactions, which allow JMS operations and database accesses to take place within a single transaction.

Messaging Domains

Before the JMS API existed, most messaging products supported either the *point-to-point* or the *publish/subscribe* approach to messaging. The JMS specification provides a separate domain for each approach and defines compliance for each domain. A stand-alone JMS provider can implement one or both domains. A J2EE provider must implement both domains.

In fact, most implementations of the JMS API support both the point-to-point and the publish/subscribe domains, and some JMS clients combine the use of both domains in a single application. In this way, the JMS API has extended the power and flexibility of messaging products.

Point-to-Point Messaging Domain

A point-to-point (PTP) product or application is built on the concept of message queues, senders, and receivers. Each message is addressed to a specific queue, and receiving clients extract messages from the queues established to hold their messages. Queues retain all messages sent to them until the messages are consumed or until the messages expire.

Telephone
connection with answering machine
is the best example to understand
the messaging domain behavior

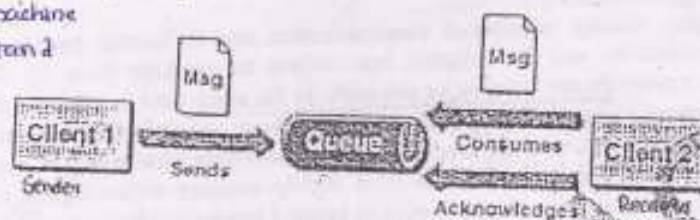


Figure 33-3 Point-to-Point Messaging

PTP messaging has the following characteristics

1. Each message has only one consumer.
2. A sender and a receiver of a message have no timing dependencies. The receiver can fetch the message whether or not it was running when the client sent the message.
3. The receiver acknowledges the successful processing of a message.

Use PTP messaging when every message you send must be processed successfully by one consumer.

Publish/Subscribe Messaging Domain

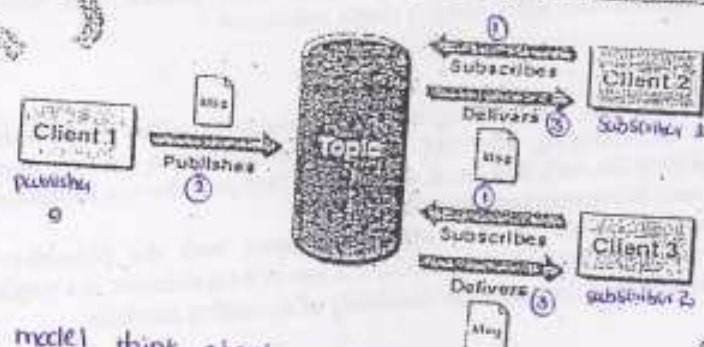
In a publish/subscribe (pub/sub) product or application, clients address messages to a topic, which functions somewhat like a bulletin board. Publishers and subscribers are generally anonymous and can dynamically publish or subscribe to the content hierarchy. The system takes care of distributing the messages arriving from a topic's multiple publishers to its multiple subscribers. Topics retain messages only as long as it takes to distribute them to current subscribers.

Pub/sub messaging has the following characteristics.

1. Each message can have multiple consumers.
2. Publishers and subscribers have a timing dependency. A client that subscribes to a topic can consume only messages published after the client has created a subscription, and the subscriber must continue to be active in order for it to consume messages.

The JMS API relaxes this timing dependency to some extent by allowing subscribers to create durable subscriptions, which receive messages sent while the subscribers are not active. Durable subscriptions provide the flexibility and reliability of queues but still allow clients to send messages to many recipients.

Use pub/sub messaging when each message can be processed by zero, one, or many consumers.



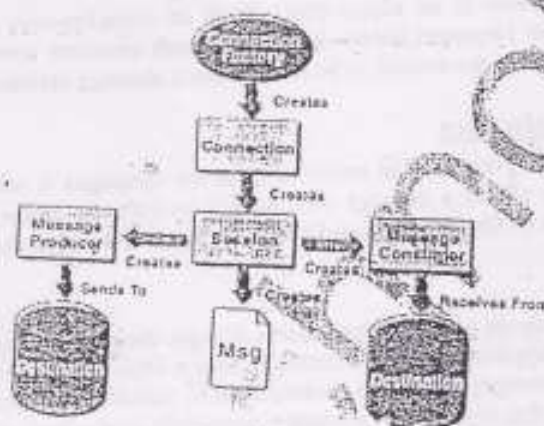
To understand pub/sub model think about satellite TV channel broadcasting system.

Message Consumption

Messaging products are inherently asynchronous: There is no fundamental timing dependency between the production and the consumption of a message. However, the JMS specification uses this term in a more precise sense. Messages can be consumed in either of two ways:

Synchronously: A subscriber or a receiver explicitly fetches the message from the destination by calling the receive method. The receive method can block until a message arrives or can time out if a message does not arrive within a specified time limit.

Asynchronously: A client can register a *message listener* with a consumer. A message listener is similar to an event listener. Whenever a message arrives at the destination, the JMS provider delivers the message by calling the listener's `onMessage` method, which acts on the contents of the message.



Administered Objects

Two parts of a JMS application—destinations and connection factories—are best maintained administratively rather than programmatically. The technology underlying these objects is likely to be very different from one implementation of the JMS API to another. Therefore, the management of these objects belongs with other administrative tasks that vary from provider to provider.

Connection Factories

A *connection factory* is the object a client uses to create a connection to a provider. A connection factory encapsulates a set of connection configuration parameters that has been defined by an administrator. Each connection factory is an instance of the `ConnectionFactory`, `QueueConnectionFactory`, or `TopicConnectionFactory` interface.

Destinations

A *destination* is the object a client uses to specify the target of messages it produces and the source of messages it consumes. In the PTP messaging domain, destinations are called queues. In the pub/sub messaging domain, destinations are called topics.

Connections

A *connection* encapsulates a virtual connection with a JMS provider. A connection could represent an open TCP/IP socket between a client and a provider service daemon. You use a connection to create one or more sessions.

Sessions

A *session* is a single-threaded context for producing and consuming messages. You use sessions to create message producers, message consumers, and messages.

Message Producers

A *message producer* is an object that is created by a session and used for sending messages to a destination. It implements the `MessageProducer` interface.

Message Consumers

A *message consumer* is an object that is created by a session and used for receiving messages sent to a destination. It implements the `MessageConsumer` interface.

A message consumer allows a JMS client to register interest in a destination with a JMS provider. The JMS provider manages the delivery of messages from a destination to the registered consumers of the destination.

Message Listeners

A *message listener* is an object that acts as an asynchronous event handler for messages. This object implements the `MessageListener` interface, which contains one method, `onMessage`. In the `onMessage` method, you define the actions to be taken when a message arrives.

Message Selectors

If your messaging application needs to filter the messages it receives, you can use a JMS API message selector, which allows a message consumer to specify the messages it is interested in. Message selectors assign the work of filtering messages to the JMS provider rather than to the application.

Messages

The ultimate purpose of a JMS application is to produce and to consume messages that can then be used by other software applications. JMS messages have a basic format that is simple but highly flexible, allowing you to create messages that match formats used by non-JMS applications on heterogeneous platforms.

A JMS message has three parts: a header, properties, and a body. Only the header is required.

Message Headers

A JMS message header contains a number of predefined fields that contain values that both clients and providers use to identify and to route messages.

Message Properties

You can create and set properties for messages if you need values in addition to those provided by the header fields. You can use properties to provide compatibility with other messaging systems, or you can use them to create message selectors.

Message Bodies

The JMS API defines five message body formats, also called message types, which allow you to send and to receive data in many different forms and provide compatibility with existing messaging formats.

Message Type	Body Contains
TextMessage	A <code>java.lang.String</code> object (for example, the contents of an Extensible Markup Language file).
MapMessage	A set of name-value pairs, with names as <code>String</code> objects and values as primitive types in the Java programming language. The entries can be accessed sequentially by enumerator or randomly by name. The order of the entries is undefined.
BytesMessage	A stream of uninterpreted bytes. This message type is for literally encoding a body to match an existing message format.
StreamMessage	A stream of primitive values in the Java programming language, filled and read sequentially.
ObjectMessage	A <code>Serializable</code> object in the Java programming language.
Message	Nothing. Composed of header fields and properties only. This message type is useful when a message body is not required.


```

76 QueueSession qsession=qconn.createQueueSession(false,Session.AUTO_ACKNOWLEDGE);
77
78 Queue queue=(Queue)ctx.lookup("QueueJndi");
79
80 String selector="Name IN('Bazee','Hotmail')"; // Message Selector to filter the messages
81 QueueReceiver qreceiver=qsession.createReceiver(queue,selector);
82
83 MyQueueReceiver mq=new MyQueueReceiver(); // current class obj
84 qreceiver.setMessageListener(mq); // creating receiver pointing to Queue destination
85
86 qconn.start(); // registering message listener with our current class object
87 // starts listening to message store to receive the messages.
88 }
89
90 public void onMessage(Message msg) // represents received msg each time
91 { // Event handling method that executes when each message comes to
92   try { // message store
93     TextMessage tmsg=(TextMessage)msg;
94     System.out.println("Received Message is:"+tmsg.getText());
95   } catch (Exception e) { // 6-2-P ("message property" + tmsg.getStringProperty("name"));
96     e.printStackTrace();
97   }
98 }
99 }
100

```

101 ----- (App2) On Topic -----

```

102 ----- TopicDemo.java (Publisher app) -----
103
104 import javax.naming.*;
105 import java.util.Properties;
106 import javax.jms.*; // JMS API
107
108 public class TopicDemo
109 {
110   public static final String TOPIC_FACTORY = "jmsConnectionFactory";
111   public static final String TOPIC="TopicJndi";
112   public static void main(String args[]) throws NamingException, JMSEException
113   {
114     Properties prop=new Properties(); // JNDI Properties of GlassFish registry
115     prop.put(Context.INITIAL_CONTEXT_FACTORY,"com.sun.enterprise.naming.SerialInitContextFactory");
116     prop.put(Context.PROVIDER_URL,"http://localhost:4848");
117     Context ctx=new InitialContext(prop);
118     TopicDemo tt=new TopicDemo(); // current class object
119     tt.doPub(ctx); // User defined method is called
120     ctx.close();
121   }
122
123   public void doPub(Context ctx) throws JMSEException, NamingException
124   {
125     Topic topic=(Topic)ctx.lookup(TOPIC); // get access to the Topic destination of GlassFish registry
126
127     TopicConnectionFactory tcf = (TopicConnectionFactory)ctx.lookup(TOPIC_FACTORY);
128     TopicConnection tc = tcf.createTopicConnection(); // (X2)
129     TopicSession ts=tc.createTopicSession(false,TopicSession.AUTO_ACKNOWLEDGE); // Gets JMS connection from ConnectionFactory
130
131     TopicPublisher tp=ts.createPublisher(topic); // Publisher is created pointing to Topic
132     tp.start();
133
134     TextMessage msg;
135     msg=ts.createTextMessage("This is from Sathya Techs112.1"); // creating one Text msg.
136     System.out.println("Publishing msg on the Destination(Topic)");
137
138     tp.publish(msg); // Sends one Text msg to the Topic destination
139     tc.close();
140   }
141 }
142
143 ----- SubscriberDemo.java -----
144
145 import javax.naming.*;
146 import java.util.Properties;
147 import javax.jms.*;
148 public class SubscriberDemo{
149   public static void main(String args[]) throws NamingException, JMSEException {
150     Properties prop=new Properties();

```

Jar files in classpath same as previous app①

1st run subscriber app for multiple times from different windows then run the publisher app for one time.


```

151 prop.put(Context.INITIAL_CONTEXT_FACTORY, "com.sun.enterprise.naming.SerialInitContextFactory");
152 prop.put(Context.PROVIDER_URL, "iiop://localhost:4848");
153
154 Context ctx = new InitialContext(prop);
155
156 SubscriberDemo tt = new SubscriberDemo(); → Current class object
157 tt.doSub(ctx); → User defined method call ref for line no: 160-177
158 ctx.close();
159 }
160 public void doSub(Context ctx) throws JMSException, NamingException{
161
162     Topic topic = (Topic) ctx.lookup("TopicJndi"); → getting access to the topic destination of GrassFish registry
163
164     TopicConnectionFactory tcf = (TopicConnectionFactory) ctx.lookup("ConFactJndi");
165     TopicConnection tc = tcf.createTopicConnection(); → gets JMS connection from Connection Factory
166
167     TopicSession ts = tc.createTopicSession(false, TopicSession.AUTO_ACKNOWLEDGE);
168
169     TopicSubscriber tsub = ts.createSubscriber(topic); → TopicSubscriber is created pointing to
170     tc.start(); Topic Destination.
171
172     TextMessage msg;
173     msg = (TextMessage) tsub.receive(); → receives msg from Topic destination
174
175     System.out.println("Received message is: " + msg.getText());
176     tc.close(); → Displays the content of the msg.
177 }
178 }

```

Applications on Spring JMS (App4) (on queue)

Sender App (on Queue)

```

184 MessageSender.java (Spring Bean) having logic to send Text message
185 import javax.jms.*;
186 import org.springframework.jms.core.*; (*) code to inject JMS template class object through setter injection
187 public class MessageSender {
188
189     private JmsTemplate jt; jar files in classpath
190
191     public MessageSender() {} Spring jar files : ②
192
193     public void setJt(JmsTemplate jt) Same 6 jar files of App① in handout
194     { GrassFish server should be in running mode
195         this.jt = jt;
196     }
197
198
199     → User Defined B. method
200     public void sendMessage()
201     {
202         MessageCreator creator = new MessageCreator()
203         {
204             public Message createMessage(Session session)
205             {
206                 TextMessage message = null;
207                 try
208                 {
209                     Creating the Text msg
210                     message = session.createTextMessage();
211                     message.setStringProperty("text", "Hello World");
212                     message.setStringProperty("from", "Sudhakar U");
213                     catch (JMSException e)
214                     {
215                         e.printStackTrace();
216                     }
217                     return message;
218                 }
219             }
220
221             202-218: here object for Anonymous inner class that implements MessageCreator (Interface) is created representing one JMS message.
222             jt.send(creator);
223         }
224         → sends the msg
225     }

```

spring.xml (Spring Config file) having the logic to inject JMS template class object.

```

<?xml version="1.0" encoding="UTF-8"?>

```


(1) Jndi properties of glassfish registry.

```

226 <!DOCTYPE beans PUBLIC "-//SPRING/DTD BEAN/EN"
227 "http://www.springframework.org/dtd/spring-beans.dtd">
228 <beans>
229 <bean id="cf" class="org.springframework.jndi.JndiObjectFactoryBean">
230 <property name="jndiEnvironment">
231 <props>
232 <prop key="java.naming.factory.initial">com.sun.enterprise.naming.SerialInitContextFactory</prop>
233 <prop key="PROVIDER_URL">iiop://localhost:4848</prop>
234 </props>
235 </property>
236 <property name="jndiName" value="@ConnectionFactory">
237 </bean>
238
239
240 <bean id="q" class="org.springframework.jndi.JndiObjectFactoryBean">
241 <property name="jndiEnvironment">
242 <props>
243 <prop key="java.naming.factory.initial">com.sun.enterprise.naming.SerialInitContextFactory</prop>
244 <prop key="PROVIDER_URL">iiop://localhost:4848</prop>
245 </props>
246 </property>
247 <property name="jndiName" value="QueueJndi">
248 </bean>
249
250
251 <bean id="template" class="org.springframework.jms.core.JmsTemplate">
252 <property name="connectionFactory" ref="cf">
253 <property name="defaultDestination" ref="q">
254 </bean>
255
256
257 <bean id="ms" class="MessageSender">
258 <property name="jt" ref="template">
259 </bean>
260 </beans>

```

Handwritten notes on the code:

- 233: *java.naming.provider.url* (4848)
- 244: *java.naming.provider.url* (4848)
- 252: *ref for line: 229*
- 253: *ref for line: 240*
- 256: *our SpringBean configuration having the logic to inject JmsTemplate class object.*
- 258: *ref for line no: 251*

```

261 ----- SendClient.java (Client app)
262 import org.springframework.context.support.*;
263 public class sendclient
264 {
265     public static void main(String args[])
266     {
267         FileSystemXmlApplicationContext ctx=new FileSystemXmlApplicationContext("spring.xml");
268         MessageSender ms =(MessageSender)ctx.getBean("ms");
269         ms.sendMessage();
270         System.out.println("message sent");
271     }
272 }

```

273 }
274
275 Receiver App

```

276 ----- MessageReceiver.java (Spring Bean)
277 import javax.jms.*;
278 import org.springframework.jms.core.*;
279
280 public class MessageReceiver {
281     private JmsTemplate jt;
282
283     public MessageReceiver() {}
284
285     public void setJmsTemplate(JmsTemplate jt)
286     {
287         this.jt = jt;
288     }
289
290     public void receiveMessage()
291     {
292         Message message = jt.receive();
293         TextMessage tmsg = null;
294         if (message instanceof TextMessage)
295         {
296             tmsg=(TextMessage)message;

```

jar files in class path; - same as SenderApp.

Receiver
 → *.jar
 → Spring.xml

Receiver> javac *.java
 > java ReceiverClient

```

301     try
302     {
303         System.out.println(msg.getText());
304         System.out.println(msg.getStringProperty("text"));
305     }
306     catch (JMSEException e)
307     {
308         e.printStackTrace();
309     }
310 } // if
311 } // method
312 } // class
313 -----spring.xml-----
314 <!DOCTYPE beans PUBLIC "-//SPRING/DTD BEAN//EN"
315 "http://www.springframework.org/dtd/spring-beans.dtd">
316 <beans>
317     <bean id="cf" class="org.springframework.jndi.JndiObjectFactoryBean">
318         <property name="jndiEnvironment">
319             <props>
320                 <prop key="java.naming.factory.initial">com.sun.enterprise.naming.SerialInitContextFactory</prop>
321                 <prop key="PROVIDER_URL">iiop://localhost:4444</prop>
322             </props>
323         </property>
324         <property name="jndiName" value="ConFactJndi"/>
325     </bean>
326     <bean id="q" class="org.springframework.jndi.JndiObjectFactoryBean">
327         <property name="jndiEnvironment">
328             <props>
329                 <prop key="java.naming.factory.initial">com.sun.enterprise.naming.SerialInitContextFactory</prop>
330                 <prop key="PROVIDER_URL">iiop://localhost:4444</prop>
331             </props>
332         </property>
333         <property name="jndiName" value="QueueJndi"/>
334     </bean>
335     <bean id="template" class="org.springframework.jms.core.JmsTemplate">
336         <property name="connectionFactory" ref="cf"/>
337         <property name="defaultDestination" ref="q"/>
338     </bean>
339     <bean id="mr" class="MessageReceiver">
340         <property name="jt" ref="template"/>
341     </bean>
342 </beans>
343 -----ReceiveClient.java-----
344 import org.springframework.context.support.*;
345 public class ReceiveClient
346 {
347     public static void main(String args[])
348     {
349         FileSystemXmlApplicationContext ctx = new FileSystemXmlApplicationContext("spring.xml");
350         MessageReceiver mr = (MessageReceiver) ctx.getBean("mr");
351         mr.receiveMessage();
352     }
353 }
354 -----App5 (on Topic)
355 -----Sender App (publisher app)
356 -----MessageSender.java (Spring Bean)
357 import javax.jms.*;
358 import org.springframework.jms.core.*;
359 public class MessageSender {
360     private JmsTemplate jt
361     public MessageSender() {}
362     public void setJt(JmsTemplate jt)
363     {
364         this.jt = jt;
365     }

```



```

376
377 public void sendMessage()
378 {
379     MessageCreator creator = new MessageCreator()
380     {
381         public Message createMessage(Session session)
382         {
383             TextMessage message = null;
384             try
385             {
386                 message = session.createTextMessage();
387                 message.setStringProperty("text", "Hello World");
388                 message.setText("this is test msg");
389             }
390             catch (JMSException e)
391             {
392                 e.printStackTrace();
393             }
394             return message;
395         }
396     };
397     JmsSend(creator); → Sends the message to Topic destination.
398 }
399 }
400 }
401 }

```

```

402 -----spring.xml (spring.cfg file)
403 <?xml version="1.0" encoding="UTF-8"?>
404 <!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
405 "http://www.springframework.org/dtd/spring-beans.dtd">
406 <beans>
407     <bean id="cf" class="org.springframework.jndi.JndiObjectFactoryBean">
408         <property name="jndiEnvironment">
409             <props>
410                 <prop key="java.naming.factory.initial">com.sun.enterprise.naming.SerialInitContextFactory</prop>
411                 <prop key="PROVIDER_URL">iiop://localhost:4048</prop>
412             </props>
413             </property>
414             <property name="jndiName" value="ConFactJndi"/>
415         </bean>
416
417     <bean id="q" class="org.springframework.jndi.JndiObjectFactoryBean">
418         <property name="jndiEnvironment">
419             <props>
420                 <prop key="java.naming.factory.initial">com.sun.enterprise.naming.SerialInitContextFactory</prop>
421                 <prop key="PROVIDER_URL">iiop://localhost:4048</prop>
422             </props>
423             </property>
424             <property name="jndiName" value="TopicJndi"/>
425         </bean>
426
427     <bean id="template" class="org.springframework.jms.core.JmsTemplate">
428         <property name="connectionFactory" ref="cf"/>
429         <property name="defaultDestination" ref="q"/>
430     </bean>
431
432     <bean id="ms" class="MessageSender">
433         <property name="jt" ref="template"/>
434     </bean>
435 </beans>

```

```

440 -----SendClient.java
441 import org.springframework.context.support.*;
442 public class sendclient
443 {
444     public static void main(String args[])
445     {
446         FileSystemXmlApplicationContext ctx = new FileSystemXmlApplicationContext("spring.xml");
447         MessageSender ms = (MessageSender) ctx.getBean("ms");
448         ms.sendMessage();
449         System.out.println("message sent");
450     }

```

Java files in CLASSPATH: Same as previous apps.

```

Sender
├── *.java(s)
└── spring.xml
Sender > javac *.java
> java SendClient

```

```

451 )
452 }
453 -----
454 Receiver App (Subscriber)
455 -----
456 -----MessageReceiver.java-----
457 import javax.jms.*;
458 import org.springframework.jms.core.*;
459
460 public class MessageReceiver {
461     private JmsTemplate jt;
462
463     public MessageReceiver() {}
464
465     public void setJmsTemplate(JmsTemplate jt)
466     {
467         this.jt = jt;
468     }
469
470     public void receiveMessage()
471     {
472         Message message = jt.receive();
473         TextMessage tmsg = null;
474         if (message instanceof TextMessage)
475         {
476             tmsg=(TextMessage)message;
477             try
478             {
479                 System.out.println(tmsg.getText());
480                 System.out.println(tmsg.getStringProperty("text"));
481             }
482             catch (JMSException e)
483             {
484                 e.printStackTrace();
485             }
486         }
487     }
488 }
489 }
490 }
491 }
492 -----spring.xml-----
493 <!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
494 "http://www.springframework.org/dtd/spring-beans.dtd">
495 <beans>
496
497 <bean id="cf" class="org.springframework.jndi.JndiObjectFactoryBean">
498     <property name="jndiEnvironment">
499         <props>
500             <prop key="java.naming.factory.initial">com.sun.enterprise.naming.SerialInitContextFactory</prop>
501             <prop key="PROVIDER_URL">iiop://localhost:4848</prop>
502         </props>
503     </property>
504     <property name="jndiName" value="ConFactJndi"/>
505 </bean>
506
507 <bean id="q" class="org.springframework.jndi.JndiObjectFactoryBean">
508     <property name="jndiEnvironment">
509         <props>
510             <prop key="java.naming.factory.initial">com.sun.enterprise.naming.SerialInitContextFactory</prop>
511             <prop key="PROVIDER_URL">iiop://localhost:4848</prop>
512         </props>
513     </property>
514     <property name="jndiName" value="QueueJndi"/>
515 </bean>
516
517 <bean id="template" class="org.springframework.jms.core.JmsTemplate">
518     <property name="connectionFactory" ref="cf"/>
519     <property name="defaultDestination" ref="q"/>
520 </bean>
521
522 <bean id="ms" class="MessageReceiver">
523     <property name="jt" ref="template"/>
524 </bean>
525 </beans>

```

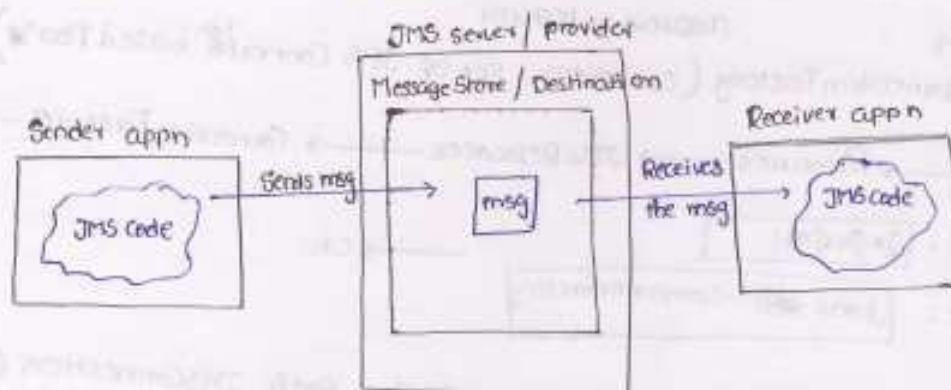
Receiver
 → *.java(2)
 → spring.xml

> javac *.java
 > java ReceiverClient

First start multiple Receiver / Subscriber
 apps ~~and~~ then start only one sender app
 (publisher app)


```
526 -----ReceiveClient.java-----
527 import org.springframework.context.support.*;
528 public class ReceiveClient
529 {
530     public static void main(String args[])
531     {
532         FileSystemXmlApplicationContext ctx=new FileSystemXmlApplicationContext("spring.xml");
533         MessageReceiver mr =(MessageReceiver)ctx.getBean("mr");
534         mr.receiveMessage();
535     }
536 }
537 }
538 -----
```

- To achieve asynchronous communication b/w browser window and web appn use either Ajax (client side Technology) or JavaPortlets (Server side Technology)
- To achieve messages based asynchronous communication between two standalone Java appns use JMS based messaging.



- JMS is given to perform Messages based Asynchronous Communication between two Java appns.

- JMS server / provider Supplies destinations having capability to store messages

temporarily

Eg:- JMS server / providers

MQSeries from IBM

MSMQ from Microsoft

WebLogic Message server from WebLogic

Glassfish Message server from Glassfish

- Every ^{Application} ~~Server~~ Server supplies one built-in JMS server.

For Basics for messaging and JMS refer page no's 1 to 4 of The supplementary handout given on 02/04/2013

- JMS supports two messaging domains

1) PTP (Point to Point) Domain

→ uses Queue as Destination / Message store

2) pub/sub Domain

→ uses Topic as the Destination / message store

For related info on these messages domain refer page no's 1 & 2 of the 02/04/2013.

(*) Procedure to create in GlassFish 2-x server

Step ①:- Start mydomain2 server of GlassFish. and open its AdminConsole.

Run \Appserver\bin> asadmin start-domain mydomain2

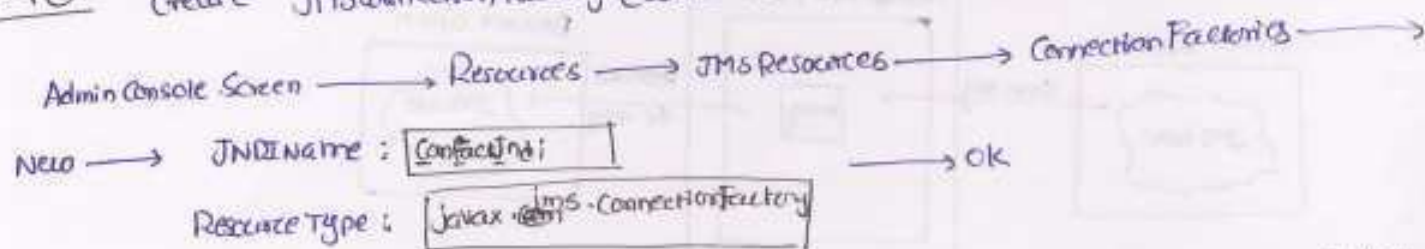
Step ②:-

http://localhost:4848

Login: testuser

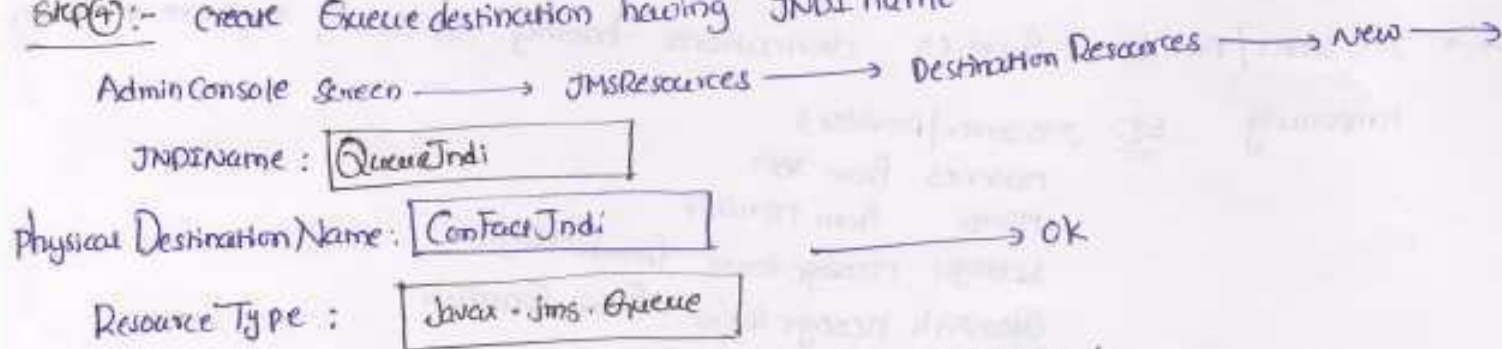
Password: testuser4

Step ③:- Create JMSConnectionFactory (Contains set of JMS Connections based pools)

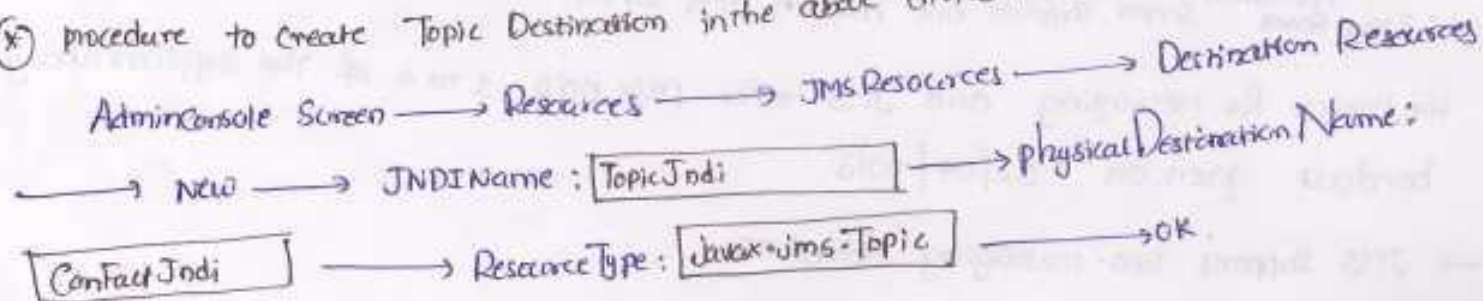


JMSConnectionFactory represents JMSConnection pool. Each JMSConnection Object represents the Connection b/w client appn and JMS server.

Step ④:- Create Queue destination having JNDI name



(*) procedure to create Topic Destination in the above environment



→ JMS is part of JEE module JMS api means working with javax.jms pkg
the jar files the JEE apis contains JMS api

in Weblogic server → weblogic.jar

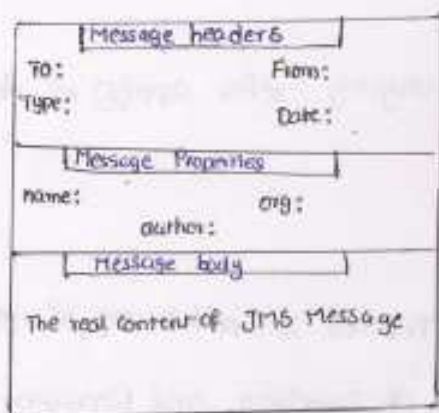
in GlassFish server → javase.jar

in JBoss server → Jboss-javase.jar

Sender/publisher App uses JMS api to send ^{msgs} to the Message store

Receiver App/subscriber App uses JMS api to receive msgs from the Message

The Structure of JMS message



- header names are Fixed
- Property names and values are programmer choice names and values
- To send additional info along with Message we need to create custom Message Properties

→ There is a possibility of developing 5 types of Messages for related info on these messages refer page no: ④ of 02/04/2013 handout.

→ Important resources of JMS api

related Queue

QueueConnectionFactory (I)

QueueConnection (I)

Queue (I)

QueueSession (I)

QueueSender

QueueReceiver

and etc....

related Topic

TopicConnectionFactory (I)

TopicConnection (I)

Topic (I)

TopicSession (I)

Topic Publisher

Topic Subscriber

and etc....

related to Messages

TextMessage

StreamMessage

MapMessage

ObjectMessage

and etc....

* For plain JMS API based Sender, receiver application dealing with Queue destination based PTOP Messaging Domain refer appn ① of the 02/04/2013 handout

→ The realtime example applications of JMS are

- ① Complaint management
- ② Bugs management
- ③ OFFline order booking and etc..

⊛ For plain JMS based pub/sub model of messaging refer app(2) of the page no's ⑥ & ⑦ of 22/02/13

Spring JMS :-

- provides abstraction layer of plain JMS. For this it supplies `org.springframework.jms.core.JmsTemplate` class.
- This `JmsTemplate` class simplifies the process of sending and receiving msgs.
- No need of working with plain JMS API for JMS operations
- Gives Callback Interfaces to ~~develop~~^{create} diff types of messages.
- To create `JmsTemplate` class obj `JmsConnectionFactory` obj, Topic/Queue destination obj's are required.
- Spring JMS gives the Callback Interface called `MessageCreator` to create diff types of message creators by using the exposed `JmsSession` object.

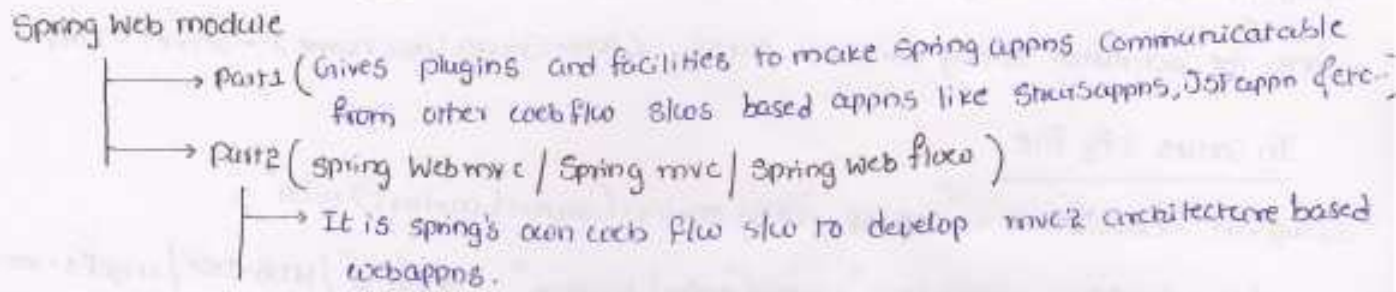
⊛ for Spring JMS based P-to-P example application refer app(4) of the page no's ⑦ to ⑨

⊛ for example app on Spring JMS based pub-sub messaging domain app(5) of the page no's ⑨ to ⑫

Module-V

Spring WebModule

Spring web module



Struts and Spring Integration :-

- In this app Struts app represents **view layer and Controller layer logics.**
(Presentation logic) (Integration logic)
- and Spring app represents **Model layer logics.**
(Business Logic & Persistence logic)

The two ways of Struts and Spring Integration.

- ① Struts with Spring → Struts and Spring apps reside and execute on the same JVM



- Struts app is local client app to Spring app
→ here Spring app must not be a Distributed app

- ② Struts to Spring

- here Struts app and Spring app resides on two different JVMs of same or different computer
→ Struts app is Remote client app to Spring app.
→ here Spring app must be a Distributed app.

→ The plugin required for Struts 1.x and Spring integration is

org.springframework.web.struts.ContextLoaderPlugin

and this plugin is supplied by

Spring slw to configure in Struts Configuration file by using `<plugin>` tag. ActionServlet recognizes all the plugins configured in Struts Configuration file the moment it is instantiated (Object creation). ActionServlet instantiation takes place either during Server startup or during the deployment of webapp. because we generally enable load-on-startup on ActionServlet.

→ The above ContextLoaderPlugin activates Webapp Context Spring Container by taking specified file name (as) ActionServlet logic name - `service.xml` as Spring Configuration file name.

→ In this process all the beans that are configured will be pre instantiated

In struts cfg file

```
<plug-in className="org.springframework.web.struts.ContextLoaderPlugIn"/>
```

here the activated spring container takes `<ActionServletLogicalName>-servlet.xml`

In struts cfg file

```
<plug-in className="org.sf.web.struts.ContextLoaderPlugIn">
```

```
  <set-property property="contextConfigLocation" value="/WEB-INF/mycfg1.xml/WEB-INF/mycfg2.xml"/>
```

```
</plug-in>
```

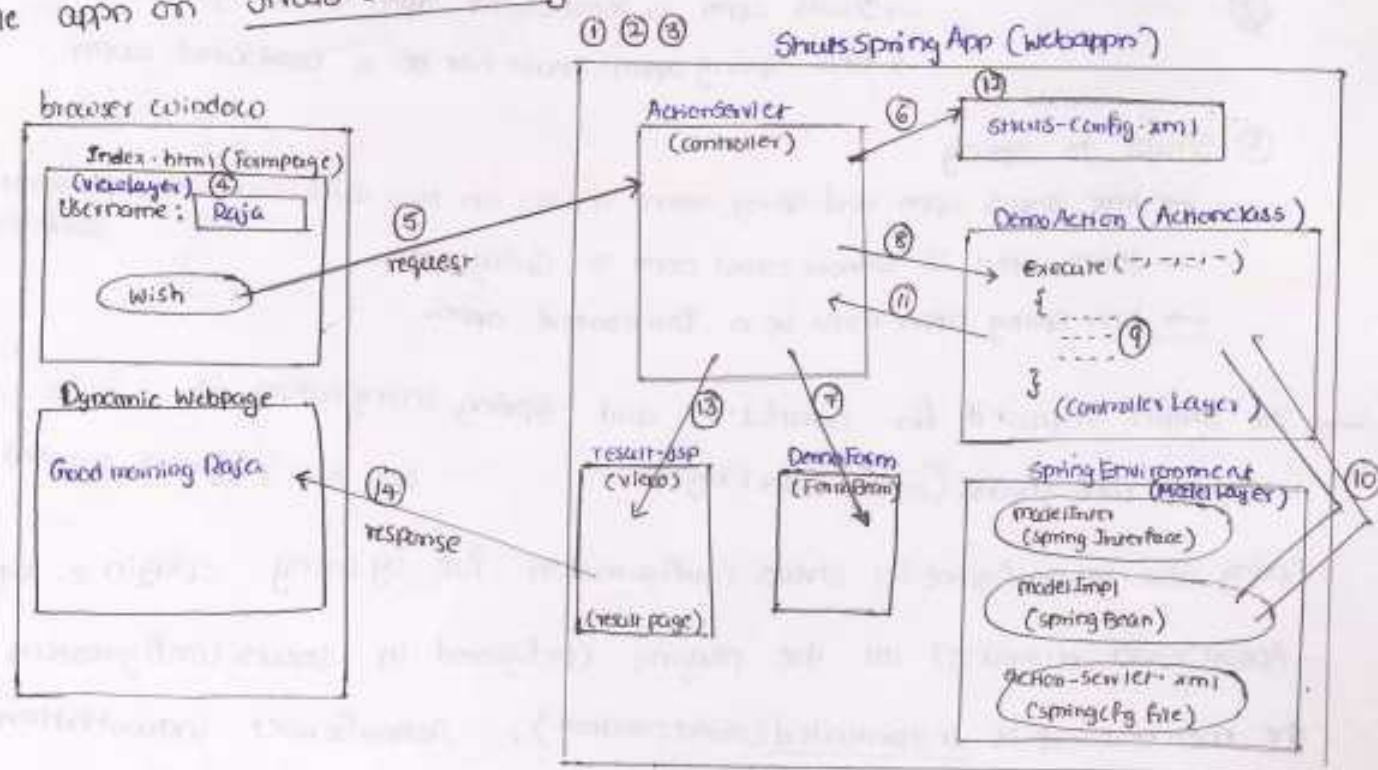
→ here the activated spring container takes mycfg1.xml, mycfg2.xml files as Spring Configuration files.

→ In struts with spring app development the struts Actionclass should contain

Client code and should call the B-methods of SpringBean available

in Spring app.

Example app on "struts with spring"



W.r.to the diagram

- ① programmer deploys struts spring application in webserver or ApplicationServer.
 - ② because of LoadStartup Servlet container creates ActionServlet class obj either during Server startup (or) during the deployment of webapp.
 - ③ ActionServlet activates ContextLoadPlugIn. this plugin activates the webApplicationContext containing by taking `<ActionServletLogicalName>-servlet.xml` as Spring Configuration file.
- In this process all the beans of Spring Configuration file will be pre-instantiated

④ & ⑤ enduser launch the Form page and submit the request to webapp.

⑥ ActionServlet traps and takes the request and uses Struts Configuration file entries to decide Formbean, and Actionclass that are required to process the request

⑦ ActionServlet writes the Received Form data to FormBean class object

⑧ ActionServlet calls execute(-,-,-,-) method of Actionclass.

⑨ This execute(-,-,-,-) method somehow manages to get SpringBean class object

⑩ execute(-,-,-,-) method calls Business method of Spring Bean and gets the result.

keeps this result in request attribute to make it visible to result page.

⑪ This execute(-,-,-,-) method returns the Control back to ActionServlet.

⑫ ActionServlet uses Struts Configuration file entries to decide ^{the} result page of Action class

⑬ ActionServlet passes the Control to result page

⑭ Result page formats the results and sends the results to browser window as the dynamic webpage content

There are two approaches to develop Struts with Spring appn.

Approach 1) By making Struts Action class as Spring Aware class with the support of

XxxSupport classes (like ActionSupport, DispatchActionSupport, ...)

Approach 2) By Injecting SpringBean class object to Struts Action class property (for this we need to cfg Struts Action class as Spring Bean in Spring cfg file)

Approach ① discussions :-

When Struts Action class extends from XxxSupport class of Spring api. The

Struts Action class ^{can get access to} ~~introduced~~ the ContextLoader PlugIn activated SpringContainer.

and we can ^{use} ~~use~~ this SpringContainer to get access to SpringBean class object and to call B. methods on it.

→ in Struts and Spring integration always prefer working with ContextLoader PlugIn ^{required}

Spring Container. because this container can be used in multiple Action classes and

this container will be activated either during Server startup (or) during the

deployment of webapp. This process improves the performance when compared to

activating Spring Container in every Action class Separately.

→ Sample Struts Action class code of approach (1) :-

```
public class TestAction extends ActionSupport
{
    // (this internally extends from Action class of Struts 1.x api)
    public ActionForward execute(-, -, -, -)
    {
        // get access to the Spring Container activated by ContextLoaderPlugin
        WebApplicationContext ctx = getWebApplicationContext();
        // get access to Spring Bean class obj
        modelinter bobj = (modelinter) ctx.getBean("mt");
        // beanid
        // call the B-method of Spring Bean
        bobj.bmd(-);
    }
}
```

(*) For the above diagram based and the above approach (1) based Struts 1.x with Spring appn refer appn (23) page no: 75-77

for different approaches of Struts & Spring appn ^{development} based related documentations refer page no's (26) to (34) of the material.

Approach (2) Discussions :-

- here we configured Struts Action class as Spring Bean class in Spring Configuration file. so that we can inject the model layer Spring Bean class object to Struts Action class to call the Business methods.
- In this process we need to configure one Dummy Action class as proxy class for original Struts Action class in Struts Configuration file to receive the request from Action Servlet and to pass the request to original Struts Action class that is configured in Spring Configuration file.
- Spring API supplies org.springframework.web.struts.DelegatingActionProxy class for this purpose

→ In struts configuration file

```
<action path="/xyz" type="org.springframework.struts.DelegatingActionProxy">
```

↳ Proxy class for original Action class

```
</action>
```

In Spring Configuration File

```
<beans>
```

```
  <bean name="/xyz" class="TestAction">
```

↳ must match with above xyz (actionpath)

↳ original Action class

```
    <property name="obj" ref="d1"/>
```

↳ logic to inject Spring Bean class object to a property of Struts Action class

```
  </bean>
```

```
  <bean id="d1" class="ModelImpl"/>
```

↳ Spring Bean

```
</beans>
```

Sample Struts Action class of the above scenario :-

```
public class TestAction extends Action
{
    ModelInter obj; // Bean property to hold Spring Bean class obj
    public void setObj(ModelInter obj)
    {
        this.obj = obj;
    }
    public ActionForward execute(...) throws Exception
    {
        String result = obj.getWish(); // calling b-method on Spring Bean class object
        ...
    }
}
```

→ Approach ② gives better performance in Struts Integration because it injects Spring Bean

class object to Struts Action class property either during server startup or during the deployment of the webapp.

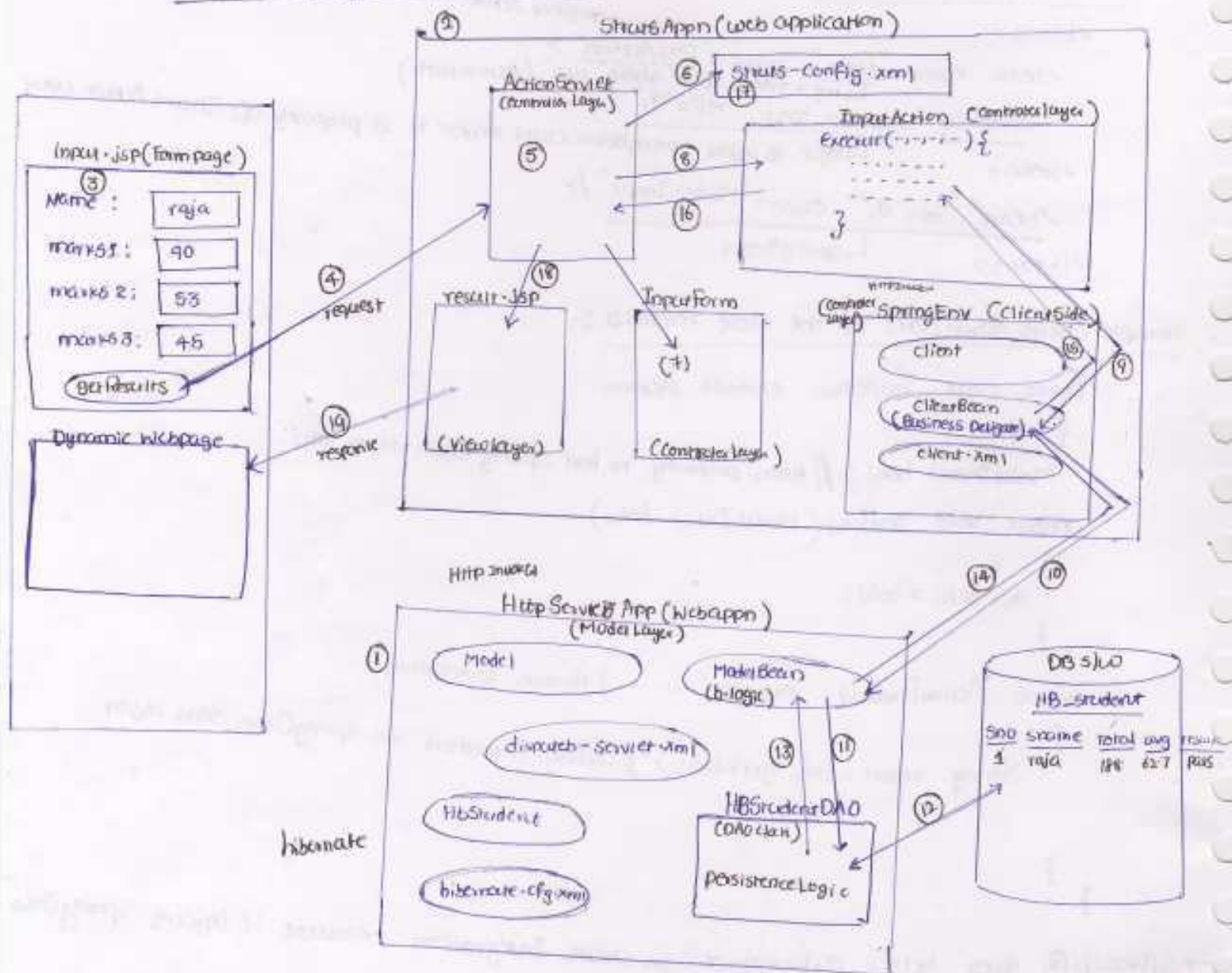
for approach no ② based Struts with Spring app development refer app no: ②②

Page no: 73 to 75

3) Miniproject showing struts, spring, hibernate Integration and also implementing mvc2, DAO, Business Delegate Design patterns.

→ The helper java class of client appn that separates Business method calls ~~from~~ ^{to} make them ^{as} Flexible to modify is called Business Delegate class.

Struts to spring with Hibernate appn



In the above diagram server appn is ^{back} HttpInvoker Server appn. which uses spring JEE, spring ORM modules to develop the business, persistence Logics.

→ Spring Environment placed in Struts appn is HttpInvoker client appn to ~~int~~ ^{help} struts Action class interacting with HTTPInvoker Server appn.

→ In the above diagram two webappns are there (1) Struts Application

(2) The HttpInvokerServer appn.

→ In the above appn Business Logic is calculating total Avg and results based on Students marks and persistence Logic is inserting record.

procedure to develop the above diagram based miniproject by using MyEclipse IDE
for server application:-

Step-I:- Create web project.

File → New → Web project → Project Name: [HttpServerApp] → Finish.

Step-II:- Configure DispatcherServlet in web.xml file having x.do url pattern and the logical name "dispatcher"

refer web.xml of page no: (66)

Step-III:- Create HB_Student table in Oracle DBS/W.

Create table HB_Student (sno number(4) primary key, sname Varchar2(20), total number(6), Avg number(8.7), result Varchar2(5));

Step-IV:- Create Database profile for oracle in MyEclipse IDE

Window → ^{open}perspective → MyEclipse Database Explorer → DB Browser →
Rightclick → New →
Driver Template: [Oracle (Thin Driver)]
Driver Name: [orap]
Connection URL: [jdbc:oracle:thin:@localhost:1521:orcl]
Username: [scott]
Password: [tiger]
addJars [ojdbc14.jar]
→ next → finish

Step-V:- Add Hibernate Capabilities to the project

Rightclick on project → myEclipse → Add Hibernate Capabilities → @Hibernate 3.2
☒ Enable Hibernate Annotation support → next → next →
DB driver: [orap] → next → ☐ create SessionFactory class? → Finish
Password: [tiger]

Step-VI:- Perform Hibernate Reverse Engineering on HB_Student table

Window menu → open perspective → myEclipse DB Explorer → DB Browser →
→ Right click on orap → open connection → password: [tiger] → ok
→ Expand orap → Expand Connected to orap → Expand scott → Expand table
Rightclick on Hb_student → Hibernate Reverse Engineering →

java src folder /HttpServerApp/src

☒ Create POJO $\leftrightarrow$ DB table

☒ Java Data Access Object (DAO)

① Add Hibernate Incomping Annotations:

☒ Generate precise find By methods.

☒ update Hibernate configuration

DAO type : ① SpringDAO

Next $\rightarrow$ ② Hibernate types $\rightarrow$ IDGenerator : Increment

Hb_student $\rightarrow$ IDGenerator : Increment $\rightarrow$ Finish.

Step-VI:- Add spring Capabilities to the project

Rightclick on project $\rightarrow$ myEclipse $\rightarrow$ Add Spring Capabilities $\rightarrow$ ☒ spring 2.5

$\rightarrow$ Select core, persistence core, JDBC, J2EE, Remoting, Misc Libraries $\rightarrow$ next

$\rightarrow$ ☐ Enable App Builder $\rightarrow$ File: dispatcher-servlet.xml $\rightarrow$ next $\rightarrow$ ☒ Add Hibernate support

SessionFactory Bean id : sesfact $\rightarrow$ finish.

Step-VII:- This step generates Hbstudent.java $\rightarrow$ HbstudentDAO.java files
 $\downarrow$ POJO class $\downarrow$ DAO class

Step-VIII:- Add springInterface, springBean^{class} to the project.

src $\rightarrow$ new $\rightarrow$ Interface : model

model.java

```
public interface Model {  
    public String generateResult (String name, int m1, int m2, int m3)  
}
```

~~Feb 17/2020~~ src $\rightarrow$ new $\rightarrow$ class : ModelBean implements model

```
public class ModelBean implements Model {  
    HbstudentDAO dao;  
    public void setDao (HbStudentDAO dao) {  
        this.dao = dao;  
    }  
    public String generateResult (String name, int m1, int m2, int m3) {  
        //Write b:logic  
        int total = m1+m2+m3;  
        float avg = (float) total/3.0f;  
        //generate result  
        String String res = "";
```

```
if (avg < 35)
    res = "fail";
```

```
else
    res = "pass";
```

```
// insert record by using DAO class
```

```
HbStudent st = new HbStudent();
```

```
st.setName(name);
```

```
st.setTotal(total);
```

```
st.setAvg((double) avg);
```

```
st.setResult(res);
```

```
dao.save(st);
```

```
return res;
```

```
} // method
```

```
} // class
```

Configure the following additional beans in Spring Configuration file.

in dispatcher-servlet.xml

```
<bean id = "
```

```
--
```

```
</bean>
```

```
<bean id = "HbStudentDAO"
```

```
--
```

```
</bean>
```

```
<bean id = "mb" class = "MockBean">
```

```
<property name = "dao" ref = "HbStudentDAO"/>
```

```
</bean>
```

```
<bean name = "/service" class = "org.springframework.remoting.httpinvoker.HttpInvokerServiceExporter"
```

```
<property name = "serviceInterface" value = "model"/>
```

```
<property name = "service" ref = "mb"/>
```

```
</bean>
```

```
<bean id = "suri" class = "org.springframework.web.servlet.handler.SimpleUrlHandlerMapping"
```

```
<property name = "mappings">
```

```
<props>
```

```
<prop key = "my.do" > /service </prop>
```

```
</props>
```

```
</property>
```

```
</bean>
```

```
</beans>
```

SimpleUrlHandlerMapping

Configure Tomcat Server with myEclipse IDE

Step-I:- Run the project

Right click on project → Run as → myEclipse Service Application → Tomcat 6.0
→ OK

Changes:-

change 1:- Copy dispatcher-servlet.xml file to WEB-INF folder of Webroot

Change 2:- Add Spring-home \ lib \ asm \ asm-2.2.3.jar to the Build Path of the project

Change 3:- If ClassVisitor problem is raised

a) remove Hibernate, Spring Libraries from the Build path of the project ^{Generated by IDE}

b) add spring, Hibernate related jar files manually into the libraries of the project

Spring jars

Spring.jar

Commons-logging.jar

Spring-Webmvc.jar

Hibernate jar files

The Regular 8 jar files

hibernate-3.jar	antlr-2.7.6.jar
(JTA-1.1.jar	slf4j-api-1.6.1.jar
(javassist-3.10.0-GA.jar	dom4j-1.6.1.jar
(hibernate-jpa-2.0-api-1.0.0.Final.jar	
(Commons-Collections-3.1.jar	

Struts appn

Step-I:- Create web project having name StrutsApp

File menu → new → web project → projectName: StrutsApp → Finish

Step-II:- add Spring capabilities to the project.

Right click on project → myEclipse → Add spring Capabilities → @spring2.5
→ Select Spring 2.5 Core Libraries, JEE Libraries, Remoting Libraries, Misc Libraries
→ next → ☐ Enable Aop Builder → File: client.xml → Folder: Webroot/WEB-INF

→ Finish

NOTE:- Copy Model.java from Server appn (HttpServerApp) to current appn (StrutsApp)

Step-III:- Develop SpringInterface, SpringBean having Logic to call the Business methods of Server appn. → client → clientBean

~~File menu~~ Src → New → Interface → Name: Client → Finish

public interface ~~Model~~ ^{client}

~~public~~ ~~String~~ ~~getBusinessMethod~~
{
 public String callMethod(String name, int m1, int m2, int m3);
}

```
src → new → class → name: ClientBean → Finish
```

```
public class ClientBean implements Client {
```

```
Model bobj;
```

```
public void setBobj (Model bobj) {
```

$$\text{this} \cdot \text{bob}_j = \text{bob}_j;$$

3

public Model $\text{getObjRef}() \{$

रत्न-बोधि;

3. Learn Bob's reference

```
public String callMethod(String name, int m, int m2,
                        int m3)
```

```

{ // call b.method of Server App

```

```
String result = bob.generateResult(name, m, m2, m3);
```

```

2 return result;

```

to STUDENT ACTION CLASS

in client.xml

```
<bean id="pfb" class="org.springframework.remoting.httpinvoker.HttpInvokerProxyFactoryBean">
```

```
<property name="serviceInterface" value="Model" />
```

`<property name="serviceUri" value="http://localhost:2020/prime"`

</begin>

```
<bean id="cb" class="ClientBean">
```

```
<property name="bob" ref="pfb"/>
```

△bean?

```
<bean name="/demo" class="InputAction">
```

`<property name="mo" ref="cb"/>`

4/bean>

beans,

NOTE:-

Step-IV:- add smurs Capabilities to the project

Step-IV:- add Struts Capabilities

Right click on project → myEclipse → Add Struts Capabilities → Struts 1.3

→ Base packages for new classes → Finish.

[illegible]

R.C. on Project → new → Others → myEclipse → WebStruts → Struts 1.3
 FormBean logic case name

→ Shows 1-3 Form, Action & jsp → next → Name if1 →

Super Class : `org.apache.struts.action.ActionForm` → Form Type : `InputForm` →

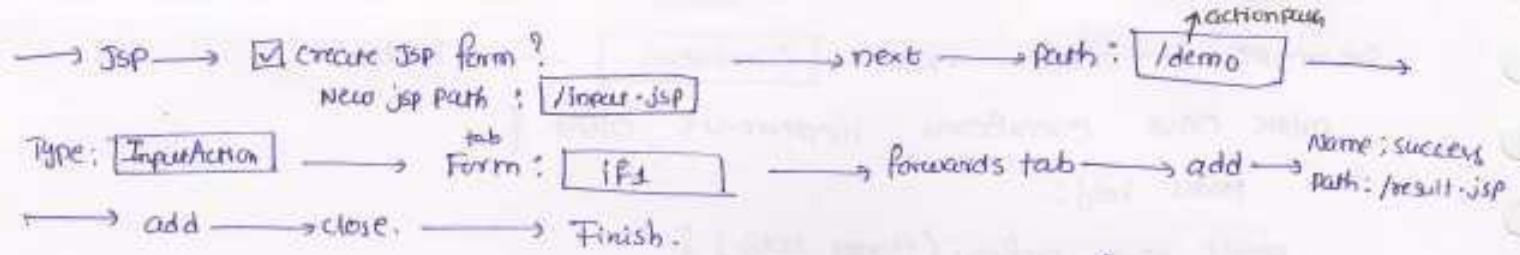
Term Properties

name
m1
m2
m3

add

methods

descend all the check boxes



Step-VI:- Do following modifications in struts configuration file.

- ① remove existing Action class configuration and configure Delegating Action Proxy
(Under <action-mappings> tag)

<action-mappings>

<action path="/demo" type="org.springframework.struts.DelegatingActionProxy" name="if1">

<forward name="success" path="/result.jsp"/>

</action-mappings>

- ② Configure ^{the} ContextLoaderPlugin as shown below.

(under <action-mappings> tag)

<message-resources parameter="ApplicationResources"/>

<plug-in className="org.springframework.struts.ContextLoaderPlugin">

<set-property property="contextConfigLocation" value="/WEB-INF/context.xml"/>

</plug-in>

Step-VII:- Write following code in Action class

//InputAction.java

public class InputAction extends Action {

client mo;

public void setMo(Client mo) {

this.mo = mo;

}

public ActionForward execute(-,-,-)

{

//read form data
InputForm if1 = (InputForm) form;

String name = if1.getName();

int m1 = Integer.parseInt(if1.getM1());

int m2 = Integer.parseInt

int m3 = Integer.parseInt

//call B. method of client Bean

String result = mo.callBmethod(name, m1, m2, m3);

```
// keep results in request attribute
request.setAttribute("msg", result);
return mapping.findForward("success");
} // method
} // class
```

Step-VIII:- Add Result.jsp to Webroot folder of the project.

R.C. on Webroot folder → new → JSP → filename: result.jsp → Finish.

Result is = `<%= request.getAttribute("msg") %>`

Step-IX:- Deploy this project in tomcat server and also make sure that the server app is also in running mode.

R.C. on project (struts App) → Run as → my Eclipse server app → Tomcat 6.

→ OK

Step X:- Test the application.

Open browser window → type this URL

`http://localhost:2020/strutsApp/Input.jsp`

Q:- What is the difference b/w programming Language, Technology and Frameworks?

→ Languages are given to programmers having basic and direct facilities to develop applications.

These Languages are useful while creating Technologies, Frameworks.

Eg:- C, C++, Java.

→ Software Technology is a S/W specification having set of rules and guidelines to develop Softwares. while developing this S/W ^{the} programming language will be used.

Eg:- JDBC, Servlet, JSP, EJB and etc..

→ Frameworks are special S/W technologies which provide abstraction layer on multiple ^{other} core technologies

Eg:- Struts, JSP, Spring & etc...


```

69 <property name="password" value="tiger"/>
70 <property name="url" value="jdbc:oracle:thin:@localhost:1521:orcl"/>
71 <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
72 </bean>
73 <context:component-scan base-package="p1" /> to recognize the Spring Beans of 'p1' package
74 </beans>
75 -----RegisterAction.java (Struts2 Action class configured as the Spring Bean)
76 package p1;
77 import javax.annotation.Resource;
78 import org.springframework.stereotype.Component;
79 @Component("regAction")
80 public class RegisterAction { here Struts2 Action class is configured as Spring Bean
81                               in Spring Configuration file.
82 @Resource(name="dbs") here Spring Bean class obj is injected to a property of
83 private DBStore dbStore; Struts Action class.
84
85 public void setDbStore(DBStore dbs) {
86 { setter method supporting setter injection
87 dbStore=dbs;
88 }
89 private String username,password; properties to hold form data.
90
91 //getters and setters
92 public String getUsername() {
93     return username;
94 }
95
96 public void setUsername(String username) {
97     this.username = username;
98 }
99
100 public String getPassword() {
101     return password;
102 }
103
104 public void setPassword(String password) {
105     this.password = password;
106 }
107 // all struts logic here
108 public String execute() {
109     System.out.println(" Execute Method called");
110     dbStore.storeUser(username,password); calling B-method of Spring Bean class
111     return "SUCCESS";
112 }
113 }
114 -----DBStore.java (Spring Interface) Spring
115 package p1;
116 public interface DBStore{
117     public void storeUser(String uname,String password); Declaration of B-method
118 }
119 -----DBStoreImpl.java (Spring Bean) Spring
120 package p1;
121 import javax.sql.*;
122 import java.sql.*;
123 import org.springframework.beans.factory.annotation.Autowired;
124 import org.springframework.stereotype.Component;
125
126 @Component("dbs") current bean id
127 public class DBStoreImpl implements DBStore{
128     Connection con; Spring Interface
129     @Autowired
130     DataSource ds; Injects DataSource object to 'ds' property through autowiring
131
132     public void storeUser(String uname,String password)
133     { B-method
134     try
135     {
136         con=ds.getConnection();

```

Injects JDBC Connection object from the Connection pool

Page 2 of 3

we are configure

In this application Struts2 Action class as Spring Bean class. The real Spring Bean class object is injected to Struts2 Action class. so that B-method of Spring Bean can be called from Struts2 Action class.

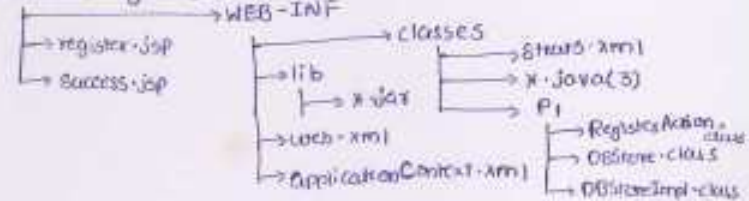
```

137 PreparedStatement ps=con.prepareStatement("insert into userdetails values(?,?)");
138 ps.setString(1, uname);
139 ps.setString(2,password);
140 int count=ps.executeUpdate();
141
142 if(count!=0)
143     System.out.println("Successful");
144 else
145     System.out.println("Not Successful");
146 con.close();
147 }
148 catch(Exception e)
149 {
150     e.printStackTrace();
151     //con.close();
152 }
153 }
154 }
155 */
156 SQL> create table userdetails
157 (
158     username varchar2(10),
159     password varchar2(10)
160 );
161 */

```

Deployment Directory Structure

Struts2Spring Anno



jar files in classpath :

Struts2-core-2.3.4.1.jar, org.springframework-3.1.1.RELEASE.jar,
~~spring-jdbc-3.1.1.RELEASE.jar~~ org.springframework-3.1.1.RELEASE.jar
~~commons-logging-1.2.2.jar~~

jar files in WEB-INF\lib folder

asm-3.3
asm-commons-3.3
asm-tree-3.3
commons-fileupload-1.2.2
commons-io-2.0.1
commons-lang3-3.1
commons-logging → from spring 2.5
freemarker-2.3.19
javassist-3.11.0.GA
ogni-3.0.5
ojdbc14 → from oracle
Struts2-core-2.3.4.1
*** Struts-spring-plugin-2.3.8
Xwork-core-2.3.4.1 } from Struts 2.3 slw
org.springframework-3.1.1.RELEASE
" " beans-3.1.1 " "
" " context-3.1.1 " "
" " core-3.1.1 " "
" " expression-3.1.1 " "
" " jdbc-3.1.1 " "
" " web-3.1.1 " "
" " web-servlet-3.1.1 " "
" " web-struts-3.1.1 " " } from Spring 3.1



In Struts 2.x

- NO form beans
- Action classes are POJO classes and they also hold form data.
- Struts configuration file name is `struts.xml`
- Controller is `ServletFilter`
 - Struts 2.0.x → `FilterDispatcher`
 - Struts 2.1.x → `StrutsPrepareAndExecuteFilter`
- main jar files representing Struts 2.x api are : `struts2-core-<version>.jar`
`struts2-xml-<version>.jar`
- Struts 2 gives built-in JSP tag libraries whose taglib uri is : `/struts-tags`
- Servlet Listeners are given to perform Event Handling on request, ^{session} response, `ServletContext` obj
- Spring 2.5 gives `org.springframework.web.context.ContextLoaderListener` as `ServletContextListener` which will be activated when `ServletContext` object is created/destroy. This Listener activates the `WebAppContext` container by taking ~~app~~ ^{applicationContext.xml} ~~application.xml~~ as the Spring configuration file. In this process all the Spring beans will be instantiated.
- ⑧ For example application on Struts 2.x and Spring 3.5 Integration refer the supplementary handout given on 10/04/2013
- Flow of execution related to Struts 2.x ~~and~~ ^{with} Spring 3 app handout of 10/04/2013
- a) Deployment
- b) Instantiation of `ServletFilter` either during server startup or during deployment of (25-32) `webapp`.
- c) → `ContextLoaderListener` activation (37-39)
- `WebAppContext` activation by taking `applicationContext.xml` as the Spring configuration file
- pre-Instantiation of ^{beans belonging to} Spring
- Spring Bean class (`DBObjectImpl` obj) will be injection to `StrutsAction` ^{mass} (Register Action)
- d) enduser gives request to Form page (`register.jsp`) this form page submits the request (ref:10)
- e) `ServletFilter` traps and takes the request (36-37)
- f) `ServletFilter` passes the request to Struts Action class configuration (52-54)
- g) `ServletFilter` calls `execute()` method of `StrutsAction` class. (108-112)

h) this `execute()` calls `storeDetails()` of `SpringBean` class (132-153). This method gives the results back to `execute()` method of `Action` class

i) `execute()` returns "SUCCESS" string to `ServletFilter` and `ServletFilter` uses this string to get the result page (ref 53)

j) `ServletFilter` passes the control to `Success.jsp` page.

Module - VI Spring AOP

- Spring AOP is not really related with AOP. AOP is the methodology to create programming Languages. Spring AOP is the methodology to apply middleware Services on Spring applications.
- Middleware Services are the additional services that are applied on the applications to make our apps more perfect or accurate in all situations.

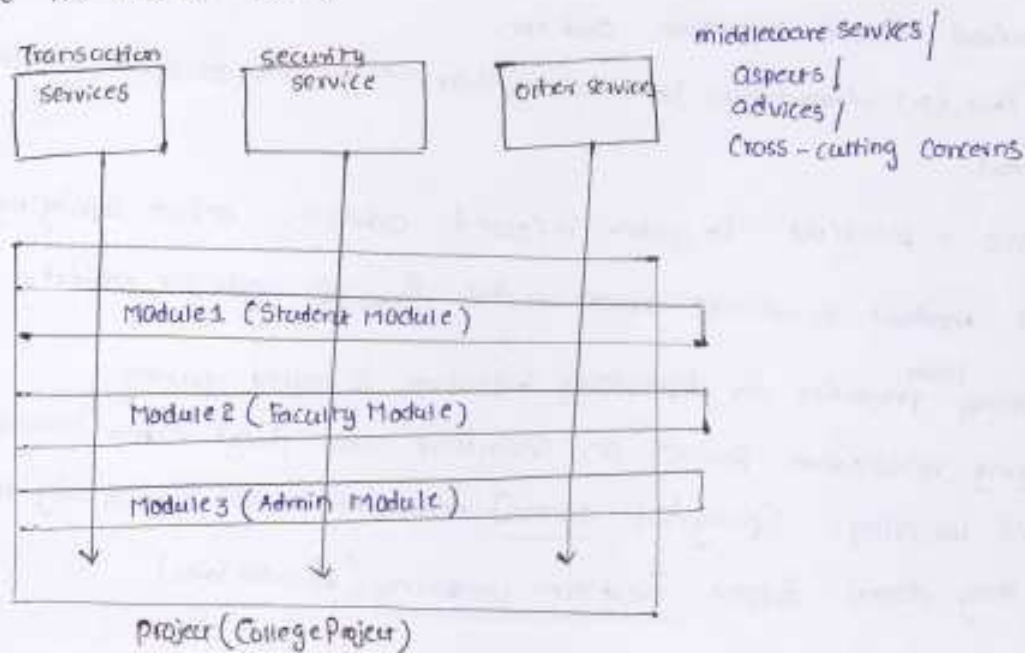
Eg:-

- Transaction management (applies do everything or nothing principle on the given code)
- Security Service (protects the application from unauthorized users)
- Logging Service (keeps track of Application execution)
- Jdbc Connection pooling (keeps set of readily available JDBC Connection objects).

- If we write code of middleware services directly in the application, middleware services code becomes specific to one application (no reusability) moreover the code of Middleware Services may dominate application code.

- Spring AOP allows us to write Application code and middleware services separately and allows to link middleware services code with application through xml files or Annotations. Due to this the code of middleware services becomes reusable.

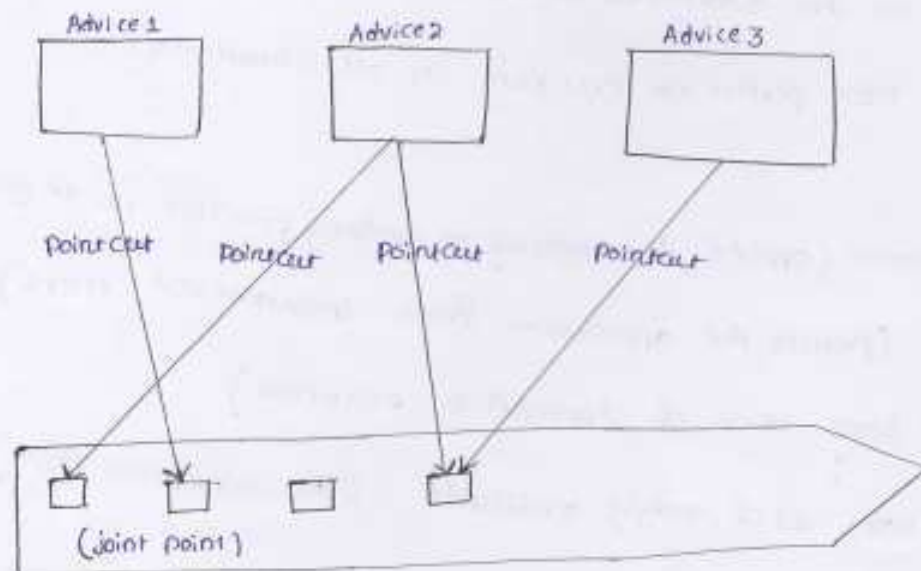
- To understand the need of Spring AOP and understanding that refer page no's 100 to 102.



Spring AOP style Middleware Services enabling

Spring AOP Terminologies

aspect	Wiring
advice	Weaving
join point	target bean / target obj
pointcut	proxy bean / proxy obj
advisor	



□ join point

aspect:- It is the specification containing rules and guidelines. To implement the middleware services practically.

advice:- Real middleware service which is the implementation of aspect

join point:- Possible positions in our Business method where advices can be applied

EG①:- beginning of B-method, end of B-method, B-method returns a value when B-method throws Exception and etc.

pointcut:- The xml (or) Annotation based entry that links advices with join points of B-method.

advisor:- advice + pointcut to gather is called advisor. Before applying advice on Business method it will be taken in the form of advisor object.

wiring:- Configuring ^{bean} properties for dependency Injection is called wiring.

weaving:- applying middleware services on Spring Bean class through Aspect Oriented Programming is called weaving. Spring AOP, aspectJ enhancement of support only method level weaving.

That means they do not support Field level weaving (Variable level).

target bean / target object :-

Spring Bean class object on which ~~we~~ we are planning to applying middleware service is called target bean / target object.

proxy bean / proxy object :-

Spring Bean class object on which middleware services are already applied is called proxy object.

→ Business methods called on target object just execute the Business method.

→ Business methods called on proxy object execute the Business method logic along with middleware services.

org.springframework.framework.ProxyFactoryBean generates proxy object based on given target object.

There are 4 types of advices

- ① Before Advice (Executes at the beginning of the b-method execution)
- ② After Advice (Executes at the end of b-method execution)
- ③ Throws Advice (Executes when exception is raised in the b-method)
- ④ Around Advice (Executes at beginning of b-method and at the end of B-method)

→ using Spring AOP, AspectJ not only we can ~~develop~~ use Spring (or) server supplied Middleware Services we can also use them to develop Userdefined Middleware Services and advices.

for the above 4 types of advices refer page no's 104 to 106.

(X) procedure to develop Spring AOP based Spring application.

Step 1:- Develop Java class as the advice class

Step 2:- develop SpringInterface, SpringBean class having Business method declarations, definitions.

Step 3:- Develop Spring Configuration file

a) cfg Spring Bean class

b) cfg Advice class

c) cfg Advisor class pointing to Advice class

d) cfg ProxyFactoryBean specifying advisor, SpringBean

NOTE:- This ProxyFactoryBean gives SpringBean class object as proxy object.

Step 4:- client Application

→ Activate SpringContainer (BeanFactory or ApplicationContext)

→ get proxy object from Spring Container

→ call B-methods on proxy object

NOTE:- Business method called on proxy object will be executed along with middleware Services (Advices).

→ Based on the advice we develop the joint point ^{position in B-method} ~~Advice~~ will be decided automatically.

→ Advisor is given to Map certain Advices with certain Business methods of SpringBean class. There are two types of Advices.

① NameMatchMethodPointCutAdvisor

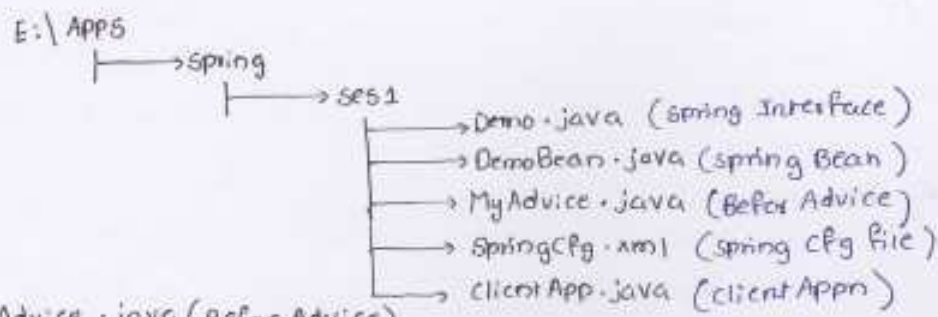
→ here the advices will be linked with b-methods based on the names of the B-methods.

② RegexMethodPointCutAdvisor

here the advices will be linked with b-methods based on regular expression

for related Information on Advisors refer Page no's 108 & 109

First Spring AOP application :-



// MyAdvice.java (Before Advice)

```
import org.springframework.*;
```

```
import java.lang.reflect.*; // Reflection API
```

```
import java.util.*;
```

```
public class MyAdvice implements MethodBeforeAdvice
```

```
{
```

```
    public void before(Method method, Object[] args, Object target)
```

```
    {
```

```
        // gathering details about b-method
```

```
        S.o.p("MyAdvice: B-method name is " + method.getName());
```

```
        S.o.p("MyAdvice: Name of Bean class is " + target.getClass());
```

```
        S.o.p(method.getName() + " execution is started at " + new Date());
```

```
        // changing B-method arg value if it is invalid
```

```
        String s1 = (String) args[0]; // gathers b-method's 1st argument value
```

```
        if (s1 == null || s1.length() <= 3) // set new value to b-method
```

```
            args[0] = "sathya"; // if it is invalid
```

```
    } // method
```

```
} // class
```

```
// Demo.java (Spring Interface)
```

```
public interface Demo
```

```
{
```

```
    public void sayHello(String name); // Decl of B-method
```

```
}
```

```
// DemoBean.java (Spring Bean class)
```

```
public class DemoBean implements Demo
```

```
{
```

```
    // implementation of B-method
```

```
    public void sayHello(String name)
```

```
    {
```

S.o.p ("DemoBean: execution of sayHello() → Name: "+name);

S.o.p("In In In");

}

}

<!-- spring.cfg.xml -->

<!DOCTYPE ---->

<beans>

<bean id="db" class="DemoBean"/> <!-- Bean class cfg -->

<bean id="adv" class="MyAdvice"/> <!-- Advice cfg -->

<!-- Adviser cfg -->

<bean id="advr" class="org.sf.aop.support.NameMatchMethodPointcutAdvisor">

<property name="advise" ref="adv"/>

<property name="mappedName" value="sayHello"/>
↳ B. method name

</bean>

<!-- this cfg gives proxy obj ~~base~~^{based} on target object by applying advice -->

<bean id="pfb" class="org.sf.aop.framework.ProxyFactoryBean">

<property name="proxyInterfaces" value="Demo"/>
↳ Spring Interface

<property name="target" ref="db"/>
↳ Bean id

<property name="interceptorNames">

<list>

<value> advr </value>

</list> ↳ Adviser object

</property>

</bean>

</beans>

// ClientApp.java

import org.sf.context.support.*;

public class ClientApp {

public static void main(String[] args)

{

FileSystemXmlApplicationContext ctx = new FileSystemXmlApplicationContext("spring.cfg.xml");

// get proxy object

Demo proxyObj = (Demo) ctx.getBean("Pfb");

// call B-methods ... on proxy obj

proxyObj.sayHello("Raja");

proxyObj.sayHello("aa");

}

}

/> javac *.java

/> java ClientApp

/> jar files in class path: Spring-jar, Commons-logging-jar

→ to use RegularExpressionPointCutAdvisor place the following code in the SpringConfig File

<!-- AdvisorCfg -->

<bean id="advr" class="org.springframework.aop.support.RegexpMethodPointcutAdvisor">

<property name="advice" ref="adv" />

<property name="pattern" value=".*" />

</bean>

Sample code of Around Advice

public class MyAdvice implements MethodInterceptor

{

public Object invoke(MethodInvocation is)

↳ represents the executing environment of the Business method.

{

..... // executes at the beginning of the B-method

is.proceed(); // calls the B-method

..... // executes at the end of the B-method

}

}

⑧ for example application of the Around Advice appn ① to appn ④ of the booklet
pages no 114 to 118

Q:- what are the Spring modules that are used in your project?

Ans:- If Spring is used only in model layer then go for Spring JEE, ORM, AOP modules.

If Spring is used only in view, controller layer then go for Spring web module.

If project is based on Struts, Spring & Hibernate Integration then go for Spring AOP, JEE, ORM, web modules.

NOTE: Core module is the Base module for every module.

13/04/2013

* Keeping the track of flow of execution is called logging operation and messages used for this are called log messages.

* Writing log messages using `System.out.println()` is not industry standard. The reasons are

- ① → `System.out.println()` can write messages only to Console monitor. That means can't be used to write msgs to Database/s/w, files, mail servers and etc...
- ② → `System.out.println()` msgs writing process is Single Threaded operation. So, multiple messages can be written at a time.
- ③ → `System.out.println()` doesnot allow to categorize the log messages.
- ④ → `System.out.println()` doesnot allow to format the log message.

To overcome these problems use Log4j api/tool to place log messages.

- ① → Log4j allows to write log messages to different destinations like file, db, mail server, Console and etc...
- ② → Log4j can write msgs Simultaneously or ~~parallelly~~ ^{parallel}.
- ③ → Log4j allows categorize log msgs
debug < info < warn < error < fatal
- ④ → Log4j allows to filter log messages while retrieving them.
- ⑤ → Log4j allows to format the log messages.

Log4j

- Logging for java
- Version: 1.x
- Vendor: apache foundation
- Open Source tool
- jar files `log4j-<version>.jar`

`<weblogic_home>\Middleware\wlserver_10.3\server\lib\consoleapp\App-INF\lib\log4j-1.2.8.jar`

file represents log4j api.

→ use "debug" level Confirmation stmts / debugging stmts

logger.debug("I am in main(-)");

→ use "Info" level when certain important work is Completed.

Eg: Connection con = DriverManager.getConnection(-, -, -);

logger.info("JDBC Con Obj is created");

→ use "warn" level to display warning messages.

```
if(rs.next())
```

```
{
```

```
....
```

```
}
```

```
else
```

```
{
```

```
logger.warn("ResultSet obj must not be empty");
```

```
}
```

→ use "error" level in the catch blocks of known Exceptions.

use "fatal" level in the catch blocks that handles unknown Exceptions.

```
try
```

```
{
```

```
....
```

```
} catch (SQLException se) // known exception handling
```

```
{
```

```
logger.error("DB problem");
```

```
}
```

```
catch (Exception e) // unknown exception handling.
```

```
{
```

```
logger.fatal("unknown problem");
```

```
}
```

There are 3 important objs in log4j programming

① Logger obj

② Appender obj

③ Layout obj

Logger obj :- (i) enables logging operation on given java class. (ii) allows to create log messages

having different categories. (iii) It is Base obj for Log4j based logging operations.

```
Logger logger = Logger.getLogger(TestApp.class);
```

Static Factory method

obj of java.lang.class pointing to TestApp class.

```
logger.debug("hello1");
```

```
logger.debug("hello2");
```

Appender obj:-

- specifies the destination object for which we can write log messages (like dbappending, fileappending, odbc appending etc..)
- will be linked with logger object to take and write log messages.
- The available appenders are
 - File Appender (to write to ^{files} ~~database~~)
 - ConsoleAppender (to write to Console)
 - JDBCAppender (to write to DB like)
 - IMAppender (to write to mail server and etc..)

Layout obj:-

- allows to specify the format and pattern of the log messages.
- It can be used to customize the look and feel of log messages.
- The available Layout classes are.

HtmlLayout

SimpleLayout

PatternLayout

XmlLayout and etc...

NOTE:- If the loggerLevel retrieve level log messages info we will get only those log messages whose logger level is $\geq$ Info.

- (*) for SpringAop example appn that uses all the 4 types of advices refer appn ⑤ of the page no's (119) to (121)

14/04/2013

Transaction management :-

deleting them

→ The process of combining the related operations into single unit by applying do everything or nothing principle is called Transaction management.

→ All sensitive logics like transfer money debit card processing must be developed by enabling Transaction mgmt Service.

→ Transfer money operation is the combination of two operations.

① Withdraw amount from Source account.

② deposit amount into destination account.

It is recommended to execute these logics by enabling do everything or nothing principle

→ we can bring the effect of Transaction Mgmt using try/catch blocks manually. but it makes code unmanageable. To overcome this problem always prefer using ServerManaged (or) Technology Supplied Transaction mgmt Service.

→ (i) In hibernate apps we use Technology supplied Transaction Service.

(ii) In Spring apps we can use either Technology supplied Transaction Service (or) Server Managed Transaction Service.

(iii) In EJB comps we use Server managed Transaction Service.

The typical code of Transaction management :-

```
public void bm1()
{
    try {
        Begin Transaction
        operation 1.....
        operation 2.....
        operation 3.....
        Commit the Transaction
    } catch (Exception e) {
        rollback Transaction
    }
}
```



Transaction mgmt gives Support for ACID properties implementation

A $\rightarrow$ Atomicity

C $\rightarrow$ Consistency

I $\rightarrow$ Isolation

D $\rightarrow$ Durability

$\rightarrow$ The Process of Combining multiple & related individual operations into single unit is called "Atomicity"

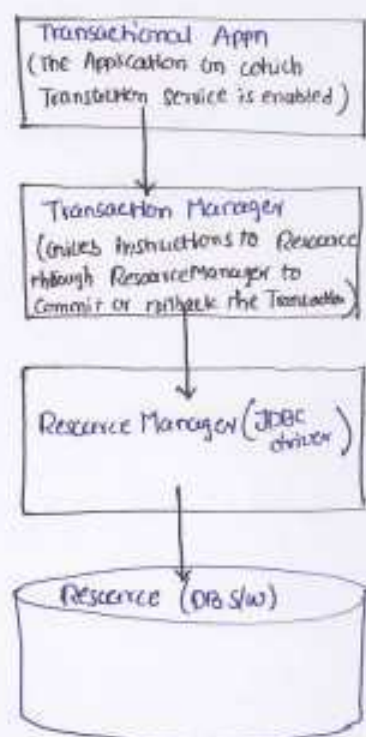
The word "Atomicity" ~~is~~ come from the word "Atom"

$\rightarrow$ Consistency:- Even though rules kept on Database data are Violated during the course of Transaction in there is a ^{like balance must not be negative} guaranty for no rule Violation at the end of Transaction mgmt code. then that code is called as Consistent Code.

$\rightarrow$ Isolation:- The process of applying ^{lots of} ^{Database} amount of data and allowing only ^{one} appn data to manipulate the data is called enabling Isolation. This Isolation prevents Concurrent and Simultaneous operations on Database data

$\rightarrow$ Durability:- ^(long life) The ability of bringing Database s/w back to normal state through logfiles and backup files when database is Corrupted. is called making DB as the Durable database.

The Architecture of Transaction Management



Transaction Manager commits or rollbacks application data in a Database s/w through Resource Manager ~~is~~ based on the Commit (or) rollback operations that comes from Transactional appn.

→ based on the no of resources (Database slvs) that are involved to perform various operations of Transactional code we can say there are 2 types of Transactional managements.

① Local Transaction (Tx) Management.

→ all the operations of Transactional code will act on Single DB slv
Eg:- transfer Money operation b/w two diff accounts of same bank.

② Distributed Transaction (Tx) Management.

→ The operations of Transactional code will act on Multiple DB slvs
Eg:- Transfer Money operation b/w two accounts of two diff banks.

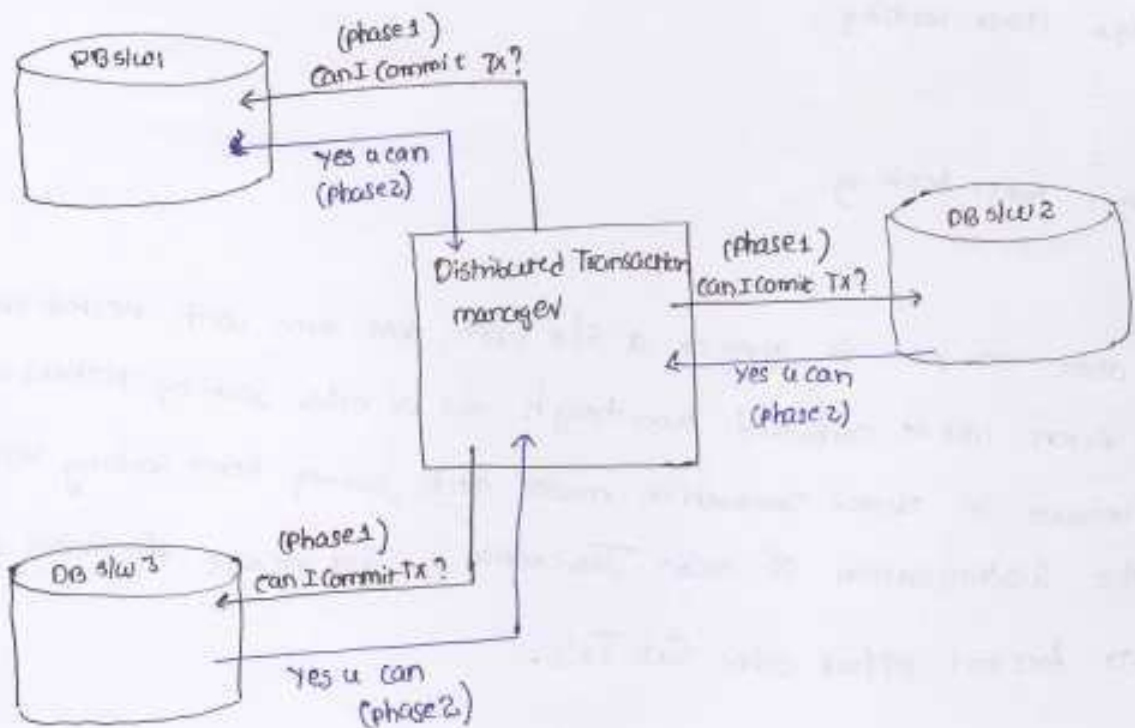
→ Hibernate, Spring, EJB Technologies support Local Transaction mgmt.
Hibernate does not support Distributed Transactions. whereas Spring, EJB supports Distributed Transactions.

15/04/2013

→ Distributed Tx's run on RPC (2 phase Commit) protocol.

In phase 1, The Distributed Tx manager will seek permission from DB slvs to Commit the Tx.

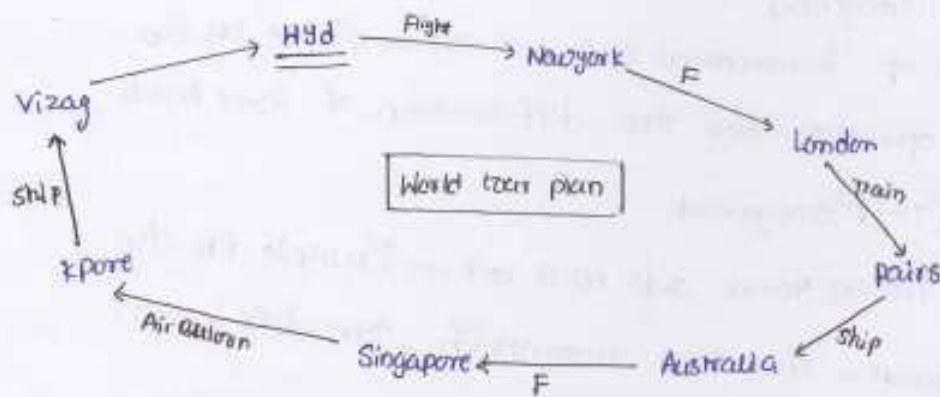
In phase 2, if all DB slvs reply yes then the Distributed Tx will be committed. Otherwise the Distributed Tx will be ~~no~~ rollback.



Transaction models

Flat Transaction Model

Nested Transaction Model



The transaction will be rolled back if one or other journey tickets are not available because, in Flat Transaction model all journey ticket booking operations will be placed as direct operations of main transaction.

Flat Transaction Model

Main Tx {

Journey 1 ticket booking

Journey 2 ticket booking

Journey 3 ticket booking

Journey 4 ticket booking

⋮

Journey n ticket booking

}

When the above tour plan is given to a GLC app that runs with nested transaction the journey tickets will be confirmed even though one or other journey tickets are not available because in Nested Transaction model each journey ticket booking will be taken as the subtransaction of main Transaction. That means the success or failure of one subtx does not effect other subtx's.

Nested Transaction model :-

main Tx {

Sub Tx 1

{
 Journey 1 Ticket booking
}

Sub Tx 2

{
 Journey 2 Ticket booking
}

...

Sub Tx n

{
 Journey n Ticket booking
}

}

- EJB, hibernate, spring supports Flat Tx model
- hibernate, EJB do not support Nested Tx Model
- Spring supports nested Transaction.

→ We can apply Transaction mgmt Service on Spring Applications in two ways

1) programmatic Approach (write code directly in Spring Bean class b-methods)

(no need of working with Spring Aop module)

2) Declarative Approach (cfg Transaction service by using Xml or annotations)

(we can use TransactionTemplate class to simplify this Tx mgmt).

(we need to work Spring Aop module or AspectJ concepts)

→ Declarative Approach is always recommended to use... it keeps our spring bean class b-methods clean and neat.

→ To enable Transaction mgmt on diff kinds of persistence logics we need to work with diff kinds of Transaction managers.

① DataSourceTransactionManager

→ To enable Tx mgmt on jdbc persistence logic.

② HibernateTransactionManager

→ To enable Tx mgmt on plain hibernate or Spring ORM hibernate persistence logic.

③ ToplinkTransactionManager

→ To enable Tx mgmt on Toplink persistence logic.

④ JDOTransactionManager

→ To enable Tx mgmt on JDO persistence logic

⑤ JtaTransactionManager

→ To enable Tx mgmt on Spring appn by using `ServiceManager Tx service`.

and etc....

* To create TransactionTemplate class obj TransactionManager obj is the base object

* For Spring DAO module based local, Flat Transaction management in programmatic approach refer appn ② of the page no's 131 & 132

TransactionTemplate class provides the abstraction layer on Programmatic Transaction management by automatically beginning the Transaction. committing (or) rollbacking the Transaction.

→ Avoid programmatic Transaction mgmt in Spring appn because it makes the programmer to write more code of Transaction mgmt along with the Business Logic.

→ To overcome this problem use Spring AOP / AspectJ base Declarative Tx mgmt.

→ Passing the Transaction started in one resource to another resource is called propagation of Transaction. In programmatic Tx mgmt ~~the client appns must also~~ the Business method always execute with new Transaction because it can't work with client supplied Transaction. This indicates Programmatic Tx mgmt doesn't support propagation of Transaction.

→ Declarative Tx mgmt supports Transaction propagation. That means B. methods can run

① In client started / ^{supplied} Tx service

② In new Tx that is started in b. method itself

③ with no Tx.

→ In Declarative Tx mgmt Business method contains only Business Logic and the logic of TransactionService will be configured through Xmi files ^(or) annotations.

16/04/2013

→ Whether the b. method on which Declarative Tx mgmt enabled runs with no Transaction (or) new Transaction (or) client supplied Transaction will be decided base on the Transaction attribute that is configured on the business method.

→ The 6 Tx attributes are

1) required

2) required new

3) supports (default)

4) Not supported

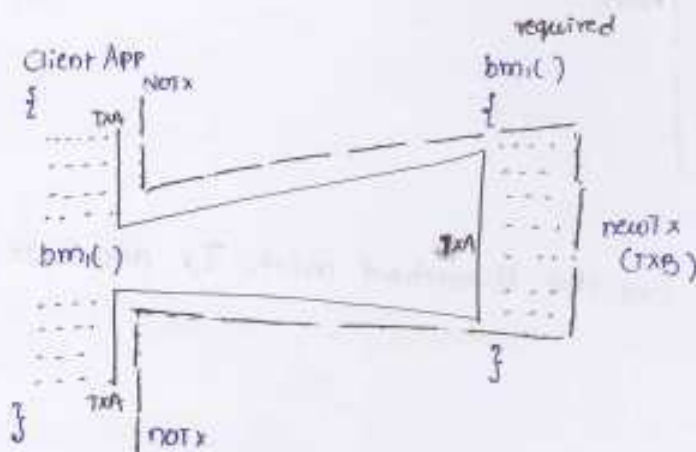
5) mandatory

6) never

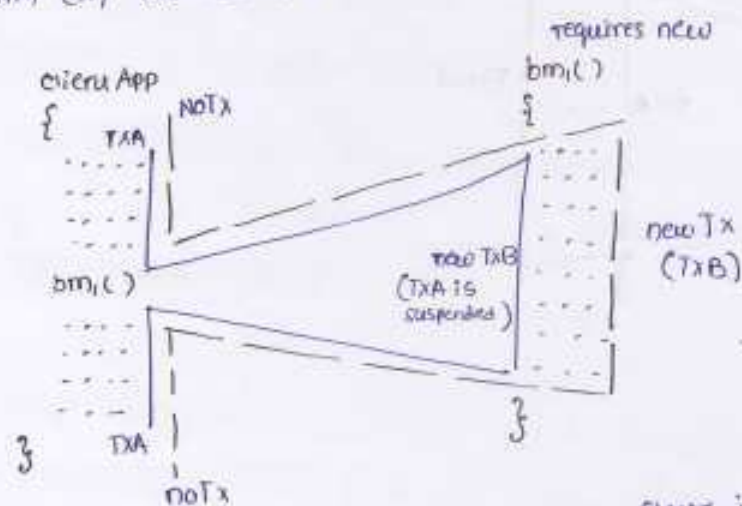
note:- Tx attributes are not required while dealing with programmatic Tx mgmt.

note:- TransactionTemplate class not required while dealing with Declarative Tx mgmt.

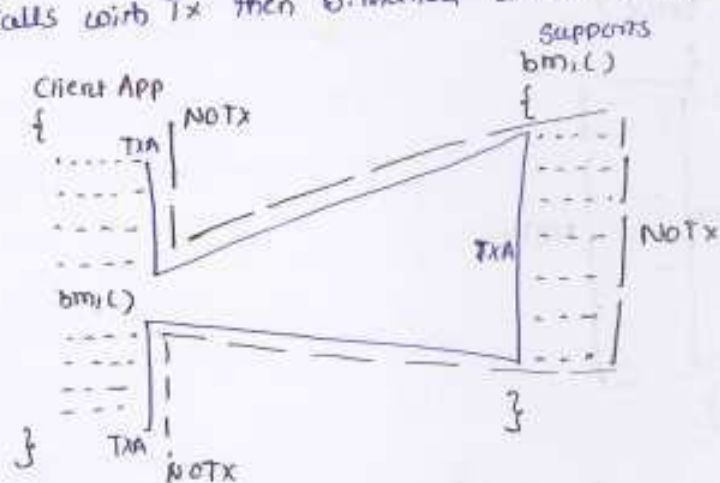
required :- B-method uses client supplied Tx if client appn calls the B-method with Tx. otherwise new Tx will be started in the B-method.



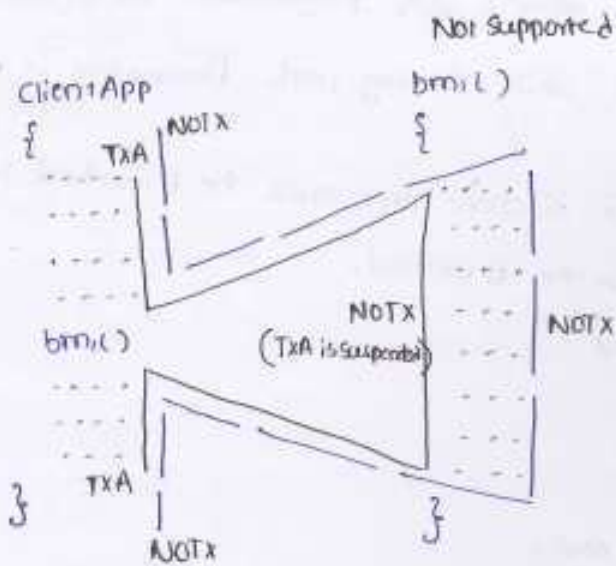
requires new:- This B-method runs with new Tx irrespective of whether client calls B-method with (or) without Tx.



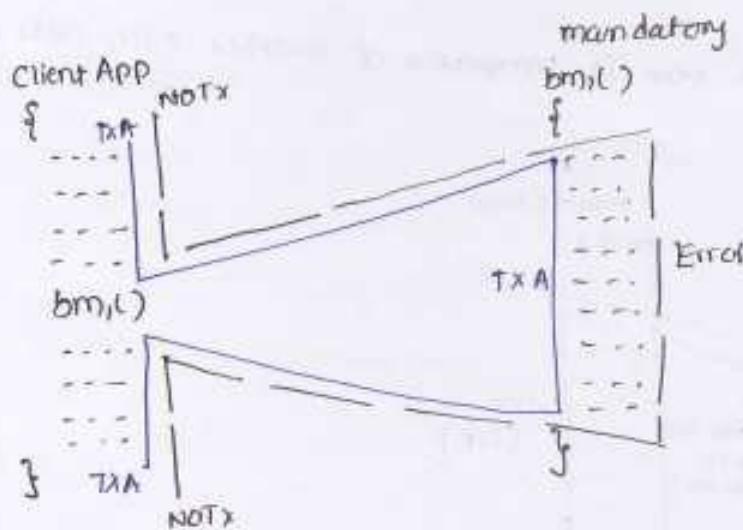
Supports:- The state of Tx in client appn also reflects in B-method. that means if client appn calls with Tx then B-method also runs in the Tx. otherwise B-method runs without Tx.



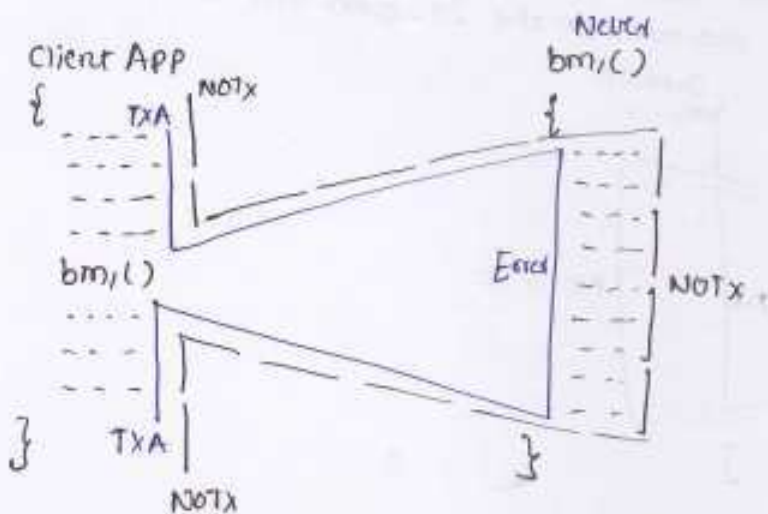
Not supported:- B-method always runs without Tx irrespective of whether client appn is calling B-method with or without Tx.



⑤ Mandatory:- The client app must call the B-method with Tx otherwise B-method throws an Exception.



⑥ Never:-



most popularly used Tx Attribute is "required".
The Default Tx attribute is "support"

The Effects of Tx attributes :-

Transaction Attribute	Client's Transaction	Bean's Transaction
Required	none	T ₂
	T ₁	T ₁
RequiredNew	none	T ₂
	T ₁	T ₂
Supports	none	none
	T ₁	T ₁
Mandatory	none	error
	T ₁	T ₁
NotSupported	none	none
	T ₁	none
Never	none	none
	T ₁	error

- org.springframework.transaction.interceptor.TransactionProxyFactoryBean returns proxy object on which Tx service is enabled.
- org.springframework.transaction.interceptor.NameMatchTransactionAttributeSource is like pointcut advisor to apply Transaction attributes on the b.methods of Spring Bean class.
- In Declarative Transaction management the Transaction will be committed when no exception is raised in the Business method. otherwise the Transaction will be rolled back.
 - ↳ RuntimeException
- (*) For example appn on Declarative Flat, Local Transaction mgmt refer appn ① of the Pgno's 130 & 131

→ Declarative Tx mgmt by using annotations :-

//Demo.java Same as page no: 130 demo.java

//DemoBean.java Same as 130 DemoBean.java but modify following things.

a) add import org.springframework.transaction.annotation.*;

b) add @Transactional (propagation = Propagation.REQUIRED) on bm1() b.method
↳ Tx attribute

//Demo.xml

a) must be schema base including "spring-beans.xsd, spring-tx.xsd, spring-aop.xsd"

b) place line no 42 to 63 of page no: 130 as it is here.

c) add following code at the end of (b) step

```
<tx:annotation-driven transaction-manager="dts" />
```

ClientApp.java :-

same as the client app. java of page no: 131

jarfiles in CLASSPATH :-

Spring.jar, Commons-logging.jar (Spring 2.5 env...)

> javac *.java

> java ClientApp

Other Tx attributes while working with `@Transactional`
Propagation.REQUIRES_NEW, Propagation.REQUIRE,
Propagation.SUPPORTS, Propagation.NOT_SUPPORTED, Propagation.MANDATORY,
Propagation.NEVER, Propagation.NESTED (to enable nested Tx's)

17/04/2013

→ To apply Spring based Transactions on SpringORM module based hibernate persistence logic use `org.sf.orm.hibernate3.HibernateTransactionManager`

→ For example application on programatic, local, Flat Tx management on SpringORM module based Hibernate Persistence Logic refer appn ① of the page no's ④ to ⑦ of Supplementary handout given on 17/04/2013

→ For example appn on Declarative, local, Flat Tx's on SpringORM module based persistence Logic refer appn ② of the page no's ① to ③ Supplementary handout given on 17/04/2013

* Global / Distributed Transactions *

18/04/2013

If multiple Databases are involved in Transaction management then it is called Global (or) Distributed Tx's. for this we need Jta Transaction manager and its implementation.

Atomikos supplies special jdbc drivers and JtaTransactionManager implementation to add Distributed Tx mgmt support on Spring apps.

u can download related resources from www.atomikos.com website.

Transfer money operation b/w two diff a/c's of two diff banks need Global Transaction Support.

→ mysql DB slw

Account 2 (db table)

aceno	holdname	bal
102	rajat	9000

→ Oracle DB slw

Account 1 (db table)

aceno	holdname	bal
101	raja	8000

→ we need Distributed Tx mgmt support to perform transfer money operations on the above two accounts.....

*) for example app on Declarative and Distributed Tx mgmt on Transfer money operation refer app (given on) available in 18/04/2013 handout

* Security *

Security

1. Network Level Security

2. Application Level Security

Network Level

→ To protect the data that is travelling over the network

→ Application developers are not responsible for this. Network admins will take care of this through Firewalls and other

Application Level

→ Authentication + Authorization

→ programmers are responsible for this operation.

→ Checking the Identity of a User is called Authentication.

→ checking the AccessPermissions of a User on resources is called Authorization.

→ It is always recommended to enable Security Services on application in Declarative mode. ~~and~~ we can use Acegi framework (or) Spring Security f/w to enable the declarative mode of security on Spring apps.

(*) For example appn on aspectJ programming refer the appns given in 20/04/2013 17/04/2013 handout.

→ AspectJ Annotations:-

- @Aspect → To make the java class as the Advice class
- @Pointcut → To point to 8 methods of Spring Bean class.
- @Before → To make the java method as the Before advice.
- @AfterReturning → To make the java method as the After advice.
- @AfterThrowing → To make the java method as Throwing Advice.
- @Around → To make the java method as the around advice.

② For above annotations based AspectJ programming refer the following example.

TestInter.java → same as page no: 10 TestInter.java of 17/04/2013 handout.

TestBean.java → " " TestBean.java of " "

MyAspect.java

```
import org.aspectj.lang.annotation.*;
```

```
@Aspect
```

```
public class MyAspect
```

```
{
```

```
@Pointcut("execution(* TestInter.*(..))") } → pointcut pointing to all the 8 methods of Spring Bean
```

```
public void testInterOK();
```

```
}
```

```
@Before("testInterOK()")
```

```
public void beforeMethod()
```

```
{
    S.o.p("iam before advice");
```

```
}
```

```
@AfterReturning("testInterOK()")
```

```
public void afterReturningMethod()
```

```
{
    S.o.p("iam after returning advice");
```

```
}
```

jar files in CLASSPATH: aspectjweaver-1.6.0.jar, spring-jar
 > javacx.jar, aspectjweaver.jar, commons-logging.jar
 > java client

Spring.xml

```
<beans ... .. include spring-beans-2.5.xsd
... .. spring-app-2.5.xsd">
```

```
<bean id="id1" class="TestBean"/>
```

```
<bean id="id2" class="MyAspect"/>
```

```
<aop:aspectj-autoproxy/>
```

```
</beans>
```

Client.java

Same as client.java of
 page no: 10 belonging to April 15th handout.

Spring WebModule Part-II

Web Framework also provide abstraction layer on core Servlet, JSP technologies and simplifies the process of developing MVC architecture based web apps.

Struts → From Apache Foundation

JSP → From Sun MS (Oracle Corp)

Webwork → From Open Symphony

Spring Web MVC / Spring MVC / Spring Web Flow → From Interface 2.1

What is the diff b/w Struts 1.x and Spring MVC?

Struts 1.x

- ① resources are API dependent
- ② does not support Annotation based programming
- ③ does not allow to use other than HTML, JSP technologies in view layer
- ④ supports dependency Injection only on Form beans
- ⑤ Validator plugin is given to perform Form Validations

Spring MVC

- ① resources can be developed as POJO classes
- ② supports annotations based programming
- ③ allows to use Velocity, Freemarker, JSP, HTML technologies in view layer.
- ④ supports dependency Injection on every Java resource.
- ⑤ uses Spring's built-in facilities to perform Form Validations

Terminology matching b/w Struts 1.x and Spring MVC

Terminology

Controller Servlet

Struts 1.x

ActionServlet

FormBean

Action class

Struts cfg file

<Any filename>.xml

ActionMapping

ActionForward

(to configure Result pages)

Spring MVC

DispatcherServlet

command class

Command Controller class

Spring cfg file

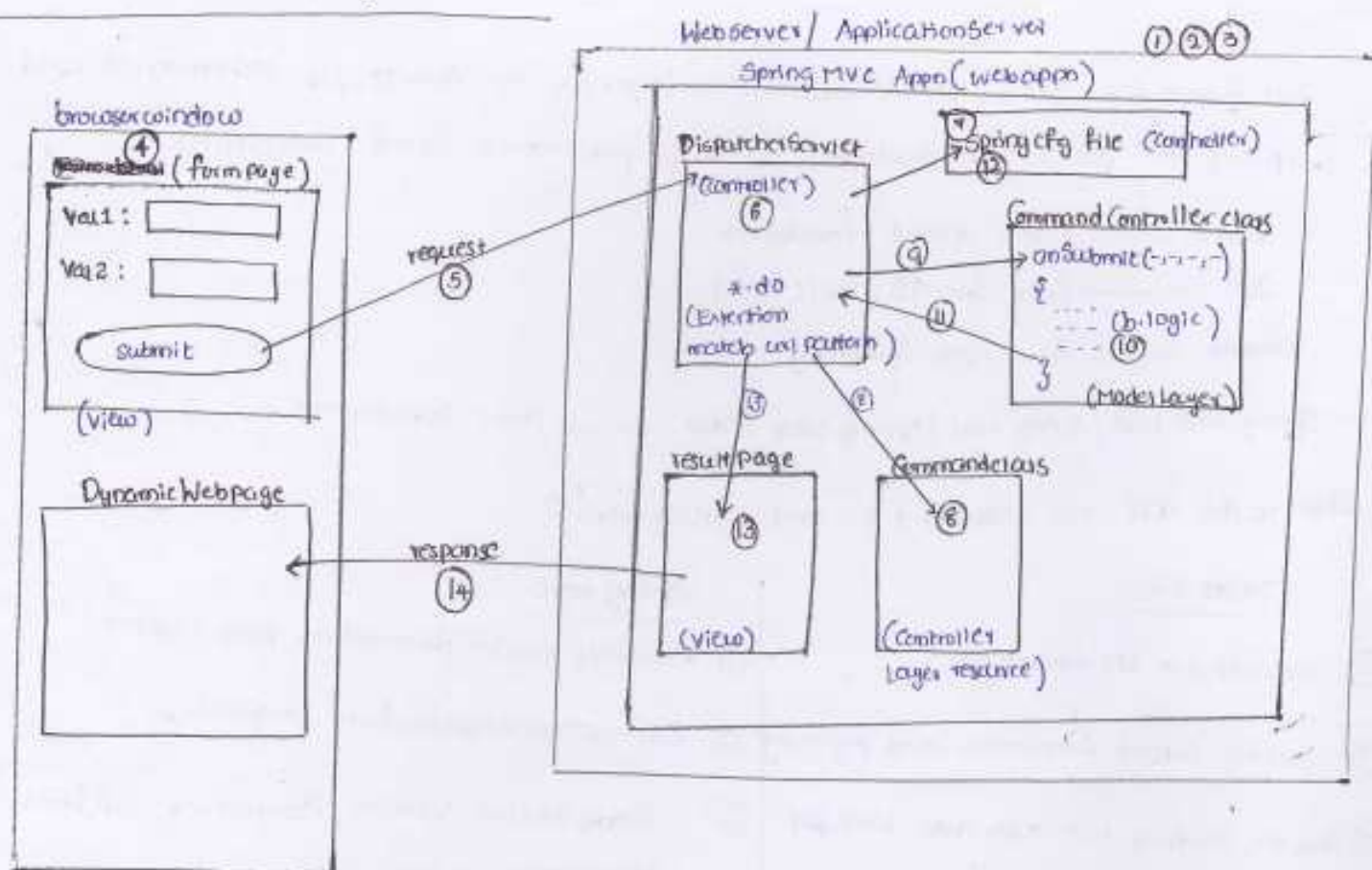
<DispatcherServlet logicalname> -
servlet.xml

url handler

View Resolvers and views

Model/View object

Flow of execution in Spring MVC appn :-



W.r to the diagram :-

- ① programmer deploys Spring MVC web appn in webserver (or) appn server.
- ② because of load-on-startup enabled on the DispatcherServlet it will be instantiated either during Server startup or during the deployment of webappn.
- ③ This DispatcherServlet activates web appn Context container by taking ~~DispatcherServlet~~ `<DispatcherServlet logical name> - servlet.xml` as the Spring Configuration file. In this process all the beans configured in Spring Configuration file will be pre-instantiated.
- ④ ^{launches} Enduser Form page on the browser window and submits the request to webappn.
- ⑤ DispatcherServlet traps and takes the request
- ⑥ DispatcherServlet uses Uri-handler Configuration to know Command Controller class, Command class that are required to process the request.
- ⑦ DispatcherServlet writes the received Form data to Command class object.
- ⑧ DispatcherServlet calls certain B-method of Command Controller class
- ⑨ & ⑩ This B-method will process the request and returns Model And View class object to DispatcherServlet.
- ⑪ DispatcherServlet uses ^{the} Views and ViewResolvers ~~and~~ configured in Spring Cfg file to decide the result pages.
- ⑫ DispatcherServlet passes the Controller to Result page.
- ⑬ The presentation Logic of Result page formats the results and sends response to browser window.

- Command class is POJO class, and Java bean
- Command class properties need not to match with FormComponent names.
- Command class properties must be bound with FormComponents by using `<spring:bind>` tag.
- Command Controller class is Spring-API dependent. This class must implement `org.springframework.web.servlet.mvc.Controller (Interface)`

the Spring Configuration file of Spring webmvc appn contains the following cfgs.

① Uri handlers

(to link ~~incoming~~ requests with Command controller classes)

② Command controller classes & Command classes.

③ View Resolvers.

View Resolver :- This configuration lets the Spring flow to specify the technology that is required to develop ~~that is~~ resources of view layer. This ViewResolver class name varies based on the view layer technology we use.

Technology	viewResolver class name
JSP	org.springframework.web.servlet.view.InternalResourceViewResolver (default)
velocity	org.springframework.web.servlet.view.velocity.VelocityViewResolver
Freemarker	org.springframework.web.servlet.view.freemarker.FreeMarkerViewResolver
tiles	org.springframework.web.servlet.view.tiles.TilesConfigurer
xslt (xml stylesheets language for transformation)	org.springframework.web.servlet.view.xslt.XsltViewResolver
Jasper reports	org.springframework.web.servlet.view.jasperreports.JasperReportsViewResolver

UriHandlers :-

- these are the classes configured in Spring cfg file to link the client generated incoming request with Command Controller classes based on their Request Uri's. There are two important uri handlers. *SimpleUrlHandler* (recommended to use)

① org.springframework.web.servlet.handler.SimpleUrlHandlerMapping :- This class allows to use bean ids, bean names and even regular expressions to link ~~from~~ from uris (request uris) with Command Controller classes.

② org.springframework.web.servlet.handler.BeanNameUrlHandlerMapping :- This class allows to use bean names (that starts with "/" symbol) to link from uris (request uris) with Command Controller classes.

Command Controllers :- There is a possibility to develop different types of Command Controller classes.

- ① org.sf.web.servlet.mvc. SimpleFormController :- useful to handle single submit button form page, here the FormPage must be launch under the control of Controller Servlet. gives onSubmit(-) method to process the request.
- ② org.sf.web.servlet.mvc. AbstractFormController :- useful to handle single submit button form page, here the form page can be launched directly. gives handleRequest() method to process the request.
- ③ org.sf.web.servlet.mvc. AbstractWizardFormController :-
- ④ org.sf.web.servlet.mvc. CancellableFormController :- Same as SimpleFormController but also provides onCancel(-) to process the request coming from Cancel button (reset btn).
- ⑤ org.sf.web.servlet.mvc. MultiActionController :-
Useful to handle the multiple request coming from the multiple Submit buttons of ^{the} FormPage.

Some points about SimpleFormController :-

22/04/2013

- When this class based CommandController class gets "GET" method based ^{request} then the form page will be launched.
- When this class based CommandController class gets "POST" method based ^{request} then the request will be processed and result will be generated.

The important properties are :-

- a) SessionForm → to specify the scope of Command class obj
 - b) formView → to specify the form page.
 - c) successView → to specify the result page.
 - d) CommandName → to specify the Command class logical name.
 - e) CommandClass → to specify the Command class name
- and etc...

above diagram w.r.to diagram

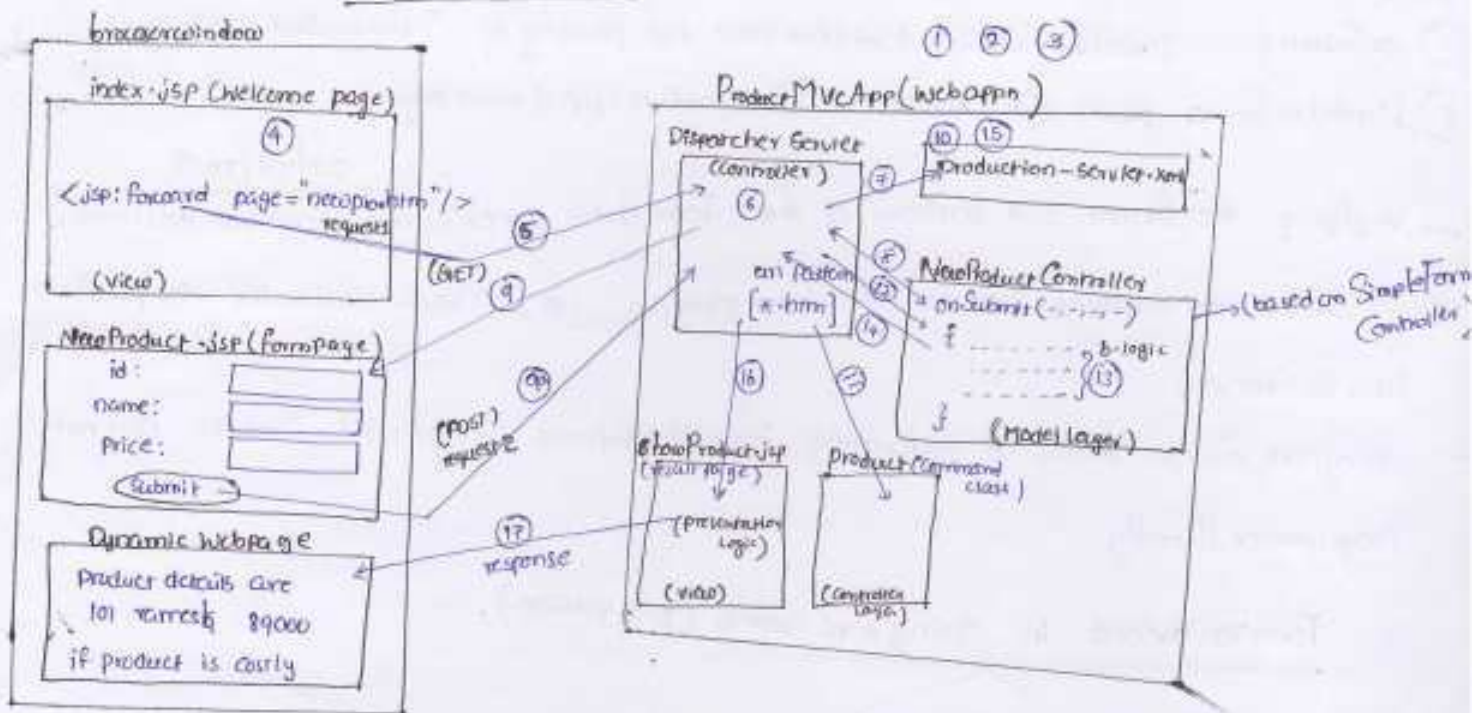
⑤ Welcome page generates "GET" method based implicit request

⑥ & ⑦ The Dispatcher Servlet Traps & takes the request.

⑧ Dispatcher Servlet links the request with Command Controller class. Since the request method is "GET" the Command Controller class makes Dispatcher Servlet to launch the FormPage.

⑨ FormPage generates "POST" method based request. ⑩ Dispatcher Servlet Traps the request and links the request with CommandController class. Since the request method is "POST" the CommandController class becomes ready to process the request.

Example Application



- (11) Dispatcher Servlet writes the received form data to Command class object.
- (12) Dispatcher Servlet calls `onSubmit()` method on Command Controller class obj.
- (*) For the above code base example appn refer apn (24) of the pg no's (78) (79)

The Flow of execution of Appn (24) page no: (78) & (79) :-

- (A) Same as step 21/04/2013 Diagram
- (B) same as step 2 of 21/04/2013 Diagram
- (C) Same as Steps of 21/04/2013 Diagram
- (D) Enduser gives request to Webapp (Spring MVC). in this process the welcomepage Index.jsp will execute and this welcome page generates implicit request to Webapp having "newprod.htm" in the request URL. (2658 refer line no)
- (E) DispatcherServlet traps and takes the request (2695-2705).
- (F) The SimpleUrlHandlerMapping class links the request with CommandController class (NewProductController) (2720, 2725)
- (G) Since Command Controller class type is SimpleForm Controller and the request method is "GET" so the form page gathered from "formView" property ("newproduct.jsp") will be displayed on the browser window through DispatcherServlet. (ref: 2729 & 2659)
- (H) Enduser fills up the form page and submits the request (ref: 2661)
- (I) DispatcherServlet traps and takes the request (2695-2705)
- (J) Same as (F)
- (K) The Command Controller class (NewProductController) becomes ready to process the request because the request method is "POST"
- (L) Dispatcher Servlet writes the received form data of form page to Command Controller class obj. (use `setXXX()` methods (ref: 2747, 44, 51))

- (11) DispatcherServlet calls `onSubmit(...)` method of `CommandController` class obj. (2761-2772)
- (12) `onSubmit(...)` method returns `ModelAndView` obj pointing to `"shopproduct.jsp"` (ref: 2770, 2780).
- (13) DispatcherServlet passes the control to `Shopproduct.jsp` (result page).

23/04/2013

- Verifying the format and pattern of the Form data comes under FormValidations.
- In struts applications we use Validator plug-in (or) XWork Validator to perform FormValidations.
- Spring MVC app's process of performing FormValidations is little bit complex. And not programmer friendly.

Form Validations in Spring MVC apps (2.5 version)

1. clients: de

Only programmatic Formvalidations core possible using javascript

2. Server side

→ only programmatic form validations are possible using java code

→ we can use various methods of ValidatorUtils class for simplifying the process of form validation.

- Spring allows to develop FormValidation logics in separate java class and this java class can be used in all kinds of Spring apps including Spring MVC apps. This class must implement `org.springframework.validation.Validator (I)` and must provide implementation for `Supports`, `Validate()` methods. While writing logics in `Validate()` method we can take the `Support` & `rejectXXX(-)` methods of `ValidationUtils` class.

⑧ Procedure to develop the Spring MVC appn having the Support of Serverside Form Validation

Step-1:- Take a java class implementing `org.springframework.validation.Validator(I)` and implement the FormValidation logic.

refer ProductValidator.java file of page no: (89) appn no: (25).

Step-II:- Specify the above class in Command Controller class. Configuration of Spring Configuration file.

refer 2868 entries of progeny: (81)

refer 2868 entries of page no: (81)

Step-III write following logic in the Form page by replacing the code of 2814 to 2818 of page no (80)

→ logical name of Command & condition class

<spring:hasBindError name="^{logical name of Command/destination class}P">

 cc:forEach var="^{iterates}c" items="^{contains all the errors messages (represents all form validation errors)}{errors.allErrors}">
 <!-- validation error in each iteration -->

<spring:message text="{0} default Message" /> → It is foreach loop so, one by one validation errors displayed

</c:forEach>

<spring:hasBindErrors>

24/04/2013

important annotations of Spring MVC (related to spring 3.x version)

- ① @Controller → makes the java class as command controller
- ② @RequestMapping → maps the input request url with CommandController class.
- ③ @ModelAttribute → keeps the parameter of given b-method in the ModelAttribute to use in View layer.
- ④ @SessionAttributes → creates Session attribute in web resource prgs.

(*) For Annotations based Spring Web MVC example appn refer the appn ① of the page no's ① & ②. 24/04/2013

→ If you want to use special Springbeans as View layer resources and to make Spring MVC appns recognizing them we need to work with Special View Resolver called BeanNameViewResolver org.springframework.web.servlet.view.BeanNameViewResolver. To generate Results in the form of Pdf document, X-cel sheets we don't have ViewResolvers. so, we write those logics in special Springbeans and we make them as View layer resource.

(*) For the example appn on the above discussion the appn ③ of the page no's ⑤⑥⑦ of 24/04/2013 handout.

→ In Spring MVC to handle the situation of multiple submit buttons of 25/04/2013

a form page we must use the CommandController class of type "MultiActionController" and we should also use org.springframework.web.servlet.mvc.multiaction.ParameterMethodNameResolver

to link the Form page generated request with methods of CommandController class.

(*) For example appn on MultiActionController refer the application ② of 24/04/2013 handout.

Spring-webservices

→ It provides the abstraction layer on JAX-RPC, JAX-WS webservice. In this process the Spring Container itself provides environment to execute webservice Component. that means there is no need of separate webservice.

→ Spring 3.x Supplies two classes to expose Java class as webservice Component and to consume webservice being from client appn.

`org.springframework.remoting.jaxws.SimpleJaxWsServiceExporter` can export given Java Class Object as webservice Component.

`org.springframework.remoting.jaxws.JaxWsPortProxyFactoryBean` can be used to consume webservice from client application.

⊗ for example appn on Spring 3.x based Spring webservice refer appn ① on 24/04/2013 handout